Essence C

A new programming language to advance link-time optimizations by faciliating the continuum of static and dynamic modules.

David Callanan

21444104

Final Year Project – 2026

B.Sc. Single Honours in Computational Thinking

Department of Computer Science

Maynooth University

Maynooth, Co. Kildare

Ireland

A thesis submitted in partial fulfilment of the requirements for the

B.Sc. Single Honours in Computational Thinking.

Supervisor: Dr. Phil Maguire

Contents

Declaration

I hereby certify that this material, which I now submit for assessment on the program of study as part of the B.Sc. Computational Thinking qualification, is entirely my own work and has not been taken from the work of others – save and to the extent that such work has been cited and acknowledged within the text of my work.

I hereby acknowledge and accept that this thesis may be distributed to future final year students, as an example of the standard expected of final year projects.

Signed: _____ Date: _____

Acknowledgements

A massive thanks to Dr. Phil Maguire, friends, and peers for inflicting levels of motivation I might never reciprocate.

Some brief gratitude to Anthropic for their powerful GenAI models, though other models were preferred due to cost reasons.

Abstract

(max 250)

The logic of computer software is primarily expressed using programming languages, but the balance between versatile and efficient programming languages remains a major challenge. Changes to the execution environment of a program can impact performance, or worse, cause incompatibilities that require major refactoring. Developers often maintain multiple branches of a codebase to keep performance to acceptable levels. One particular difference between execution environments is whether individual modules are statically linked at compile-time or dynamically resolved at runtime. In this research project, we propose a prototype programming language called "Essence C" that better supports the continuum of static and dynamic modules, by allowing the programmer to write code *once*, that is agnostic to the linking strategy. By taking advantage of LLVM's intermediate representation, it is possible to advance the state-of-the-art of link-time optimizations across module boundaries.

Chapter 1 – Introduction (930)

1.1 Overview

This project is concerned with the research and development of a novel programming language, Essence C, that facilitates the continuum of static and dynamic modules in low-level software development. The prototype henceforth discussed is accomplished by designing unique language features and implementing bespoke optimizations pursuant to these features. Kernel development is used as a case study throughout the research phase and evaluation phase alike.

1.2 Motivation

Software development is becoming very popular, with graduates now unable to find jobs due to a massive spike in educational resources allocated to this field. With the use of LLM agents gaining traction, keeping a codebase clean may be harder than ever before. There is now serious competition in programming language design due to two reasons. Firstly, an LLM's ability to navigate a codebase effectively depends on the structure imposed directly by the language, given the reduced structural input from the programmer themself. Techniques that minimize duplicated effort in a codebase is expected to minimize context rot and reduce the agentic workflow bills. Secondly, agents are at risk of optimizing their development in highly trained-on environments (e.g. JavaScript, web-based, and *React.js* environments), leaving less-prevalent low-level languages behind unless the agent can learn from on-the-fly tool calls. Highly structured languages with a superb type system, high cohesion of language concepts, and excellent documentation may make up for the lack of training. An effective module system alongside other novel language features has the potential to transform the way humans and agents reason about their software.

1.3 Methodology

Thesis structure, and a resesarch strategy.

To address the objectives of this research, a design-science methodology is employed with a focus on continued interation and evaluation of "Essence C". This approach was selected because the project is concerned with the implementation of a novel programming language that advances the flexibility and performance of modules in software development, and thus requires a design and implementation process that is informed by ongoing evaluation.

The structure of this thesis is organized to reflect this methodology. Chapter 2 reviews existing literature.

1.4 Success Criteria

Success is evidenced by the following deliverables:

(1) A functional prototype compiler that implements the designed module system;

(2) Documentation of the module system and other core language features;

(3) A simple IDE extension that provides syntax highlighting to the programmer;

(4) A body of research into the continuum of static and dynamic modules in software development, with kernel development as a case study;

(5) An implementation of link-time optimization techniques that leverage the designed module system;

(6) An evaluation of the effectiveness of these optimizations through the benchmarking of sample programs;

(7) A monorepo containing extensive git history, demonstrating ongoing development progress;

(8) A final written report (that you are reading) detailing the research, design, implementation and evaluation of the project.

1.5 Use of Generative AI

Two notable GenAI tools have been used to assist with the research and development of this project.

(1) The first is t3.chat, a web application that consolidates numerous large language models into a streamlined chat interface [1]. The user can experiment with different models to understand which models are best suited for the task at hand. Both research and programming tasks have benefited from the use of this tool. The tool was primarily used to steer the work in the right direction, and to identify the concepts, libraries and resources through which further research could be conducted. For transparency, all LLM prompts have been included in **Appendix C**. This appendix was generated programatically by using the JSON export facility and filtering relevant chats.

(2) The second is the tab-complete feature of GitHub Copilot. This tool has encouraged rapid development and iteration through its behaving as a glorified autocomplete. From experience, the tool's context is severely limited, forcing the developer to have genuine knowledge of the codebase and its architecture. This is beneficial for projects that require significant unique input from the developer, in contrast to rehash projects that benefit more directly from LLM assistance. The latest iteration of this feature allows completions for lines that are away from the cursor, which has been a game-changer.

Some experimentation with GitHub Copilot agent mode was conducted, with notable issues identified. Firstly, the development speed was impeded by slow model response times, lack of funds, and internet connectivity issues. Secondly, there was a great difficulty in referencing contributions made by this tool due to (1) tool calls that modify the codebase directly and (2) significant implicit pollution of the context window as the agent navigates the respository. As such, usage of this tool was suspended.

Chapter 2 – Technical Background (1200)

2.1 Literature Review

2.1.1 Static vs dynamic dispatch

Methods of class instances (or equivalent constructs) dispatch calls either statically or dynamically. When the exact implementation of a method is known at compile-time, static dispatch is typically used, involving a direct assembly call to the method. Since the method is known at compile-time, the compiler can use an immediate addressing, which is most efficient. In contrast, when the exact implementation of a method is not decided until runtime (or if the compiler cannot prove otherwise), dynamic dispatch is used, which involves an indirect call through a vtable or similar structure.

2.1.2 Static vs dynamic linking

There is much overlap in the literature between dispatch and linking, but linking focuses on larger units of code, often referred to as modules or libraries, as opposed to class instances. This concept exists in C where there is no inbuilt notion of static vs dynamic dispatch.

When the exact source code (implementation) of a module is known at compile-time, static linking is typically used, involving the direct inclusion of the module's code into the final executable. Then, the compiler can use immediate addressing to invoke functions or access data from this module. In contrast, when the exact source code of a module is not decided until runtime, dynamic linking is used which typically requires a Procedure Linkage Table (PLT) and Global Offset Table (GOT).

However, various techniques are already employed to optimize dynamic linking:

(1) Some architectures use the "relaxation" technique to replace indirection at runtime by modifying the assembly instructions.

(2) PLT Rewriting provided by compilers.

(3) Bespoke replacement (e.g. linux kernel static_call).

However, two primary concerns of interest are still evident:

(1) There is always some level of pointer indirection that cannot be avoided (nvm);

(2) Various techniques such as inlining become impossible due to this indirection.

Terrible PIMPLE idiom. Headers of headers.

2.1.3 Module loading in the Linux kernel

It was found that the potential inefficiencies of dynamically loaded modules were already considered by the Linux kernel developers, and boutique infrastructure was developed to partially mitigate this.

The Linux kernel benefits from their "static_call" system, which uses a combination of C macros, custom compilation/linking steps, and runtime logic to patch function calls to be more efficient. Any calls that may need to be dynamically swapped at runtime can engage with the "static_call" system to improve performance and automate the repetetive assembly logic. The process is summarized below.

Macro system

Each static-call function is implemented using the "DEFINE_STATIC_CALL" macro, which takes in (1) a handle name, and (2) a pointer to the initial implementation for this handle. This macro is used at the top-level. Then, the programmer can use the "static_call_update" macro at any point, taking in the same parameters, to alter the implementation for a particular handle at runtime. Alas, the programmer can use the "static_call" macro to invoke the handle like a traditional function. Here is some example pseudo-code:

```
void implementation_a(int value) { ... }
void implementation_b(int value) { ... }

DEFINE_STATIC_CALL(foo, implementation_a);

void main() {
```

```
  static_call(foo)(27);
  static_call_update(foo, implementation_b);
  static_call(foo)(27);
}
```

The “static_call” macro also uses the inline assembly “.pushsection” and “.popsection” directives to temporarily relocate the code to a unified object section that can be picked up in the custom compilation step.

A crucial takeaway is that the lack of direct language support has forced the need for a custom macro setup. Understandably, programmers in different codebases may not be so adamant to commit resources to such a setup in the same way the Linux kernel developers have.

Custom compilation step (objtool)

The kernel needs to remember where the call sites are when the implementation is swapped at runtime. To achieve this, the kernel includes an extra build step that investigates the aforementioned object section to locate all relevant call sites. It then builds up a table that can be inspected at runtime to ensure no call sites are missed. There are many more intricacies that are not mentioned and have not been studied in too much detail.

If you explore the source code for the Linux kernel, you will come across the static_call_types.h file, containing the following:

```
/*
 * The static call site table needs to be created by external tooling (objtool
 * or a compiler plugin).
 */
struct static_call_site {
    s32 addr;
    s32 key;
};
```

Trampoline solution

The easy solution to alter the implementation of a function at runtime is to use a trampoline (an extra dynamic jump instruction that is resolved at runtime). This is slow and is merely a backup option.

Insert diagram.

Inlining solution

Great news, inlining refers to the trampoline call, not inlining the function body itself.

(Well not so great news, but at least there is more merit to this project).

Spectre

Since the infamous Spectre exploit in 20XX,

2.1.4 Link-time optimizations

Traditionally, many optimizations were performed at the language level, before the program was compiled to object files. For instance, inlining could not be performed at link-time because there was insufficient information in the object files to reason about the semantics of the program. While dedicated link-time optimizations were introduced, their scope was limited for this reason.

However, LLVM IR sits in between the level of abstraction of source code and object files, and maintains higher-level semantics. By keeping the program in this intermediate representation longer (and treating this as modern object files), recent developments have made link-time optimizations more powerful, allowing, for instance, for inlining across module boundaries.

2.1.5 Inlining and IR-level linking

2.2 Technical Material

2.2.1 LLVM Intermediate Representation

LLVM was found to be a practical compilation target for low-level programming languages due to (1) its target-independent code generation [2] and (2) its integration capabilities with existing programming languages thanks to ABI interoperability (including some recent improvements [3]). Furthermore, the extensive optimization passes shared amongst languages targetting LLVM [4] should make *Essence C* directly comparable with existing languages, allowing benchmarks to be solely influenced by the additional optimization techniques employed. For instance, C can be compiled with LLVM via the popular Clang compiler [5]. It was also noticed that LLVM has experimental support for inline assembly [6], preventing the layer of indirection of a separate compilation unit (useful in kernel

development [7]). Additionally, LLVM benefits greatly from intermodular optimizations at link-time [8], because its intermediate representation (IR) preserves the necessary higher-level constructs. Critically, the LLVM API exposes a powerful interface to programatically manipulate existing LLVM IR files [9] [10]. It follows that inlined compatibility wrappers can be generated to minimize optimization barriers between compilation units originating from languages other than *Essence C*.

Chapter 3 – The Problem (900)

3.1 Indirect Calls

The core of the problem is removing indirection.

3.2 Functional Requirements

3.3 Non-Functional Requirements

Chapter 4 – The Solution (2110)

4.1 Design Elements

4.1.1 Choice of Language

The languages selected for this project were JavaScript and C++. While JavaScript has been my go-to language before classes were even a thing, the primary LLVM API is sadly exposed through C++. Other language bindings exist, but do not appear mature enough. It would not have been smart to gamble a project of this calibre on the stability of such bindings. JavaScript was still suitable for the parsing phase, as my pre-existing parsing library (called *blurp*) was already written in JavaScript. The speed of compilation is mostly irrelevant, as this does not impact the performance of the compiled artifacts. Thus minimal effort was wasted on such a triviality.

4.1.2 Unification of Modules and Classes

Languages have historically offered various features like classes, modules, packages, and so on, that have much overlap in their functionality. One of the core design decisions was to unify these elements into a single system that consists of a smaller unit called a "map" (which bares some resemblence with Rust- and Go-like structs), and a larger organizational unit called a "module". Most importantly, every module still uses a map for instance-management purposes, and so the same capabilities are available whether or not a program is split into more modules or not. This differs from languages like C, where the choice to have separate compilation units has an impact on the final program [ref]. The decision for this unification was inspired by Rust's philosophy of "zero-cost abstractions" [ref], an acknowledgement that organizational differences in a program that aid with design and iteration should not impact the performance of the final artifact.

4.1.3 Map-Enum Equivalence

Three features in programming languages were found to share an ermegent goal. These are structs (or classes), functions (or methods), and enums (specifically, tagged unions).

Essence C unifies structs and functions into a single feature called a "map". This takes inspiration from Pure Mathematics, where the pattern of notational definitions is often used for multiple purposes – functions ($f(x)$), operations ($x+y$), subscripts (x_n), accessors ($\star_G$). Little special treatment is given to these different cases, as all could be denoted with function-like syntax ($f(x), +(x, y), x(n), \star(G)$).

In Essence C, there are no struct accessors like ".first_name". Rather, a struct is a "map" that takes in a symbol (e.g. ":first_name") as its only parameter and returns the member from its internal storage. These semantics are not so bizarre, as the technique to store data alongside functions (often known as closures) is already commonplace. When a "map" has no virtual methods, its behaving as a struct results in the same low-level data representation.

Insert beautiful diagram here.

Furthermore, tuples are syntactic sugar for maps, adding positional syntax. One notable outcome is that there is no concept of multiple function parameters; instead, there is a single parameter which reuses the existing tuple syntax to achieve multiple parameters. It's maps all the way down.

Next, Essence C unifies maps and enums via the so-called Map-Enum Equivalence technique.

Recall Rust-like enums (tagged unions) where a symbol (:foo_bar) is associated with some data (:foo_bar("hello")).

A syntactical ambiguirty arises. If we write map:foo_bar("hello"), are we calling the map with a symbol (a bare enum) and then calling the resulting map with ("hello"), or, are we calling the map with a fully populated enum? The solution is to unify these two approaches, by demanding that all maps obey map-enum equivalence, that is, that calling a map with a symbol and then another map, is equivalent to calling the map with the fully populated enum. This bears partial resemblance with currying. What's amazing now is that populated enums are really just pre-curried symbols. The vast majority of code written by the programmer now comes from maps and symbols.

4.1.4 Body-Map Unification

4.1.5 Type-Value Unification

Old-fashioned languages like C treat types and values as entirely distinct.

In the traditional sense, a type offers two functions:

- It details the runtime representation of the data, i.e. the possible runtime values that may be encoded and how many bits of data are needed.
- It provides a constraint to the programmer themself, preventing the programmer from using values that the code is not built for.

It turns out that it may be appropriate for these two intents to diverge. This results in two separate notions of a type:

- **Logical Type:** A set of allowable values. A constraint of what theoretical values are permissible, i.e. subject the programmer's intended business logic.
- **Encoding Type:** A bijection between a set of values and a set of bit patterns. This allows types to be designed and abstracted independent of the encoding used.

In this sense, a value is just a singleton logical type.

encoding value vs theoretical value.

both the concept unified, and then the syntax is unified too.

4.1.6 Additional benefit of inlining: interacting with existing languages without indirection.

4.2 Picking of overall solution

4.3 Design

4.3.1 Associativity

In most programming languages, addition and subtraction are treated as unary operations in an additive batch, whereras operations like multiplication and division are instead

treated as binary operations in a multiplicative tree. However, it turns out that any associative operation (such as multiplication) can be written in the former form, where operations in the same family (such as division) control the unary effect via an operator that prefixes the operand. As such, a consistent syntax was employed in this language, allowing both the unary and binary approach for all associative operations and their respective intra-family operations. This includes the following families:

- Addition and subtraction (like existing languages);
- Multiplication and division;
- Logical AND;
- Logical OR;

The syntax was designed to follow the crystal/pistol pattern, some terminology devised to mean the following:

- Crystal: A unary batch of operator/operand pairs (an extension of how existing languages handle addition and subtraction).
- Pistol: A binary tree of operations (like how existing languages handle multiplication and division).

For precedence reasons, we have eight separate rules: OR crystal -> OR pistol -> AND crystal -> AND pistol -> additive crystal -> additive pistol -> multiplicative crystal -> multiplicative pistol, with each embedding the next in the hierarchy. Those further to the right of the chain enjoy higher precedence.

More details on this pattern can be gathered by viewing my article on **boundly associative operations**, my article on **making complicated if statements cleaner**, and my YouTube video on **rethinking operation order**.

4.3.2 Module Resolution

Some (debatably legacy) approaches to module resolution are:

- Use the filesystem alongside symlink functionality to ensure access to the intended modules. This is effective for source code, because, apart from version upgrades, a single build of a project will not have to switch between versions of source code. It may deploy multiple versoins simultaneously, but types to do with source code

are fixed. Many developers consider using the filesystem to be a lot of hassle, but this language takes the stance that if a built-in feature that all developers use (a filesystem-based codebase) can handle module resolution for types, then why re-invent the wheel. Thus, Essence C promotes the symlink appraoch for this purpose. This has two consequnences:

(1) Language-level Ownership: A module takes ownership of type dependencies. While a module may elect to use an external package management solution, symlinks, etc. to resolve the interfaces, by the time the dependencies reach the compiler, the compiler treats the interface as under the ownership of that module. This is favourable to those who follow the dependency inversion principle, but may feel foreign to those who delegate type ownership to external systems.

(2) Monorepos or "Physical" Ownership: Those who like to use monorepos, or like to own their dependencies physically (that is, not just within the language, but as a business, as a larger business function or princoiple), they will have no trouble choosing to bake copies of dependencies into their repo. This is a huge reason that many developers (including myself) switched over to the “pnpm” package manager for JavaScript projects.

- Paths: Compare the common use of paths to the Plan 9 operating system. Using paths is arguably messy as it is easy to pollute the global namespace.

One must distinguish instance resolution, source code resolution, and type resolution.

As Essence C wants serious static analysis of modules, passing instances is the only solution for the instance part. Spaghetti is avoided by using the factory approach as detailed in the module system.

However, the language takes a balanced approach when it comes to soruce code and type resolution, allowing you to use internal languages features to control the propogatoin of such dependencies. This allows a library to put the burden of populatoin on the user of the library. While this transfers a certain burden of ownership, is unlocks a flexibility that allows developers to not be inmpacted by personal resolution preferences of developers. This is a trade-off that requires further contemplation.

4.3.3 Constant By Default

4.3.4 No Standard Library

With kernel development as the primary target audience for this language, the language should be freestanding and not have any external dependencies. In kernel development, it does not make sense to have built-in functions for most features, as the kernel developer is expected to implement all features themselves (printing, keyboard input, memory allocation, etc.) by communicating directly with the hardware using assembly instructions. Thus, the language only externs one function, "puts" , for logging purposes. This function is easy enough to implement early on in kernel development, but the developer can also choose to use a no-op and avoid logging, if they so please.

Rather than a standard library, first class integration with LLVM IR and the C ABI was implemented. This means the code can interact with any code written outside the language, preventing the need for the developer to migrate existing code to Essence C.

4.4 Implementation

4.4.1 Parsing

Prior to this project I had already developed a simple parsing library in JavaScript, which allows for:

(1) Parsing terminal rules using regular expressions.

(2) Forming non-terminal rules by combining other rules in the following ways, sufficient to develop any complex grammar:

    (1) "or" making rule optional

    (2) "join" sequencing rules

    (3) "multi" repeating rules zero or more times

    (4) "opt" making rule optional

(3) Map parsed data to custom structures using the "mapData" function.

4.4.2 Mutability

LLVM IR is in SSA form, where registers are immutable. One cannot reuse a register, and must instead create a new register for every mutation. Then, one takes advantage of the phi instruction to pick which register is the correct one at runtime. However, the alloca (stack allocation) model operates under the familiar mindset of mutability, as it works with raw memory addresses. Conveniently, it was observed that the standard mem2reg optimization pass does a fantastic job at converting stack allocations into SSA registers where legal, bypassing the tough development of a custom SSA mutability mechanism. Thus, variables are entirely implemented using stack allocations. This saved an estimated week of development time.

4.4.3 Tooling and Infrastructure

Parser Trace Explorer

Parsing issues were frequently encountered during the development cycle, and manual debugging became infeasible as complexity increased. Hence, a *Parser Trace Explorer* was developed, consisting of a JSON trace logger in the compiler frontend, and a separate web-app tool for exploring these logs. A simple SolidStart + Tanstack Router tech stack was selected for this tool due to existing familiarity with this stack, and because of the straightforward requirements of the tool.

Since parsers try a vast number of rules and are inherently recursive, the logs quickly exceed $500,000$ lines for small source files. To deal with this, the logs are structured as a tree, and the user of the tool is expected to navigate this tree by selecting the precise rule attempts that the parser is supposed to succeed on, thus hiding irrelevant attempts. A dedicated lexer, in hindsight, would have reduced the exponential growth of the logs, but might not have easily parsed some of the unique language features (such as semicolons acting as both terminators and comments).

4.4.4 Defensive Programming

As Essence C is designed with strict semantics, the backend has no need to recover from illegal states. As such, defensive programming is used, where the program is crashed with a useful error message whenever an invariant is violated. Such errors could originate from semantic violations in the user's code, or due to subtle bugs in the compiler itself; either way, the issues can be traced effectively. Naturally, not all subtleties can be pre-envisioned, so this was done on a best-effort basis to avoid compromising development speed.

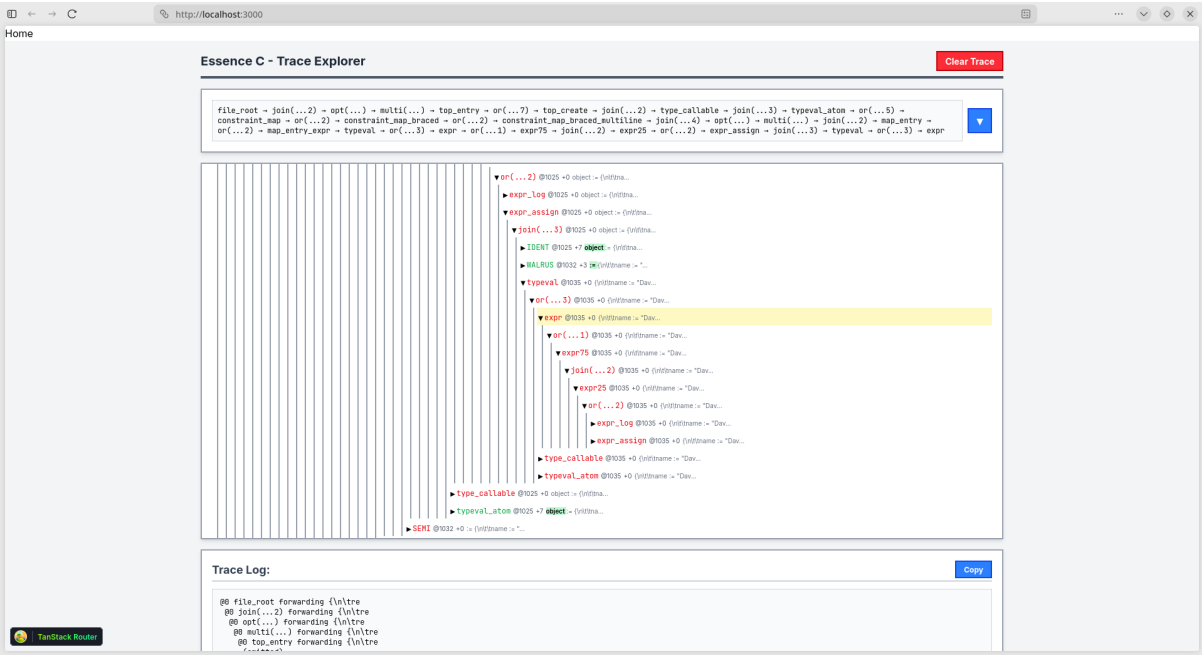

Figure 4.1: Parser Trace Explorer

4.4.5 Debugging Primitives

While the primary mechanism of performing external communication is to link with external code via the C ABI (which may be LLVM IR), debugging functionality is often preferable within the language directly in case there are issues with the interoperability layer proper. Thus, two basic logging functions were implemented, “log” and “log_d”. The former takes a null-terminated ascii string and prints this output in an implementation-specific manner. The latter takes an arbitrarily-sized value (like an integer or a byte array) and prints its raw binary representation (in 64-bit chunks), including both its hex representation, ascii representation, and decimal representation. The implementation of this functionality was chosen to be implementation-specific. In the prototype implementation, the external “puts” function was selected to implement this functionality, available in many environments. In case the environment does not support it, a dummy no-op implementation can be defined, effectively dropping any logs.

4.4.6 Bit-width Analysis

Determining the maximum bits needed to hold an integer is an important optimization. While LLVM may be able to detect certain constraints about numbers, much of the compiler information is lost at this point that a modern programming language might be interested in tracking. For example, software verification languages may keep track of exact min and max values for an integer type, which is tracked extensively as the type system

navigates the codebase. For non-integer types, an optimal type system implementation might allow the user to use formal logic to keep track of the theoretically-smallest set of runtime possibilities (a provably accurate picture of the possible values that can be assigned at runtime, with no wasted space).

Only simple bit-tracking is performed in the prototype compiler. When a hardcoded integer constant is used, the number of bits is trivially calculated using the logarithm. Addition was more tricky to implement – one must recursively track the bits needed to hold an arbitrary sum of other integers. The first attempt of taking the maximum of the operand bits and adding one (to account for a potential carry) was insufficient for more than two operands. While an additional carry bit could be added for each additional operand, this estimate would be too conservative. The correct solution was to sum the maximum value each operand could take on (interpreted as data to avoid to intricacies of signed integers), and then take the logarithm of this sum. This can achieved easily in languages that support arbitrarily sized integers out-of-the-box. However, it was easier to implement an approximate solution in C++ by using the double data type, with the final result rounded up. While this sounds bizarre and inaccurate at first glance, recall that floats are internally implemented using an exponent and mantissa. Thus, it is very similar to running the numbers through a logarithmic transformation, which would yield the final result directly. Furthermore, an epsilon value was included to deal with precision errors, favouring a more conservative bit estimate. It was not proven whether this overall approach is mathematically sound up to a particular bit count, but it can always be substituted if the need arises.

4.4.7 Build Environment

Early on in development, I was adamment to ensure a fully reproducible build environment. All building is done using the terminal, so that the build process is independent from the editor used. The JavaScript portion of the project uses *Node.js* and *pnpm*, which are easy to install on different operating systems. Build scripts are then organized into *package.json* files. However, the C++ portion, consisting of a suite of tools (*clang*, *make*, llvm, etc.), is more difficult to set up consistently. The build scripts for this portion are organized using a Makefile, and the development environment itself is setup using a Dockerfile. Some developer scripts were implemented in Bash to orchestrate the overall process between the JavaScript and C++ portions, the Docker layer, and the developer's own environment. The choice to use Bash means the developer must either use a Linux environment or a WSL environment if using Windows. A docker-in-docker strategy could theoretically have been used instead, requiring minimal platform-specific scripts (I thought I did use docker in docker, I better check), but this complexity was avoided. MacOS users

may suffer as a result. Additionally, I recently switched to NixOS on my own system, and its philosophy was applicable here. NixOS allows you to write description-based setup descriptions, which can act as reproducible development enviornments. Thus I set up a repository-level nix script for those on NixOS who want to fast-track the development environment. Thus both the build layer and host interaction layer are highly reproducible.

4.4.8 Underlying Type Tracking

4.4.9 Singletonish Optimization

A custom "singletonish" optimization was implemented that concerns itself with both the module optimizatoin goal (detecting something to be static) and the acknowledgement of the benefits of singleton types (by treating lonely values as singleton types). It also is meets the demands of the map enum equivalence, central to the design of the module system, as it allows different aspects of maps which are inherently static (e.g. the functional nature of maps) to be optimized to a form that behaves like traiditionali programming languages. This means the map-enum equivalence is not just theoretical, but acutually works.

Chapter 5 – Evaluation (930)

5.1 Manual Snapshot Testing

Throughout the development cycle, manual testing was conducted by compiling sample source files and inspecting the terminal output after running the final executable. This mirrors the snapshot testing philosophy commonly implemented in web-development projects (although accomplished in a less automated fashion here), where input/output consistency is checked before each commit. Down the road, this strategy could be extended to a full test suite that prevents all sorts of regressions. One could take this a step further by checking consistency in the compiled LLVM IR code itself, but this is typically only done when a language reaches a pivotal level of maturity; the compiler is in too much flux at this point for this technique to have any merit.

Much of the snapshot testing was performed using the language's data logging facility which prints the hexidecimal, ascii, and decimal values of arbitrarily-sized binary sequences. This allows straightforward investigation of any runtime values, whether that be integers, floats, pointers, strings, enums, and so on.

5.2 User Acceptance Testing

Four students from Maynooth University were tasked with implementing a program in the language. Each student was asked to provide positive and negative feedback of the solution.

The users were given the following code with no explanation, but were allowed to ask me questions.

Table 5.1: Feedback survey

Question	**Answer**
Student A	
What is the sample program doing?	[Answer]
What semantic or syntactic element of the sample program strikes you the most, and why?	[Answer]
Implement a program that does blah. Provide one point of positive feedback about the language.	[Answer]
Provide one harsh criticism about the language.	[Answer]
Student B	
What is the sample program doing?	[Answer]
What semantic or syntactic element of the sample program strikes you the most, and why?	[Answer]
Implement a program that does blah. Provide one point of positive feedback about the language.	[Answer]
Provide one harsh criticism about the language.	[Answer]
Student C	
What is the sample program doing?	[Answer]

Question	**Answer**
What semantic or syntactic element of the sample program strikes you the most, and why?	[Answer]
Implement a program that does blah. Provide one point of positive feedback about the language.	[Answer]
Provide one harsh criticism about the language.	[Answer]
Student D	
What is the sample program doing?	[Answer]
What semantic or syntactic element of the sample program strikes you the most, and why?	[Answer]
Implement a program that does blah. Provide one point of positive feedback about the language.	[Answer]
Provide one harsh criticism about the language.	[Answer]

5.3 GenAI Comprehension Testing

With kernel development as the core case study for this project, the ability for LLMs to understand *Essence C* was assessed by tasking an LLM agent with porting a previously-written PoC operating system kernel from *C* to *Essence C*. The original kernel was developed as part of my YouTube tutorial series [todo], and its latest source code is housed in a separate repository here. The ported kernel is included in main repository for this project.

5.4 Benchmarking

To kill two birds with one stone, benchmarking was performed on the actual kernel ported by the AI agent, instead of creating artificial benchmarks.

5.5 Validation

Chapter 6 – Conclusion (930)

Given the time constraints for this project, the evaluation only scratched the surface. One would want to spend the following time:

- 200 more hours on this.
- 200 more hours on that.

Build times consistently exceeding 90 seconds. This is dreadful when debugging and making small tweaks. A professional C++ developer would likely have experience optimizing the build process.

6.1 Contribution to state-of-the-art

- inlining - built in language constructs to avoid likes of kernel static_call - continuum of dispatch, not just static vs dynamic, but by merging the notion of class instances (and fusing classes and modules), if we detect multiple instances we can switch to an even more dynamic dispatch mechanism. - more efficient inter-language interoperability

6.2 Limitations

- The prototype compiler does not yet support cross-compilation. This means a Windows executable cannot be built from within a Linux build environment, or vice versa. Thus, everything had to be kept to Linux/WSL for now.
- The parser does not produce helpful error messages. Thus, one must either use a pair of eyes to identify syntax errors, or use the bulky *Parser Trace Explorer* to investigate more significant issues.
- With more time, the entire kernel could have been migrated from *GCC* to *Clang*. This would have offered more realistic benchmarks given *Clang*'s superior link-time

optimizations over *GCC*. Nonetheless, the effort of such a migration reminds us of the struggles developers face when writing software in C today – struggles that might be less apparent in a language like Essence C that treats LLVM IR as a first-class citizen.

References

[1] DigiVirus. "Meet T3 chat AI: Your all-in-one, super-fast AI assistant," Accessed: Nov. 16, 2025. [Online]. Available: https://digivirus.in/meet-t3-chat-ai-your-all-in-one-super-fast-ai-assistant/.

[2] The LLVM Project. "The LLVM target-independent code generator," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/CodeGenerator.html.

[3] N. Sreekumar. "GSoC 2025: Introducing an ABI lowering library," Accessed: Nov. 7, 2025. [Online]. Available: https://blog.llvm.org/posts/2025-08-25-abi-library/.

[4] The LLVM Project. "LLVM's analysis and transform passes," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/Passes.html#domtree-dominator-tree-construction.

[5] The Clang Project. "Clang: C language family frontend for LLVM," Accessed: Nov. 17, 2025. [Online]. Available: https://clang.llvm.org/.

[6] The LLVM Project. "LLVM language reference manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LangRef.html#inline-assembler-expressions.

[7] D. Roy. "GCC inline assembly and its usage in the linux kernel," Accessed: Nov. 6, 2025. [Online]. Available: https://dl.acm.org/doi/fullHtml/10.5555/3024956.3024958.

[8] The LLVM Project. "LLVM link time optimization: Design and implementation," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LinkTimeOptimization.html.

[9] The LLVM Project. "LLVM programmer's manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/ProgrammersManual.html.

[10] The LLVM Project. "Writing an LLVM pass," Accessed: Nov. 2, 2025. [Online]. Available: https://llvm.org/docs/WritingAnLLVMNewPMPass.html.

Appendix A – Screenshots

Appendix B – Source Code

Included in this appendix is a snapshot of the project repository as of the date of submission of this interim report. The full repository, including the entire git history, is available upon request. Please request access to the private repository located at https://github.com/davidcallanan/mu-fyp.

Build artifacts and other uninteresting files have been omitted from this appendix. This includes, but is not limited to, all files mentioned inside .gitignore.

.gitmodules

```
[submodule "compiler/frontend/uoe"]
    path = compiler/frontend/uoe
    url = https://github.com/davidcallanan/uoe
```

2510_syntax_ideas/idea01.ec

```
what i want to do is write my own language, but allow to user to link in with llvm ir
    ↪ should any functionality not be feasible in my language (e.g. specific
    ↪ optimization flags etc).

sources :intf {
    /x/y/z /blah/wah/frah
}

sources :impl {
    /x/y/z /blah/wah/frah
}

deps {
    :print (/print, :static(/x86_64/print, :object_wide:global)); // uses one inter-
    ↪ object implementation
    :print (/print, :static(/x86_64/print, :object_wide:named(/my_lovely_instance)));
    :print (/print, :static(/x86_64/print, :local)); // any instance of this module has
    ↪ its own implementation
    :print (/print :)
}

no because i think the implementation of being static etc actually depends on the bigger
    ↪  picture

deps {
    :print (/print, :concrete(/x86_64/print, :local, :static));
    :print (/print, :concrete(/x86_64/print, :named(/my_implementation), :static));
    :print (/print, :passed:named(/my_implementation));
}
```

```
# this is tough

maybe a distinction for each "module" of what's "internal" to that module and what is "
    ↪ external". anything internal it is the responsibility of the module author to
    ↪ instantiate one with internal aspects prepopulated.

in another module, loading up a module it is assumed that anything internal is pre-
    ↪ populated.

when something is internal it is understood that you cannot create "multiple instances"
    ↪ (not of the module but of the module factory i mean). thus, that module can be
    ↪ configured at a global level.

instantiate modsource ();

:x86_64:print

do i really need a / variant?

print := import_intf:print;

the idea is that / is subject to translation.

any functions that work on the same "realm" will automatically use the designated
    ↪ translator when taking input. [for now, this translator is not going to exist, but
    ↪  the point is it could eventually].
```

2510_syntax_ideas/idea02.ec

```
; this is a comment because why not! i think it would be pretty cool.
; double slash means go to parent root
; triple slash means go to parent root and find its parent root
; there is a cap on triple slash. if you need to go further, you must then start doing
    ↪ mappings within the parent root "sources" entry.
; this ensures we dont go absolutely crazy.

reset-double-root; this resets // to current module
reset-triple-root; this resets /// to current module

sources {
    /// / ; maybe this is a better way to reset?
    // /
    /// reset; or reset like this
    /deps/foo //deps/foo
    /deps/bar ///deps/bar
    print("hello");
    ; only sources can access // and ///, using this anywhere else would be illegal, so
    ↪ you have to create a local mapping.
    // reset; must come at the end because previous mappings will refer to old version
    ↪ ... or not, the user can just have the common sense to know that this only impacts
    ↪  child modules.
    /// resetchildren; this resets only for children.
    ; or preferably
    double-reset;
    triple-reset;
    triple-reset-children;
    double-reset-children;
    ; or maybe
    reset-double;
    reset-triple;
    reset-triple-children;
    reset-double-children;
}
```

2510_syntax_ideas/idea03.ec

```
sources {
    /intf/foo //intf/foo;
    /intf/bar //intf/bar;
    ; if reset /// appears here, it wouldn't be accessible in forwarding blocks
}

forwarded sources {
    reset ///;
    ; if reset /// appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks
}

public namespace foo = import /intf/foo;

namespace bar = import /intf/bar {
    reset //;
    //intf/foo /intf/foo
    //banana/apple //banana/apple
    //banana/grape ///banana/grape
}

type Foo = foo::Foo;

public type Bar = bar::Bar;

; should anything in triple be automatically available in double?
; maybe the other way around? if we access triple, it will fallback to double?
; no it doesn't do justice.
```

2510_syntax_ideas/idea04.ec

```
sources {
    /intf/foo //intf/foo;
    /intf/bar //intf/bar;
    /intf/baz //intf/com.dcallanan/baz;
    ; if reset // appears here, it wouldn't be accessible in forwarding blocks
}

forwarded sources {
    reset //;
    ; if reset // appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks
}

public namespace foo import /intf/foo;

namespace foo_bar import /intf/bar {
    reset //;
    //intf/foo /intf/foo;
    //banana/apple //banana/apple;
}

type Foo foo_bar::FooBarf;

public type Bar foo_bar::Bar;

type Foo map {
    :vfs map (Foo);
    :blah map (*Bar);
    :wah Foo; syntactic sugar for :wah map (Foo)
}

type Bar enum {
    :my_entry_name;
    :my_other_entry_name (
```

```
    );
}

type DeliciousInteger i33;
```

2510_syntax_ideas/idea05.ec

```
sources {
    /intf/foo //intf/foo;
    /intf/bar //intf/bar;
    /intf/baz //intf/com.dcallanan/baz;
    ; if reset // appears here, it wouldn't be accessible in forwarding blocks
}

forwarded sources {
    reset //;
    ; if reset // appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks
}

public namespace foo import /intf/foo;

namespace foo_bar import /intf/bar {
    reset //;
    //intf/foo /intf/foo;
    //banana/apple //banana/apple;
}

type Foo foo_bar::FooBarf;

public type Bar foo_bar::Bar;

type Bar enum {
    :my_entry_name;
    :my_other_entry_name map (
        :vfs Foo;
        :bar i32;
    );
}

type Foo map {
    :vfs Foo; syntactic sugar for :vfs map (: Foo);
    :blah *Bar;
    :point map (i32, i32, i32);
    :distance(:x i32, :y i32) i32;
    (:x i32, :y i32) f64;
    : Foo;
}

type DeliciousInteger i33;
```

2510_syntax_ideas/idea06.ec

```
sources {
    /intf/foo //intf/foo;
    /intf/bar //intf/bar;
    /intf/baz //intf/com.dcallanan/baz {
        ; allows controlling with higher priority what is populated when this is later
    ↪ used
        ; for example if this is later passed down to other modules,
        ; it will base off the "//" at this scope,
        ; not what "//" later happens to be.
        /blah/wah //blah/wah;
```

```
        ; / always means the "module root" even though we could be touching a directory
    ↪ that does not reset the module root, recall the difference
        ; between the raw filesystem and the module hierarchy.
        ; thus / is the current module root irrespective of the directory or file
    ↪ mentioned.
        ; and // is the current "outer root" that this module is operating under.
    }
    ; if reset // appears here, it wouldn't be accessible in forwarding blocks
}

forwarded sources {
    reset //;
    ; if reset // appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks
}

public namespace foo import /intf/foo;

namespace foo_bar import /intf/bar {
    reset //;
    //intf/foo /intf/foo;
    //banana/apple //banana/apple;
}

type Foo foo_bar::FooBarf;

public type Bar foo_bar::Bar;

type Bar enum {
    :my_entry_name;
    :my_other_entry_name map (
        :vfs Foo;
        :bar i32;
    );
}

type Foo map {
    :vfs Foo; syntactic sugar for :vfs map (: Foo);
    :blah *Bar;
    :point map (i32, i32, i32);
    :distance(:x i32, :y i32) i32;
    (:x i32, :y i32) f64;
    : Foo;
}

type DeliciousInteger i33;
```

2510_syntax_ideas/idea07.ec

```
sources {
    /intf/foo //intf/foo;
    /intf/bar //intf/bar;
    /intf/baz //intf/com.dcallanan/baz {
        ; allows controlling with higher priority what is populated when this is later
    ↪ used
        ; for example if this is later passed down to other modules,
        ; it will base off the "//" at this scope,
        ; not what "//" later happens to be.
        /blah/wah //blah/wah;
        ; / always means the "module root" even though we could be touching a directory
    ↪ that does not reset the module root, recall the difference
        ; between the raw filesystem and the module hierarchy.
        ; thus / is the current module root irrespective of the directory or file
    ↪ mentioned.
        ; and // is the current "outer root" that this module is operating under.
    }
    ; if reset // appears here, it wouldn't be accessible in forwarding blocks
```

```
}

forwarded sources {
    reset //;
    ; if reset // appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks
}

public namespace foo import /intf/foo;

namespace foo_bar import /intf/bar {
    reset //;
    //intf/foo /intf/foo;
    //banana/apple //banana/apple;
}

type Foo foo_bar::FooBarf;

public type Bar foo_bar::Bar;

type Bar enum {
    :my_entry_name;
    :my_other_entry_name map (
        :vfs Foo;
        :bar i32;
    );
}

type Foo map {
    :vfs Foo; syntactic sugar for :vfs map (: Foo);
    :blah *Bar;
    :point map (i32, i32, i32);
    :distance(:x i32, :y i32) i32;
    (:x i32, :y i32) f64;
    : Foo;
}

type DeliciousInteger i33;
```

2510_syntax_ideas/idea08.ec

```
sources {
    /intf/foo //intf/foo;
    /intf/bar //intf/bar;
    /intf/baz //intf/com.dcallanan/baz {
        ; allows controlling with higher priority what is populated when this is later
    ↪ used
        ; for example if this is later passed down to other modules,
        ; it will base off the "//" at this scope,
        ; not what "//" later happens to be.
        /blah/wah //blah/wah;
        ; / always means the "module root" even though we could be touching a directory
    ↪ that does not reset the module root, recall the difference
        ; between the raw filesystem and the module hierarchy.
        ; thus / is the current module root irrespective of the directory or file
    ↪ mentioned.
        ; and // is the current "outer root" that this module is operating under.
    }
    ; if reset // appears here, it wouldn't be accessible in forwarding blocks
}

forwarded sources {
    reset //;
    ; if reset // appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks
}
```

```
public namespace foo import /intf/foo;

namespace foo_bar import /intf/bar {
    reset //;
    //intf/foo /intf/foo;
    //banana/apple //banana/apple;
}

type Foo foo_bar::FooBarf;

public type Bar foo_bar::Bar;

type Bar enum {
    :my_entry_name;
    :my_other_entry_name map (
        :vfs Foo;
        :bar i32;
    );
}

type Foo map {
    :vfs Foo; syntactic sugar for :vfs map (: Foo);
    :blah *Bar;
    :point map (i32, i32, i32);
    :distance map map (:x i32, :y i32) i32;
    :distance map (:x i32, :y i32) -> i32; i think this looks better
    map map (:x i32, :y i32) f64;
    map (:x i32, :y i32) -> f64; i think this is good stuff
    : Foo;
}

type DeliciousInteger i33;
```

2511_interim_report/references.bib

```
@online{llvmcpp,
  author  = {{The LLVM Project}},
  title   = {Frequently Asked Questions ({FAQ})},
  url     = {https://llvm.org/docs/FAQ.html#in-what-language-is-llvm-written},
  urldate = {2025-11-17}
}

@online{llvmcodegen,
  author  = {{The LLVM Project}},
  title   = {The {LLVM} Target-Independent Code Generator},
  url     = {https://llvm.org/docs/CodeGenerator.html},
  urldate = {2025-11-17}
}

@online{llvminterop,
  author  = {Sreekumar, N.},
  title   = {{GSoC} 2025: Introducing an {ABI} Lowering Library},
  url     = {https://blog.llvm.org/posts/2025-08-25-abi-library/},
  year    = {2025},
  month   = {11},
  day     = {3},
  urldate = {2025-11-07}
}

@online{llvmoptim,
  author  = {{The LLVM Project}},
  title   = {{LLVM}'s Analysis and Transform Passes},
  url     = {https://llvm.org/docs/Passes.html#domtree-dominator-tree-construction},
  urldate = {2025-11-17}
}

@online{llvmclang,
```

```
  author  = {{The Clang Project}},
  title   = {Clang: {C} Language Family Frontend for {LLVM}},
  url     = {https://clang.llvm.org/},
  urldate = {2025-11-17}
}

@online{llvmasm,
  author  = {{The LLVM Project}},
  title   = {{LLVM} Language Reference Manual},
  url     = {https://llvm.org/docs/LangRef.html#inline-assembler-expressions},
  urldate = {2025-11-17}
}

@online{inlinekern,
  author  = {Roy, D.},
  title   = {{GCC} Inline Assembly and Its Usage in the Linux Kernel},
  url     = {https://dl.acm.org/doi/fullHtml/10.5555/3024956.3024958},
  year    = {2016},
  month   = {11},
  day     = {1},
  urldate = {2025-11-06}
}

@online{llvmlto,
  author  = {{The LLVM Project}},
  title   = {{LLVM} Link Time Optimization: Design and Implementation},
  url     = {https://llvm.org/docs/LinkTimeOptimization.html},
  urldate = {2025-11-17}
}

@online{llvmtransformations,
  author  = {{The LLVM Project}},
  title   = {{LLVM} Programmer's Manual},
  url     = {https://llvm.org/docs/ProgrammersManual.html},
  urldate = {2025-11-17}
}

@online{llvmpasses,
  author  = {{The LLVM Project}},
  title   = {Writing an {LLVM} Pass},
  url     = {https://llvm.org/docs/WritingAnLLVMNewPMPass.html},
  urldate = {2025-11-02}
}

@online{t3chat,
  author  = {DigiVirus},
  title   = {Meet {T3} Chat {AI}: Your All-in-One, Super-Fast {AI} Assistant},
  url     = {https://digivirus.in/meet-t3-chat-ai-your-all-in-one-super-fast-ai-
    ↪ assistant/},
  year    = {2025},
  month   = {8},
  day     = {28},
  urldate = {2025-11-16}
}
```

2511_syntax_ideas/idea01.ec

```
sources {
    ; probably will scrap a regular sources block because we already have a layer of
    ↪ naming indirection when "importing" should that not be sufficient?
    ; each import then involves a concious choice of using either "/" (i own the module
    ↪ !) or "//" (i am using something external). it doesn't require much effort to
    ↪ change between them if you change your mind.
}

forwarding {
    reset //;
```

```
    ; if reset // appears here, it wouldn't be forwarded but would still be accessible
    ↪ in forwarding blocks for individual imports to override if so wished.
    ; the default strategy is not to reset, so that forwarding is automatic.
    ; LHS can only be // here, but RHS can be / or //.
    ; resets must be the very first thing if wanted, otherwise no reset.
    ; every module implicitely resets /.
}

; the key is that a module always owns its types when it comes to the type system (
    ↪ irrespective of whether it owns the codebase when it comes to importing).
; thus it seems rather convenient to be able to export a namespace.
public types foo import /intf/foo;

types foo import /intf/foo;

types foo_bar import /intf/bar forwarding {
    reset //;
    //intf/foo /intf/foo;
    //banana/apple //banana/apple;
    ; like other forwarding blocks, LHS must be //, but RHS can be / or //.
}

type Foo foo_bar::FooBarf;

public type Bar foo_bar::Bar;

type Bar enum {
    :my_entry_name;
    :my_other_entry_name map (
        :vfs Foo;
        :bar i32;
    );
}

type Foo map {
    :vfs Foo; syntactic sugar for :vfs map (: Foo);
    :blah *Bar;
    :point map (i32, i32, i32);
    :distance map map (:x i32, :y i32) i32; i don't want to use this syntax
    :distance map (:x i32, :y i32) -> i32; i think this looks better
    ; the idea is that "->" automatically means a map that takes in non-sym input
    ; of course then in the following line we make the current map itself callable with
    ↪ non-sym.
    ; internally it is taught of as a sort of ":call" symbol, because i like making
    ↪ anything callable.
    map (:x i32, :y i32) -> f64; i think this is good stuff
    : Foo; this is what the type is when not treated as a map. the "leaf value" as i
    ↪ have called it for many years.
}

type DeliciousInteger i33;

; now in source modules, we need to think how we take in dependencies and all that and
    ↪ instantiate a module
; before i think i was thinking that the source module specifies things like how it
    ↪ wants to grab these dependencies (static vs dynamic), but i think the module user
    ↪ should be able to control that.
; really dependencies is just a map?

dependencies map {
    :foo Foo; we want an instance of foo
    :bar_factory BarFactory;
}

options map {}

; however i don't want to create special language constructs for this. i wonder is it
    ↪ possible to make module instantiating a regular function. well, it is indeed a
    ↪ comptime function, or is it.
; no it's not, it can be runtime, so we can pass in dependencies.
```

```
; but at some stage we can specify that a variable is "static" or something like this,
    ↪ and the compiler has to trace that and remember it.
; the tracing might be difficult.

; i guess first, we should look at how do we import a source module compared to intf,
    ↪ and probably similar.

types foo import /impl/foo;

; i guess ‘foo‘ now has some types in it but then there would also be a way to
    ↪ instantiate it.
; there could be a cool thing i mention in my report "compile-time tracing of the static
    ↪  vs dynamic nature through runtime code".

; maybe we just take stuff "in".

in map {
    :dependencies map {
        :foo Foo; we want an instance of foo
        :bar_factory BarFactory;
    }

    :options map {
        :lru_size i32;
    }
};

; one can repeat "in map" blocks and they get merged.
; we might go ahead and force "in" map to be a non-tuple map to enable this
    ↪ straightforward merge behaviour.

; any instance of a module keeps track of whether it is a runtime or comptime instance.
; some simple rules can then be applied:
; when we create an instance of a module, and pass it a dependency, if the dependency is
    ↪  clearly comptime (say at the global scope), the instance remembers this.
; if runtime then also remembers this.
; but the key is that comptime can be reduced to runtime (if any chance of runtime) but
    ↪ runtime can not ever go back up to comptime.
; we can enforce comp-time perhaps through a little keyword (and simply the compiler
    ↪ would choose to bail out if it can’t prove it).
; by default when you create an instance of a module, it is made as comptime if the
    ↪ instance itself is comptime (and maybe instead of comptime we are thinking of the
    ↪ word "static").

; im trying to think it through. i guess the root "entry" module we compile ought to be
    ↪ comptime.
; if we access something straight from "input" then this comptime is preserved.
; but let’s say we call some function or something to obtain the instance, then comptime
    ↪  is gone.
; wait, but what about when we genuinely instantiate a module instance.... ahhh

; i think it comes back to, do I own the instance or not? (similar to do i own the code
    ↪ ?)

; let’s suppose we instantiate a module instance...
; i guess a namespace can have attached to it an implementation, allowing instantiation.
; it’s like the idea of leaf functions, but here, we are thinking to ourselves that
    ↪ there are no leaf functions anymore, just the one "instantiation" function.
    ↪ further non-leaf instantiation functions can always be created if so desired.

my_module := instantiate foo(
    :dependencies {
        ; blah
    }
    :options {
        ; blah
    }
);

; notice that this instantiation is done at the global scope, thus it can inherit the
```

```
    ↪ comptime of the current module instance.
; while properties in maps could also keep track of comptime vs runtime, i feel this
    ↪ might be too complex for a fyp that is due in March.
; so let's just simplify for now and so VARIABLES at module scope keep track of comptime
    ↪  vs runtime.
; variables are easier to deal with.. don't got the complexities of a map with types and
    ↪  all that.

; note though: the "input" is a map, so we do have to keep track through maps ahhhhh

; right how via gonna track via maps? remember we have a few possibilities, we have
    ↪ static, we have thread-local storage or global variable, and we have dynamic.
; however, we can assume tls and gv are static, as a static implementation can wrap the
    ↪ dynamic nature.
; we can use LTO (link time optimization) to easily do this (which would be great to
    ↪ talk about in my report).

; in my report: "technically C does not have to even compile to object files, it could
    ↪ compile each file to an executable file and then use an overkill runtime linkage
    ↪ mechanism, the point is that there is room for alteration in this".

; so all "sym" map instances (don't care about non-sym variants) must keep track of
    ↪ their comptime vs runtime status, collapsing to the worst-case scenario (kind of
    ↪ like how a typesystem collapses its possible values).

; by default, dispatch is "static", but there would be perhaps three options for this
    ↪ demo.

my_module := instantiate foo(

) dispatch {
    static force
};

my_module := instantiate foo(

) dispatch {
    global
};

my_module := instantiate foo(

) dispatch {
    dynamic
};

; force will ensure that the compiler bails out if not possible, but by default, it is
    ↪ just a recommendation.

; when one instantiates a module in another scope, then dispatch is ignored and it is
    ↪ just treated as dynamic, like instantiating a class.

; however, what if we instantiate two modules that both implement that interface and
    ↪ might pass through?
; i think when we say dynamic, we do in fact mean double dispatch, unfortunately. there
    ↪ is an element of saying the implementation is fixed, but its dependencies etc. is
    ↪ dynamic, so maybe we can consider dyanamic-single vs dynamic-double.
; even some of these things can be put through as considerations in my report, whether i
    ↪  implement or not (because implementing dynamic gonna be hell of a lot easier).

; i think i can make effort to propogate dyanmic-single vs dynamic-double,,, we'll seee
    ↪ ..



; im thinking when importing a source module it will instaed be

mod my_module import /impl/my_module;

; we distinguish between "mod" and "ns" because "mod" has instantiation function.
```

2511_syntax_ideas/idea02.ec

```
; i was leaning towards not having special treatment for module instances and that we
    ↪ can have a receiver-like system for any sort of instances.
; but we would force that modules require an instance, in the sense that we have leaf
    ↪ maps that can't do much by themselves.

; it just could get messy if a module has the ability to create a vector etc.
; i might go with implicit module receivers (for anything called create_mod)?
; so instead of this:mod:blah we'd just have mod:blah.
; or maybe module gets special treatement i don't know at this point.

create(:my_dependency Foo, :math Math) {
    this:my_dependency @= my_dependency; // @= says "declare and assign if not already
    ↪ declared", whereas ":= is for variables and can have shadowing, whereas @= we know
    ↪  like there's genuinely only one version".
    this:math @= math;
}

@:create_world() {
    mod:my_dependency:do_something;
}

@:create_world:create_vector(x f64, y f64, z f64) {
    mod:my_dependency:do_something_or_another;

    this:x @= x;
    this:y @= y;
    this:z @= z;
}

@:create_world:create_vector:get_x() -> f64 {
    return this:x;
}

@:create_world:create_vector:get_y() -> f64 {
    return this:y;
}

@:create_world:create_vector:get_z() -> f64 {
    return this:z;
}

@:create_world:create_vector:get_distance(other T:create_world:create_vector) -> f64 {
    return mod:math:sqrt(
        + (this:x - other:get_x) * (this:x - other:get_x)
        + (this:y - other:get_y) * (this:y - other:get_y)
        + (this:z - other:get_z) * (this:z - other:get_z)
    );
}

; i can talk about how forcing no ability for global variables forces the compiler to
    ↪ track everything. preparing us nicely for optimizations.

; need some self-loop of wanting to actually get the exposed version of "this", like "
    ↪ this~" or something.

; ah what now about private functions within my shtuff! now i need a symbol for private
    ↪ too?

; also look now carefully how awful the typing syntax is...
; maybe i do need to have type for returned instances, but damn i didn't want this.
; also note that i want want different factories for a type, but really the other
    ↪ factories should wrap a main internal factory kind of.

; also create_vector doesn't know any context about create_world unless the information
    ↪ of world is stored in vector's this... so it's kinda like not really appropriate
    ↪ to even have this create_world mentioned...
```

```
; note also that maybe if we do have like a Vector declaration that we need only include
    ↪  mutable things in here (to prevent the need for @=) and anything constant can be
    ↪ extended in the same way methods are.
```

2511_syntax_ideas/idea03.ec

```
; let's have another go at idea02.

type Mod map {
    :_my_dependency Foo;
    :_math Math;
}

create(:my_dependency Foo, :math Math) {
    ; if constructor finds something not assigned, compile-time error.
    mod:_my_dependency = my_dependency;
    mod:_math = math;
    ; i'm thinking of _ here but in practice i think i want to follow the public
    ↪ approach where there is no such thing as module-level private.
}

type World map {
}

; the idea is that one should be able to immediately start doing stuff with @Mod without
    ↪  even using "create" or "type Mod map" unless they actually need it.

@Mod:create_world() -> World { ; we want () here to allow later extensibility.. this is
    ↪ just a convention that programmers would likely follow.
    mod:_my_dependency:do_something;
}

type Vector map {
    :_x f64;
    :_y f64;
    :_z f64;
}

@World:create_vector(x f64, y f64, z f64) -> Vector {
    mod:_my_dependency:do_something_or_another;

    this:_x = x;
    this:_y = y;
    this:_z = z;
}

@Vector:get_x() -> f64 {
    return this:_x;
}

@Vector:get_y() -> f64 {
    return this:_y;
}

@Vector:get_z() -> f64 {
    return this:_z;
}

@Vector:get_distance(other T:Vector) -> f64 {
    return mod:_math:sqrt(
        + (this:_x - other:get_x) * (this:_x - other:get_x)
        + (this:_y - other:get_y) * (this:_y - other:get_y)
        + (this:_z - other:get_z) * (this:_z - other:get_z)
    );
}
```

2511_syntax_ideas/idea04.ec

```
; the goal of this iteration is to determine how public private should work
; and to resolve the notations of maps (RHS) of @ extensions.
; i'd also like to determine how to make things mutable.

; anything mutable must be defined in the type. how about we suppose that this is not
    ↪ necessary for sake of exploration.

@Mod:my_dependency Foo;
@Mod:math Math;

create(:my_dependency Foo, :math Math) {
    mod:my_dependency = my_dependency;
    mod:math = math;
}

type World map;

pub @Mod:create_world map () -> World . { ; . means implicit? this distinguishes between
    ↪  type details and actual body.
    mod:my_dependency:do_something;
}

type Vector map;

mut @Vector:x f64; mut kinda means non-deterministic
mut @Vector:y f64; these should mutable because maybe we have a method to mutate them
mut @Vector:z f64;

pub @World:create_vector map (x f64, y f64, z f64) -> Vector . {
    mod:my_dependency:do_something_or_another;

    ; this is problematic. this could be referring to world, or it could be referring to
    ↪  vector instance.

    this:x = x;
    this:y = y;
    this:z = z;
}

pub @Vector:get_x map . {
    return this:x;
}

pub @Vector:get_y map . {
    return this:y;
}

pub @Vector:get_z map . {
    return this:z;
}

pub @Vector:get_distance map (other Vector) -> f64 . {
    return mod:math:sqrt(
        + (this:x - other:get_x) * (this:x - other:get_x)
        + (this:y - other:get_y) * (this:y - other:get_y)
        + (this:z - other:get_z) * (this:z - other:get_z)
    );
}

; on the next iteration i need to resolve some things:
; - this does not work in factories very nicely... ahhh!!!
; - . for implicit is weird... i need to think that through more.
```

2511_syntax_ideas/idea05.ec

```
; i think we already decided prior that the whole point of prefixing maps with "map" at
    ↪ the type level is so that we can take advantage of () and {} notation for
    ↪ instances.

; so i wonder can we get going with that immediately.

@Mod:my_dependency Foo;
@Mod:math Math;

create(:my_dependency Foo, :math Math) {
    :my_dependency my_dependency;
    :math math;
}

type World map;

pub @Mod:create_world() -> World { ; here World is kinda optional, and we can make
    ↪ everything optional because we know the difference between type and instance at
    ↪ the lexer level almost.
    mod:my_dependency:do_something;
}

type Vector map;

mut @Vector:x f64;
mut @Vector:y f64;
mut @Vector:z f64;

pub @World:create_vector(:x f64, :y f64, :z f64) -> Vector {
    mod:my_dependency:do_something_or_another;

    ; normally we would return things like this:

    :x x;
    :y y;
    :z z;

    ; but remember! these are private!
    ; seems dodgy.
    ; then again, the notion of private is a module-level thing (Go ftw).
    ; there's not supposed to be the ability to do private.
    ; if you need truly private and temporary, then use a local variable
    ; otherwise you just gotta use a field.
    ; you can internally decide on a _x convention if wanted, but my thinking is that
    ↪ this is unnecessary
    ; since i myself have accessed _x things outside where i thought i would in
    ↪ javascript many times, that's the whole point of having a module level of privacy.
}

pub @Vector:get_x f64 {
    return this:x;
}

pub @Vector:get_y f64 {
    return this:y;
}

pub @Vector:get_z f64 {
    return this:z;
}

pub @Vector:get_distance(other Vector) -> f64 {
    return mod:math:sqrt(
        + (this:x - other:get_x) * (this:x - other:get_x)
        + (this:y - other:get_y) * (this:y - other:get_y)
        + (this:z - other:get_z) * (this:z - other:get_z)
    );
}

; this actually looks quite good.
```

2511_syntax_ideas/idea06.ec

```
@Mod:my_dependency Foo;
@Mod:math Math;

create(:my_dependency Foo, :math Math) {
    :my_dependency my_dependency;
    :math math;
}

type World map;

pub @Mod:create_world() -> World {
    mod:my_dependency:do_something;
}

type Vector map;

mut @Vector:x f64;
mut @Vector:y f64;
mut @Vector:z f64;

pub @World:create_vector(:x f64, :y f64, :z f64) -> Vector {
    mod:my_dependency:do_something_or_another;

    :x x;
    :y y;
    :z z;
}

pub @Vector:get_x f64 {
    return this:x;
}

pub @Vector:get_y f64 {
    return this:y;
}

pub @Vector:get_z f64 {
    return this:z;
}

pub @Vector:get_distance(other Vector) -> f64 {
    return mod:math:sqrt(
        + (this:x - other:get_x) * (this:x - other:get_x)
        + (this:y - other:get_y) * (this:y - other:get_y)
        + (this:z - other:get_z) * (this:z - other:get_z)
    );
}
```

2511_syntax_ideas/idea07.ec

```
types foo import //intf/foo;

mod bar import /impl/bar forwarding {
    reset //; todo: i need to update my code to allow resetting any "folder"?
    reset //foo/bar/baz;
    //intf/foo //intf/foo;
}

type Foo foo::Foo;
type Math bar::Math;

@Mod:my_dependency Foo;
@Mod:math Math;

create(:my_dependency Foo, :math Math) -> {
```

```
    :my_dependency my_dependency;
    :math math;
}

type World map;

pub @Mod:create_world() -> World {
    mod:my_dependency:do_something;
}

type Vector map;

mut @Vector:x f64;
mut @Vector:y f64;
mut @Vector:z f64;

pub @World:create_vector(:x f64, :y f64, :z f64) -> Vector {
    mod:my_dependency:do_something_or_another;

    :x x;
    :y y;
    :z z;
}

pub @Vector:get_x f64 {
    return this:x;
}

pub @Vector:get_y f64 {
    return this:y;
}

pub @Vector:get_z f64 {
    return this:z;
}

pub @Vector:get_distance(other Vector) -> f64 {
    first_part := (this:x - other:get_x) * (this:x - other:get_x)
    second_part := (this:y - other:get_y) * (this:y - other:get_y)
    third_part := (this:z - other:get_z) * (this:z - other:get_z)

    : mod:math:sqrt(
        + first_part
        + second_part
        + third_part
    );
}
```

2603_final_report/references.bib

```
@online{llvmcpp,
  author  = {{The LLVM Project}},
  title   = {Frequently Asked Questions ({FAQ})},
  url     = {https://llvm.org/docs/FAQ.html#in-what-language-is-llvm-written},
  urldate = {2025-11-17}
}

@online{llvmcodegen,
  author  = {{The LLVM Project}},
  title   = {The {LLVM} Target-Independent Code Generator},
  url     = {https://llvm.org/docs/CodeGenerator.html},
  urldate = {2025-11-17}
}

@online{llvminterop,
  author  = {Sreekumar, N.},
  title   = {{GSoC} 2025: Introducing an {ABI} Lowering Library},
```

```
  url     = {https://blog.llvm.org/posts/2025-08-25-abi-library/},
  year    = {2025},
  month   = {11},
  day     = {3},
  urldate = {2025-11-07}
}

@online{llvmoptim,
  author  = {{The LLVM Project}},
  title   = {{LLVM}'s Analysis and Transform Passes},
  url     = {https://llvm.org/docs/Passes.html#domtree-dominator-tree-construction},
  urldate = {2025-11-17}
}

@online{llvmclang,
  author  = {{The Clang Project}},
  title   = {Clang: {C} Language Family Frontend for {LLVM}},
  url     = {https://clang.llvm.org/},
  urldate = {2025-11-17}
}

@online{llvmasm,
  author  = {{The LLVM Project}},
  title   = {{LLVM} Language Reference Manual},
  url     = {https://llvm.org/docs/LangRef.html#inline-assembler-expressions},
  urldate = {2025-11-17}
}

@online{inlinekern,
  author  = {Roy, D.},
  title   = {{GCC} Inline Assembly and Its Usage in the Linux Kernel},
  url     = {https://dl.acm.org/doi/fullHtml/10.5555/3024956.3024958},
  year    = {2016},
  month   = {11},
  day     = {1},
  urldate = {2025-11-06}
}

@online{llvmlto,
  author  = {{The LLVM Project}},
  title   = {{LLVM} Link Time Optimization: Design and Implementation},
  url     = {https://llvm.org/docs/LinkTimeOptimization.html},
  urldate = {2025-11-17}
}

@online{llvmtransformations,
  author  = {{The LLVM Project}},
  title   = {{LLVM} Programmer's Manual},
  url     = {https://llvm.org/docs/ProgrammersManual.html},
  urldate = {2025-11-17}
}

@online{llvmpasses,
  author  = {{The LLVM Project}},
  title   = {Writing an {LLVM} Pass},
  url     = {https://llvm.org/docs/WritingAnLLVMNewPMPass.html},
  urldate = {2025-11-02}
}

@online{t3chat,
  author  = {DigiVirus},
  title   = {Meet {T3} Chat {AI}: Your All-in-One, Super-Fast {AI} Assistant},
  url     = {https://digivirus.in/meet-t3-chat-ai-your-all-in-one-super-fast-ai-
    ↪ assistant/},
  year    = {2025},
  month   = {8},
  day     = {28},
  urldate = {2025-11-16}
}
```

README.md

```
# Essence C (Maynooth University - Final Year Project)

This respository acts as the definitive record of work for my final year project. I will
    ↪  try to keep as much of my work as possible within this repo - including code,
    ↪ writeups, notes, random thoughts.

The goal of this project is to develop a custom programming language known as "Essence C
    ↪ ".

The language should include bespoke mechanism(s) to facilitate a flexible module
    ↪ dispatch mechanism - primarily achieved by fusing the notions of classes and
    ↪ modules that are traditionally separate in existing programming languages.

The end goal of this is to enhance the compiler's potential to perform link-time
    ↪ optimization on the resulting artifacts, resulting in small performance
    ↪ optimizations relative to other low-level languages (if things work out smoothly).

Additional research (if time permits) will be to study how this facilitates the
    ↪ continuum of both static and dynamic modules in kernel development. This may (if
    ↪ time permits) involve writing some simple kernel code in my programming language
    ↪ and analyzing performance gains relative to C code. The goal will be to get the
    ↪ project working before focusing on these aspects though.

One core aspect of the project will be attempting to maintain interconnectivity with
    ↪ existing low-level infrastructure, as the language must be suitable for kernel
    ↪ development. It is also pretty much guaranteed that LLVM IR will be the target (
    ↪ and I want to look at Clang's link-time optimizations when it uses LLVM IR). If I
    ↪ am using LLVM IR, the easiest approach will be to support interconnectivity
    ↪ between my language and LLVM IR, rather than offering direct integration with any
    ↪ other programming languages. LLVM IR also supports inline assembly really
    ↪ effectively (and this should be adequate for kernel development although I would
    ↪ have to look into this). My compiler might have to do some analysis of outputted
    ↪ LLVM IR files and even do some minor mutations to them to facilitate my
    ↪ performance optimizations (I think this is likely given that we may link foreign
    ↪ LLVM IR code and we may need to touch this code to get the best out of our
    ↪ optimizations across module boundaries, which is what I would want to consider
    ↪ this an absolute success).

Of course, this final year project is really just initial research and development of
    ↪ something that could take years of work. So I must manage my expectations and
    ↪ remember that I am not creating a fully-functional programming language that is
    ↪ ready for production use, or anything close to it.

## Project Structure

- `compiler` houses the actual compilation process which takes in language code and
    ↪ spits out a final executable.
- `extension` houses the VS Code extension that offers syntax highlighting to the
    ↪ programmer.
- `XXXX_syntax_ideas` contains some early syntax ideas ideas which have been
    ↪ significantly deviated from.

## Git Submodules

**Mandatory:** When cloning this repo, make sure git "submodules" are enabled and are
    ↪ recursively cloned.

If you have accidentally cloned without submodules, it is no problem, simply run this
    ↪ command:

```
git submodule update --init --recursive
```
```

TODO.txt

```
LANGUAGE FEATURES:
add values to enums (or bypass this for now with a simpler approach by using one side of
    ↪  the map enum equivalence)
add callable map and respective extat implementation.
add call_with_map
add comptime llvm/c abi [as a hardcoded ct variable, no comptime variables for now? or
    ↪ perhaps that is needed and system detects it as static]
when using C abi, just extract values from struct in order probably
reorganize evluate_structval.cpp

today: @Mod + start work on extern linking.
Sunday: finish extern linking.
Monday: start on optimization for single module instance.
Tuesday: finish optimization.
Wednesday: Bencharmking.
Thursday: Finish benchmarking.
Friday: Synthesize everything.
Satruday: Submit.
optional: positional varaibles just to make function call loook nicer.

MAIN GOALS:
add module system
add optimizations
add evaluation
maybe a kernel test case of writing characters to the screen (one by one) to show
    ↪ inlining performance diff of direct assembly execution? (or find some kernel-
    ↪ related thing that requires an assembly instruction).
```

appendix_generator/llm_prompts/README.md

```
# Appendix Generator - LLM Prompts

This quick and dirty tool takes the entire LLM history from my LLM provider (which I
    ↪ have manually dumped to ‘input.json‘) and filters by thread id and message id.
    ↪ This allows me to include only the threads relevant to this FYP without including
    ↪ any of my personal LLM threads. The data is dumped into ‘appendix_llm_prompts.tex‘
    ↪  for inclusion in both the interim and final report.

Proper formatting and escape logic was implemented to get this to work nicely.

Run ‘node app.js‘ from this directory and the result will be outputted to ‘
    ↪ appendix_llm_prompts.tex‘.
```

appendix_generator/llm_prompts/app.js

```
import fs from "fs/promises";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const DEBUG_QUICK_TESTING = false;
const DEBUG_PRINT_IDS = false;

const input_path = path.join(__dirname, "input.json");
const output_path = path.join(__dirname, "appendix_llm_prompts.tex");

// this allows me to filter FYP-related threads from all my personal threads on the same
    ↪  account.
const thread_filter = [
    { tid: "7add1779-4337-44f1-a533-c37c4eb71efe" }, // done
    { tid: "765f7232-673f-4bf1-a4e4-a927957dbf00" }, // done
    { tid: "054b94b1-6432-4f9c-bcb6-42b0e0b916c5" }, // done
```

```
    {
        tid: "28da42c3-1ae9-4987-b23e-0595750ca2e5",
        mids: [ // done
            "j977xk3e5cmtr0k2w9emv2qdm17hy3dv",
            "j972dx1905kmscg1nn7b3xd9rx7hy2dd",
        ]
    },
    { tid: "6deac944-2edd-4644-9955-c8a780772eb2" }, // done
    { tid: "a9183424-da00-4790-9489-d0f65cddffef" }, // done
    { tid: "f97198a6-f399-4912-a023-8740dc2583dd" }, // done
    { tid: "42e8ae10-5d13-4ecb-8821-841f9ec04e0e" }, // done
    { tid: "84682898-db2e-4b16-83d1-a793f25c8ed0" }, // done
];

// latex is struggling with unicode - for now just gonna remove a bunch of it.
const wipe_unicode = (text) => {
    return text.replace(/[^\x00-\x7F]/g, "");
};

const escape_latex = (text) => {
    const clean_text = wipe_unicode(text);

    return (clean_text
        .replace(/\\/g, "\\textbackslash{}")
        .replace(/\^/g, "\\textasciicircum{}")
        .replace(/~/g, "\\textasciitilde{}")
        .replace(/[{}&%$#_]/g, "\\$&")
    );
};

const escape_verbatim = (text) => {
    const clean_text = wipe_unicode(text);

    return (clean_text
        .replace(/@/g, "@\\char\"40@")
        .replace(/\\end\{lstlisting\}/g, "@\\textbackslash{}end\{lstlisting\}@")
    );
};

const format_date_nicely = (iso_string) => {
    const date = new Date(iso_string);
    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart(2, "0");
    const day = String(date.getDate()).padStart(2, "0");
    const hours = String(date.getHours()).padStart(2, "0");
    const minutes = String(date.getMinutes()).padStart(2, "0");

    return `${year}/${month}/${day} ${hours}:${minutes}`;
};

const decide_message_color = (message) => {
    const role = message.role ?? "";

    if (role === "user") {
        return "blue!15";
    }

    if (role === "assistant") {
        return "red!15";
    }

    return "gray!10";
};

const should_include_message = (thread_id, message_id) => {
    const filter_entry = thread_filter.find((entry) => entry.tid === thread_id);

    if (!filter_entry) {
        return false;
    }
```

```
    if (!filter_entry.mids) {
        return true;
    }

    return filter_entry.mids.includes(message_id);
};

const generate_latex = async () => {
    console.log("Reading input from file:", input_path);

    const content = await fs.readFile(input_path, "utf8");

    const input_data = JSON.parse(content);

    console.log("top level keys", Object.keys(input_data).join(", "));

    const filtered_threads = [];

    for (const thread of (input_data.threads ?? []).reverse()) {
        const filter_entry = thread_filter.find((entry) => entry.tid === thread.id);

        if (!filter_entry) {
            continue;
        }

        console.log(`Found desired thread: ${thread.id}`);

        let filtered_messages = [];

        const messages_in_thread = (input_data.messages ?? []).filter((msg) => msg.
    ↪ threadId === thread.id);
        console.log(`thread has ${messages_in_thread.length} messages.`);

        for (const message of messages_in_thread) {
            if (should_include_message(thread.id, message.id)) {
                filtered_messages.push(message);
            }
        }

        if (filtered_messages.length > 0) {
            if (DEBUG_QUICK_TESTING) { // just chop off any more than 2 messages to
    ↪ speed up latex compilation
                filtered_messages = filtered_messages.slice(0, 2);
            }

            filtered_threads.push({
                ...thread,
                messages: filtered_messages,
            });
        }
    }

    console.log(`Filtered data to ${filtered_threads.length} relevant threads.`);

    let latex = "";
    latex += "% This is an auto-generated appendix using my appendix generator script\n
    ↪ ";
    latex += "% Date of generation: " + new Date().toISOString() + "\n\n";
    latex += "\\lstset{\n";
    latex += "  breaklines=true,\n";
    latex += "  breakatwhitespace=false,\n";
    latex += "  postbreak={\\mbox{\\textcolor{red}{$\\hookrightarrow$}\\space}},\n";
    latex += "  columns=fixed,\n";
    latex += "  keepspaces=true,\n";
    latex += "  showstringspaces=false,\n";
    latex += "  tabsize=4,\n";
    latex += "}\n\n";

    let message_count = 0;
```

```
    for (const thread of filtered_threads) {
        let message_number = 1;

        const thread_start_time = format_date_nicely(thread.created_at);

        latex += "\\vspace{0.5cm}\n";
        latex += "\\noindent\\colorbox{lightgray}{\\parbox{\\dimexpr\\textwidth-2\\
    ↪ fboxsep}{\\small\\textbf{Thread (" + thread.messages.length + " messages) \\
    ↪ newline ID: " + thread.id;

        if (DEBUG_PRINT_IDS) {
            latex += " | ID: " + thread.id;
        }

        latex += "\\hfill " + thread_start_time + "}}}\n\n";
        latex += "\\vspace{0.2cm}\n\n";

        for (const message of thread.messages) {
            const message_date_time = format_date_nicely(message.created_at);
            const escaped_content = escape_verbatim(message.content || "");
            const message_color = decide_message_color(message);
            const role = message.role.toLowerCase();
            const role_display = role === "user" ? "USER" : "AGENT";
            const model_name = escape_latex(message.model ?? "unknown-model");

            latex += "\\noindent\\colorbox{" + message_color + "}{\\parbox{\\dimexpr\\
    ↪ textwidth-2\\fboxsep}{\\small \\textbf{" + role_display + "} (Message " +
    ↪ message_number + ") [" + model_name + "]";

            if (DEBUG_PRINT_IDS) {
                latex += " | ID: " + message.id;
            }

            latex += "\\hfill " + message_date_time + "}}\n\n";
            latex += "\\vspace{0.2cm}\n\n";
            latex += "\\begin{lstlisting}[language={}, escapechar=@, basicstyle=\\
    ↪ ttfamily\\footnotesize, breaklines=true, breakatwhitespace=false]\n";
            latex += escaped_content;

            if (!escaped_content.endsWith("\n")) {
                latex += "\n";
            }

            latex += "\\endlstlisting\n\n";

            message_count++;
            message_number++;
        }
    }

    await fs.writeFile(output_path, latex, "utf8");
    console.log(`\nLatex appendix has been successfully dumped to: ${output_path}`);
    console.log(`Total number of threads is ${filtered_threads.length}`);
    console.log(`Total number of messages is ${message_count}`);
};

generate_latex();
```

appendix_generator/llm_prompts/package.json

```
{
    "type": "module"
}
```

appendix_generator/source_code/README.md

```
# Appendix Generator - Source Code

This quick and dirty tool takes all files in the codebase (except some explicitly
    ↪ ignored patterns) and dumps their contents into a latex file for easy inclusion in
    ↪  the interim and final report.

Proper formatting and escape logic was implemented to get this to work nicely.

Run ‘node app.js‘ from this directory and the result will be outputted to ‘
    ↪ appendix_source_code.tex‘.
```

appendix_generator/source_code/app.js

```
import fs from "fs/promises";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const root_path = path.resolve(__dirname, "../../");
const output_path = path.join(__dirname, "appendix_source_code.tex");

const ignored_paths = new Set([
    "uoe",
    "pnpm-lock.yaml",
    ".gitignore",
    ".vscodeignore",
    "json.hpp",
]);

const ignored_extensions = new Set([
    ".pdf",
    ".png",
    ".tex",
    ".ico",
]);

const escape_latex = (text) => {
    return (text
        .replace(/\\/g, "\\textbackslash{}")
        .replace(/\^/g, "\\textasciicircum{}")
        .replace(/~/g, "\\textasciitilde{}")
        .replace(/[{}&%$#_]/g, "\\$&")
    );
};

const escape_verbatim = (text) => {
    // don't ask me how many hours it took to figure this out

    return (text
        .replace(/@/g, "@\\char\"40@")
        .replace(/\\end\{lstlisting\}/g, "@\\textbackslash{}end\{lstlisting\}@")
        .replace(/★/g, "@$\\bigstar$@")
        .replace(/▽/g, "@$\\triangledown$@")
        .replace(/▷/g, "@$\\triangleright$@")
        .replace(/•/g, "@$\\bullet$@")
        .replace(/→/g, "@$\\rightarrow$@")

        // .replace(/\\/g, "(*@\\textbackslash{}@*)")
        // .replace(/%/g, "(*@\\%@*)");
    );
};

const parse_gitignore = async (path) => {
```

```
    const patterns = [];

    try {
        var content = await fs.readFile(path, "utf8");
    } catch (err) {
        console.log("No .gitignore found at:", path);
        return [];
    }

    const lines = content.split("\n");

    for (let line of lines) {
        line = line.trim();

        if ((false
            || line === ""
            || line.startsWith("#")
        )) {
            continue;
        }

        const is_negated = line.startsWith("!");

        if (is_negated) {
            line = line.substring(1);
        }

        patterns.push({ pattern: line, is_negated });
    }

    return patterns;
};

const pattern_to_regex = (pattern) => {
    const is_root = pattern.startsWith("/");

    if (is_root) {
        pattern = pattern.substring(1);
    }

    const is_directory_only = pattern.endsWith("/");

    if (is_directory_only) {
        pattern = pattern.substring(0, pattern.length - 1);
    }

    let regex_str = "";

    for (let i = 0; i < pattern.length; i++) {
        const char = pattern[i];

        if (char === "*") {
            if ((true
                && i + 1 < pattern.length
                && pattern[i + 1] === "*"
            )) {
                regex_str += ".*";
                i++;
                if ((true
                    && i + 1 < pattern.length
                    && pattern[i + 1] === "/"
                )) {
                    i++;
                }
            } else {
                regex_str += "[^/]*";
            }
        } else if (char === "?") {
            regex_str += "[^/]";
        } else if ((false
```

```
            || ".*+?^${}()|[]\\".includes(char)
        )) {
            regex_str += "\\" + char;
        } else {
            regex_str += char;
        }
    }

    if (is_root) {
        regex_str = "^" + regex_str;
    } else {
        regex_str = "(^|/)" + regex_str;
    }

    if (is_directory_only) {
        regex_str += "(/|$)";
    } else {
        regex_str += "(/.*|$)";
    }

    return new RegExp(regex_str);
};

const load_gitignore_rules = async (base_dir) => {
    const rules_by_dir = new Map();

    const analyze_directory = async (dir, rel_path) => {
        rel_path ??= "";

        const gitignore_path = path.join(dir, ".gitignore");
        const patterns = await parse_gitignore(gitignore_path);

        if (patterns.length > 0) {
            const normalized_rel_path = rel_path.replace(/\\/g, "/");
            rules_by_dir.set(normalized_rel_path, patterns);
        }

        const entries = await fs.readdir(dir, { withFileTypes: true });

        for (const entry of entries) {
            if ((true
                && entry.isDirectory()
                && entry.name !== ".git"
            )) {
                const new_rel_path = rel_path ? path.join(rel_path, entry.name) : entry.
    ↪ name;
                await analyze_directory(path.join(dir, entry.name), new_rel_path);
            }
        }
    };

    await analyze_directory(base_dir);

    return rules_by_dir;
};

const is_ignored = (file_path, _is_directory, rules_by_dir) => {
    const normalized = file_path.replace(/\\/g, "/");
    const segments = normalized.split("/");

    let is_ignored = false;

    for (let i = 0; i <= segments.length; i++) {
        const dir_path = segments.slice(0, i).join("/");
        const remaining = segments.slice(i).join("/");

        const rules = rules_by_dir.get(dir_path === "" ? "" : dir_path) ?? [];

        for (const { pattern, is_negated } of rules) {
            const regex = pattern_to_regex(pattern);
```

```
            if (regex.test(remaining)) {
                is_ignored = !is_negated;
            }
        }
    }

    if ((false
        || normalized === ".git"
        || normalized.startsWith(".git/")
    )) {
        is_ignored = true;
    }

    for (const part of segments) {
        if (ignored_paths.has(part)) {
            is_ignored = true;
            break;
        }
    }

    for (const ext of ignored_extensions) {
        if (normalized.endsWith(ext)) {
            is_ignored = true;
            break;
        }
    }

    return is_ignored;
};

const collect_files = async (dir, base_path, rules_by_dir) => {
    const results = [];
    const entries = await fs.readdir(dir, { withFileTypes: true });

    for (const entry of entries) {
        const full_path = path.join(dir, entry.name);
        const rel_path = base_path ? path.join(base_path, entry.name) : entry.name;

        if (is_ignored(rel_path, entry.isDirectory(), rules_by_dir)) {
            continue;
        }

        if (entry.isDirectory()) {
            results.push(...await collect_files(full_path, rel_path, rules_by_dir));
        } else if (entry.isFile()) {
            results.push(rel_path);
        }
    }

    return results;
};

const generate_latex = async () => {
    console.log("Collecting all files from:", root_path);
    console.log("Loading .gitignore rules...");

    const rules_by_dir = await load_gitignore_rules(root_path);
    console.log(`Loaded .gitignore rules from ${rules_by_dir.size} directories.`);

    const files = await collect_files(root_path, "", rules_by_dir);
    files.sort();

    console.log(`Found ${files.length} files to include in this report's appendix.`);

    let latex = "";
    latex += "% This is an auto-generated appendix using my appendix generator script\n
    ↪ ";
    latex += "% Date of generation: " + new Date().toISOString() + "\n\n";
    latex += "\\newcommand{\\tighttt}{\\fontsize{8pt}{8pt}\\selectfont\\ttfamily\\
```

```
    ↪ pdfliteral direct{-0.3 Tc}}\n\n";
    latex += "\\lstset{\n";
    latex += "  breaklines=true,\n";
    latex += "  breakatwhitespace=false,\n";
    latex += "  postbreak={\\mbox{\\textcolor{red}{$\\hookrightarrow$}\\space}},\n";
    latex += "  columns=fixed,\n";
    latex += "  keepspaces=true,\n";
    latex += "  showstringspaces=false,\n";
    latex += "  tabsize=4,\n";
    latex += "  basicstyle={\\tighttt},\n";
    latex += "}\n\n";

    let i = 0;

    for (const file_path of files) {
        console.log(`Processing: ${file_path}`);

        const full_path = path.join(root_path, file_path);

        const content = await fs.readFile(full_path, "utf8");

        const normalized_path = file_path.replace(/\\/g, "/");
        const escaped_path = escape_latex(normalized_path);
        const escaped_content = escape_verbatim(content);

        latex += "\\vspace{0.5cm}\n";
        latex += "\\noindent\\colorbox{lightgray}{\\parbox{\\dimexpr\\textwidth-2\\
    ↪ fboxsep}{\\small\\textbf{" + escaped_path + "}}}\n\n";
        latex += "\\vspace{0.2cm}\n\n";
        latex += "\\begin{lstlisting}[language={}, escapechar=@, breaklines=true,
    ↪ breakatwhitespace=false]\n";

        // change to limit how many files are outputted for debugging purposes.
        if (i <= 999) { // 17
            latex += escaped_content;
        }

        if (!escaped_content.endsWith("\n")) {
            latex += "\n";
        }

        latex += "\\endlstlisting\n\n";

        i++;
    }

    await fs.writeFile(output_path, latex, "utf8");
    console.log(`\nLatex appendix has been succesfully outputted to: ${output_path}`);
    console.log(`Total number of files: ${files.length}`);
};

generate_latex();
```

appendix_generator/source_code/package.json

```
{
    "type": "module"
}
```

compiler/README.md

```
# Compiler

This directory contains the main entrypoint to the compiler.
```

```
The compilation process is separated into two phases:

- ‘frontend‘ (JavaScript): Parsing + initial analysis and transformations.
- ‘backend‘ (C++): Remaining analysis and transformations + codegen using LLVM.

While I intended to write the ‘backend‘ in JavaScript, the primary interface for LLVM is
    ↪ ‘C++‘-based, so I’m writing this phase in ‘C++‘ for simplicity.

## Setup

- If on Windows, use WSL, this is mandatory. Otherwise use Linux.
- Install ‘nvm‘ ([Node Version Manager](https://github.com/nvm-sh)).
- Install the latest version of Node.js (^25.6.1) via ‘nvm install node‘ (or install
    ↪ Node.js using an alternative method).
- Install the latest version of ‘pnpm‘ (^10.24.0) via ‘npm install -g pnpm‘.

## Run

- ‘node ec.js compile <src> <dest>‘ where ‘<dest>‘ configurese where the output
    ↪ artifacts go.
```

compiler/backend/Dockerfile

```
FROM ubuntu:24.04

RUN apt-get update

RUN apt-get install -y \
    make \
    # LLVM toolchain
    llvm-17 \
    llvm-17-dev \
    clang-17 \
    lld-17 \
    # One can add cross-compilation dependencies here in future
    && rm -rf /var/lib/apt/lists/*

RUN ln -sf /usr/bin/clang-17 /usr/bin/clang && \
    ln -sf /usr/bin/clang++-17 /usr/bin/clang++

ENV LLVM_CONFIG=llvm-config-17

WORKDIR /app

COPY . .

RUN mkdir -p /app/out /volume/out

RUN make -j$(nproc) all

RUN chmod +x live.sh

VOLUME ["/volume/out"]
VOLUME ["/volume/in"]

# Final steps that are dynamic (using live volumes) are delegated to this dedicated
    ↪ shell script.
CMD ["bash", "./live.sh"]
```

compiler/backend/Makefile

```
CXX                 := clang++
AR                  := ar
```

```
CXXFLAGS            := -std=c++20 -O2 -Wall -Wextra -Wpedantic -Wno-unused-parameter
LDFLAGS             :=

LLVM_COMPONENTS     := core support irreader target mc

LLVM_CXXFLAGS       := $(shell $(LLVM_CONFIG) --cxxflags)
LLVM_LDFLAGS        := $(shell $(LLVM_CONFIG) --ldflags)
LLVM_LIBS           := $(shell $(LLVM_CONFIG) --libs $(LLVM_COMPONENTS))
SYSTEM_LIBS         := $(shell $(LLVM_CONFIG) --system-libs)

BIN                 := bin/backend

src_files := $(wildcard src/*.cpp)
obj_files := $(patsubst src/%.cpp,build/%.o,$(src_files))

build/%.o: src/%.cpp | build
    $(CXX) $(LLVM_CXXFLAGS) $(CXXFLAGS) -fexceptions -c $< -o $@

$(BIN): $(obj_files) | bin
    $(CXX) -o $@ $^ $(LDFLAGS) $(LLVM_LDFLAGS) $(LLVM_LIBS) $(SYSTEM_LIBS)

build:
    mkdir -p build

bin:
    mkdir -p bin

.PHONY: all
all: $(BIN)
```

compiler/backend/README.md

```
# Compiler Backend

Language: C++.

## Requirements

- Docker
- Linux or WSL (Windows)

## Build & Run

**Linux or WSL**:

Build:

‘‘‘
chmod +x ./host/build.sh
./host/build.sh
‘‘‘

Run:

‘‘‘
chmod +x ./host/run.sh
./host/run.sh <path_to_module>
‘‘‘

**Other platforms**

Good luck trying, it won’t be possible.
```

compiler/backend/host/build.sh

```
docker build -t davidcallanan--mu-fyp--compiler-backend .
```

compiler/backend/host/buildrun.sh

```
# i've stopped keeping this script up-to-date, it's easier to keep the two scripts
    ↪ separate because running requires CLI args

./host/build.sh && \
./host/run.sh
```

compiler/backend/host/run.sh

```
module_path="$1"

if [ -z "$module_path" ]; then
    echo "Usage: $0 <module_path>"
    exit 1
fi

mkdir -p "$(pwd)/in"
mkdir -p "$(pwd)/out"

cp "${module_path}/frontend.out.json" "$(pwd)/in/frontend.out.json"

docker run \
    --init \
    --rm \
    -it \
    -v "$(pwd)/in:/volume/in" \
    -v "$(pwd)/out:/volume/out" \
    davidcallanan--mu-fyp--compiler-backend
```

compiler/backend/live.sh

```
shopt -s nullglob

/app/bin/backend || exit 1

if [ -f /app/out/hello.o ]; then
  echo "Shell script is linking..."
  clang /app/out/hello.o -o /app/out/hello
  chmod +x /app/out/hello
  echo "Executable ready, let's run it for fun"
  echo "=#=#=#=#=#=#=#=#=#=#=#=#=#=#=#"
  /app/out/hello
  echo "=#=#=#=#=#=#=#=#=#=#=#=#=#=#=#"
fi

files=(/app/out/*)
if (( ${#files[@]} )); then
  cp -r "${files[@]}" /volume/out/
fi
```

compiler/backend/src/access_member.cpp

```
#include <cstdio>
#include <cstdlib>
#include <memory>
```

```
#include <string>
#include "llvm/IR/DerivedTypes.h"
#include "access_member.hpp"
#include "t_types.hpp"
#include "t_smooth.hpp"
#include "get_underlying_type.hpp"
#include "llvm_to_smooth.hpp"
#include "is_type_singletonish.hpp"
#include "evaluate_singletonish.hpp"
#include "smooth_type.hpp"
#include "is_structval_leaf_physical.hpp"

Smooth access_member(
    IrGenCtx& igc,
    std::shared_ptr<SmoothStructval> target_smooth,
    const std::string& sym
) {
    if (auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&target_smooth->type)) {
        const auto& v_map = *p_v_map;

        std::string sym_key = ":" + sym;

        if (v_map->sym_inputs.find(sym_key) == v_map->sym_inputs.end()) {
            fprintf(stderr, "Symbol %s not really available here", sym.c_str());
            exit(1);
        }

        const Type& unclear_type = *v_map->sym_inputs.at(sym_key);
        Type sym_type = get_underlying_type(unclear_type);

        if (is_type_singletonish(unclear_type)) {
            return evaluate_singletonish(igc, unclear_type);
        }

        // i know this logic is terrible but performance is not a concern for me.

        size_t field_index = is_structval_leaf_physical(target_smooth) ? 1 : 0;

        for (const auto& [sym_name, sym_type] : v_map->sym_inputs) {
            if (sym_name == sym_key) {
                break;
            }

            if (is_type_singletonish(*sym_type)) {
                continue;
            }

            field_index++;
        }

        llvm::Value* extracted = igc.builder.CreateExtractValue(target_smooth->value,
    ↪ field_index);

        // pointers are typically disallowed on their own, and must be wrapped into a
    ↪ leaf map.
        // exception is made for syms of maps to prevent infinite recursion.
        // this is why we must deal with raw pointer here and wrap it back up.
        // we gradually wrap up pointers as they are used, lazily.

        if (auto p_v_pointer = std::get_if<std::shared_ptr<TypePointer>>(&sym_type)) {
            fprintf(stderr, "is it even getting here. %s\n", sym.c_str());

            llvm::StructType* wrapped_pointer = llvm::StructType::get(igc.context, llvm
    ↪ ::ArrayRef<llvm::Type*>{ extracted->getType() });
            llvm::Value* final_pointer = llvm::UndefValue::get(wrapped_pointer);
            final_pointer = igc.builder.CreateInsertValue(final_pointer, extracted, 0);

            auto actual_map = std::make_shared<TypeMap>(TypeMap{
                unclear_type,
                std::nullopt,
```

```
                nullptr,
                nullptr,
                {},
                {},
            });

            return std::make_shared<SmoothStructval>(SmoothStructval{
                actual_map,
                final_pointer, // todo: is this the actual struct
                true,
                std::make_shared<SmoothPointer>(SmoothPointer{
                    unclear_type,
                    extracted,
                }),
                nullptr, // this is problematic.
            });
        }

        return llvm_to_smooth(igc, unclear_type, extracted);
    } else {
        fprintf(stderr, "Impossible to call a non-map with a symbol, what are you doing
    ↪ ?\n");
        exit(1);
    }
}
```

compiler/backend/src/access_member.hpp

```
#pragma once

#include <memory>
#include <string>
#include "t_types_fwd.hpp"
#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_smooth.hpp"

Smooth access_member(
    IrGenCtx& igc,
    std::shared_ptr<SmoothStructval> target_smooth,
    const std::string& sym
);
```

compiler/backend/src/access_variable.cpp

```
#include <cstdio>
#include <cstdlib>
#include <optional>
#include <string>
#include "access_variable.hpp"
#include "t_types.hpp"
#include "t_smooth.hpp"
#include "create_value_symbol_table.hpp"
#include "llvm_to_smooth.hpp"
#include "is_type_singletonish.hpp"
#include "evaluate_singletonish.hpp"

Smooth access_variable(
    IrGenCtx& igc,
    const Type& node
) {
    std::string target_name;
    std::shared_ptr<Type>* underlying_type = nullptr;
```

```
    if (auto p = std::get_if<std::shared_ptr<TypeVarAccess>>(&node)) {
        target_name = (*p)->target_name;
        underlying_type = &(*p)->underlying_type;
    } else if (auto p = std::get_if<std::shared_ptr<TypeVarWalrus>>(&node)) {
        target_name = (*p)->name;
        underlying_type = &(*p)->underlying_type;
    } else if (auto p = std::get_if<std::shared_ptr<TypeVarAssign>>(&node)) {
        target_name = (*p)->name;
        underlying_type = &(*p)->underlying_type;
    } else {
        fprintf(stderr, "only TypeVarWalrus, TypeVarAssign, and TypeVarAccess can access
    ↪  a variable!\n");
        exit(1);
    }

    std::string var_name = "m_" + target_name;
    std::optional<ValueSymbolTableEntry> o_entry = igc.value_table->get(var_name);

    if (!o_entry.has_value()) {
        std::string sym_var_name = "ms_:" + target_name;
        o_entry = igc.value_table->get(sym_var_name);

        if (!o_entry.has_value()) {
            fprintf(stderr, "This variable %s was not actually present in our value
    ↪ table\n", target_name.c_str());
            exit(1);
        }
    }

    const ValueSymbolTableEntry& entry = o_entry.value();

    *underlying_type = std::make_shared<Type>(entry.type);

    if (is_type_singletonish(entry.type)) {
        return evaluate_singletonish(igc, entry.type);
    }

    llvm::Value* loaded = igc.builder.CreateLoad(
        entry.ir_type,
        entry.alloca_ptr
    );

    return llvm_to_smooth(igc, entry.type, loaded);
}
```

compiler/backend/src/access_variable.hpp

```
#pragma once

#include "t_types_fwd.hpp"
#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_smooth.hpp"

Smooth access_variable(
    IrGenCtx& igc,
    const Type& node
);
```

compiler/backend/src/build_compare.cpp

```
#include <cstdio>
#include <cstdlib>
#include <string>
```

```
#include "build_compare.hpp"
#include "build_compare_int.hpp"
#include "build_compare_float.hpp"
#include "build_compare_struct.hpp"

llvm::Value* build_compare(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const std::
    ↪ string& operator_) {
    llvm::Type* type = a->getType();

    if (type->isIntegerTy()) {
        return build_compare_int(igc, a, b, operator_);
    }

    if (type->isFloatingPointTy()) {
        return build_compare_float(igc, a, b, operator_);
    }

    if (type->isStructTy()) {
        return build_compare_struct(igc, a, b, operator_);
    }

    fprintf(stderr, "Cannot compare something that is not numeric or structish\n");
    exit(1);
}
```

compiler/backend/src/build_compare.hpp

```
#pragma once

#include <string>
#include "llvm/IR/Value.h"
#include "t_ctx.hpp"

llvm::Value* build_compare(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const std::
    ↪ string& operator_);
```

compiler/backend/src/build_compare_float.cpp

```
#include <cstdio>
#include <cstdlib>
#include <string>
#include "build_compare_float.hpp"

llvm::Value* build_compare_float(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const
    ↪ std::string& operator_) {
    if (operator_ == "==") return igc.builder.CreateFCmpOEQ(a, b);
    if (operator_ == "!=") return igc.builder.CreateFCmpONE(a, b);
    if (operator_ == "<")  return igc.builder.CreateFCmpOLT(a, b);
    if (operator_ == ">")  return igc.builder.CreateFCmpOGT(a, b);
    if (operator_ == "<=") return igc.builder.CreateFCmpOLE(a, b);
    if (operator_ == ">=") return igc.builder.CreateFCmpOGE(a, b);

    fprintf(stderr, "Bizarre operator %s\n", operator_.c_str());
    exit(1);
}
```

compiler/backend/src/build_compare_float.hpp

```
#pragma once

#include <string>
```

```
#include "llvm/IR/Value.h"
#include "t_ctx.hpp"

llvm::Value* build_compare_float(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const
    ↪ std::string& operator_);
```

compiler/backend/src/build_compare_int.cpp

```
#include "build_compare_int.hpp"

#include <cstdio>
#include <cstdlib>
#include <string>

llvm::Value* build_compare_int(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const std
    ↪ ::string& operator_) {
    // todo: what about distinguishing between signed and unsigned.

    uint32_t a_bits = a->getType()->getIntegerBitWidth();
    uint32_t b_bits = b->getType()->getIntegerBitWidth();

    if (a_bits < b_bits) {
        a = igc.builder.CreateZExt(a, b->getType());
    } else if (b_bits < a_bits) {
        b = igc.builder.CreateZExt(b, a->getType());
    }

    if (operator_ == "==") return igc.builder.CreateICmpEQ(a, b);
    if (operator_ == "!=") return igc.builder.CreateICmpNE(a, b);
    if (operator_ == "<")  return igc.builder.CreateICmpULT(a, b);
    if (operator_ == ">")  return igc.builder.CreateICmpUGT(a, b);
    if (operator_ == "<=") return igc.builder.CreateICmpULE(a, b);
    if (operator_ == ">=") return igc.builder.CreateICmpUGE(a, b);

    fprintf(stderr, "Unhandled operator %s\n", operator_.c_str());
    exit(1);
}
```

compiler/backend/src/build_compare_int.hpp

```
#pragma once

#include <string>
#include "llvm/IR/Value.h"
#include "t_ctx.hpp"

llvm::Value* build_compare_int(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const std
    ↪ ::string& operator_);
```

compiler/backend/src/build_compare_struct.cpp

```
#include <cstdio>
#include <cstdlib>
#include <string>
#include "llvm/IR/DerivedTypes.h"
#include "llvm/IR/Constants.h"
#include "build_compare_struct.hpp"
#include "build_compare.hpp"

// this is probably really inefficient, but i don't have time to worry.
```

```
llvm::Value* build_compare_struct(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const
    ↪ std::string& operator_) {
    llvm::StructType* struct_type = llvm::cast<llvm::StructType>(a->getType());
    unsigned num_fields = struct_type->getNumElements();

    llvm::Type* i1 = llvm::Type::getInt1Ty(igc.context);

    if (operator_ == "==" || operator_ == "!=") {
        llvm::Value* result = llvm::ConstantInt::get(i1, 1);

        for (unsigned i = 0; i < num_fields; i++) {
            llvm::Value* field_a = igc.builder.CreateExtractValue(a, i);
            llvm::Value* field_b = igc.builder.CreateExtractValue(b, i);

            result = igc.builder.CreateAnd(result, build_compare(igc, field_a, field_b,
    ↪ "=="));
        }

        if (operator_ == "!=") {
            return igc.builder.CreateNot(result);
        }

        return result;
    }

    // we implement lexicographical ordering here.

    bool should_tiebreak = operator_ == "<=" || operator_ == ">="; // if not strict,
    ↪ then equality is a tiebreaker.
    llvm::Value* result = llvm::ConstantInt::get(i1, should_tiebreak ? 1 : 0);
    std::string op_strict = operator_ == "<" || operator_ == "<=" ? "<" : ">";

    // i'm trusting that the llvm optimizer pass will convert this to a binary tree or
    ↪ something else smart.
    // this is only a prototype compiler - optimizations like this can be hand-written
    ↪ at a later stage if neccessary.

    for (unsigned i = num_fields - 1; i < num_fields; i--) { // unsigned underflow
    ↪ intentionally handled.
        llvm::Value* field_a = igc.builder.CreateExtractValue(a, i);
        llvm::Value* field_b = igc.builder.CreateExtractValue(b, i);
        llvm::Value* comp_strictful = build_compare(igc, field_a, field_b, op_strict);
        llvm::Value* comp_equaliful = build_compare(igc, field_a, field_b, "==");

        result = igc.builder.CreateSelect(
            comp_strictful,
            llvm::ConstantInt::get(i1, 1),
            igc.builder.CreateSelect(
                comp_equaliful,
                result,
                llvm::ConstantInt::get(i1, 0)
            )
        );
    }

    return result;
}
```

compiler/backend/src/build_compare_struct.hpp

```
#pragma once

#include <string>
#include "llvm/IR/Value.h"
#include "t_ctx.hpp"

llvm::Value* build_compare_struct(IrGenCtx& igc, llvm::Value* a, llvm::Value* b, const
```

```
    ↪ std::string& operator_);
```

compiler/backend/src/create_bundle_registry.cpp

```
#include "create_bundle_registry.hpp"

uint64_t BundleRegistry::install(Bundle bundle) {
    uint64_t id = _next_id++;
    _bundles[id] = bundle;
    return id;
}

Bundle* BundleRegistry::get(uint64_t bundle_id) {
    auto it = _bundles.find(bundle_id);

    if (it == _bundles.end()) {
        return nullptr;
    }

    return &it->second;
}

BundleRegistry create_bundle_registry() {
    return BundleRegistry{};
}
```

compiler/backend/src/create_bundle_registry.hpp

```
#pragma once

#include <cstdint>
#include <map>
#include "t_bundles_fwd.hpp"

class BundleRegistry {
private:
    uint64_t _next_id = 0;
    std::map<uint64_t, Bundle> _bundles;

public:
    uint64_t install(Bundle bundle);
    Bundle* get(uint64_t bundle_id);
};

BundleRegistry create_bundle_registry();
```

compiler/backend/src/create_dummy_igc.cpp

```
#include "create_value_symbol_table.hpp"

#include <memory>
#include "llvm/IR/BasicBlock.h"
#include "llvm/IR/DerivedTypes.h"
#include "llvm/IR/Function.h"
#include "llvm/IR/IRBuilder.h"
#include "create_dummy_igc.hpp"

DummyIgc create_dummy_igc(IrGenCtx& base) {
    llvm::FunctionType* dummy_func_type = llvm::FunctionType::get(llvm::Type::getVoidTy(
    ↪ base.context), false);
    llvm::Function* dummy_func = llvm::Function::Create(dummy_func_type, llvm::Function
    ↪ ::PrivateLinkage, "__ec_dummy", base.module);
```

```
    llvm::BasicBlock* dummy_block = llvm::BasicBlock::Create(base.context, "entry",
    ↪ dummy_func);

    auto dummy_builder = std::make_unique<llvm::IRBuilder<>>(base.context);
    dummy_builder->SetInsertPoint(dummy_block);

    IrGenCtx igc = {
        base.context,
        base.module,
        *dummy_builder,
        std::make_shared<ValueSymbolTable>(create_value_symbol_table()),
        base.puts_func,
        base.log_data_func,
        base.log_data_deref_func,
        nullptr,
        base.toc,
    };

    return DummyIgc{
        igc,
        std::move(dummy_builder),
        dummy_func,
    };
}
```

compiler/backend/src/create_dummy_igc.hpp

```
#pragma once

#include <memory>
#include "llvm/IR/Function.h"
#include "llvm/IR/IRBuilder.h"
#include "t_ctx.hpp"
#include "create_value_symbol_table.hpp"

struct DummyIgc {
    IrGenCtx igc;
    std::unique_ptr<llvm::IRBuilder<>> dummy_builder;
    llvm::Function* dummy_func;
};

DummyIgc create_dummy_igc(IrGenCtx& base);
```

compiler/backend/src/create_type_symbol_table.cpp

```
#include "create_type_symbol_table.hpp"
#include <cstdio>
#include <cstdlib>

std::vector<std::string> TypeSymbolTable::_split_trail(const std::string &trail) {
    std::vector<std::string> segments;
    size_t start = 0;
    size_t end = trail.find("::");

    while (end != std::string::npos) {
        segments.push_back(trail.substr(start, end - start));
        start = end + 2;
        end = trail.find("::", start);
    }

    segments.push_back(trail.substr(start));

    return segments;
}
```

```
TypeSymbolTable::TypeSymbolTable() {
    _root = std::make_unique<MapEntry>();

    const std::vector<std::string> float_types = {"f16", "f32", "f64", "f128"};

    for (const auto &leaf_type : float_types) {
        auto map = std::make_shared<TypeMap>();

        auto rotten = std::make_shared<TypeRotten>();
        rotten->type_str = leaf_type;
        map->leaf_type = Type(rotten);

        map->call_input_type = nullptr;
        map->call_output_type = nullptr;

        auto entry = std::make_unique<MapEntry>();
        entry->type_value = UnderlyingType{map};
        _root->children[leaf_type] = std::move(entry);
    }

    {
        auto v_enum = std::make_shared<TypeEnum>();
        v_enum->syms = {"false", "true"};

        auto entry = std::make_unique<MapEntry>();
        entry->type_value = UnderlyingType{v_enum};
        _root->children["bool"] = std::move(entry);
    }
}

void TypeSymbolTable::set(const std::string &trail, const UnderlyingType &value) {
    auto segments = _split_trail(trail);

    if (segments.empty()) {
        fprintf(stderr, "Empty trail was provided for some reason\n");
        exit(1);
    }

    MapEntry *curr_entry = _root.get();

    for (const auto &segment : segments) {
        if (curr_entry->children.find(segment) == curr_entry->children.end()) {
            curr_entry->children[segment] = std::make_unique<MapEntry>();
        }

        curr_entry = curr_entry->children[segment].get();
    }

    curr_entry->type_value = value;
}

std::optional<UnderlyingType> TypeSymbolTable::get(const std::string &trail) {
    auto segments = _split_trail(trail);

    if (segments.empty()) {
        fprintf(stderr, "Empty trail was provided for some reason\n");
        exit(1);
    }

    MapEntry *curr_entry = _root.get();

    for (size_t i = 0; i < segments.size(); i++) {
        const auto &segment = segments[i];

        if (curr_entry->children.find(segment) == curr_entry->children.end()) {
            if (i == segments.size() - 1) {
                break;
            }
```

```
            return std::nullopt;
        }

        curr_entry = curr_entry->children[segment].get();
    }

    const std::string &key = segments[segments.size() - 1];

    if (curr_entry->type_value.has_value()) {
        return curr_entry->type_value;
    }

    if (true
        && curr_entry == _root.get()
        && segments.size() == 1
        && key.length() >= 2
        && (key[0] == 'i' || key[0] == 'u')
        && key[1] >= '1'
        && key[1] <= '9'
    ) {
        bool is_valid_intish = true;

        for (size_t j = 2; j < key.length(); j++) {
            if (!isdigit(key[j])) {
                is_valid_intish = false;
                break;
            }
        }

        if (is_valid_intish) {
            auto map = std::make_shared<TypeMap>();

            auto rotten = std::make_shared<TypeRotten>();
            rotten->type_str = key;
            map->leaf_type = Type(rotten);

            map->call_input_type = nullptr;
            map->call_output_type = nullptr;

            auto entry = std::make_unique<MapEntry>();
            entry->type_value = UnderlyingType{map};

            std::optional<UnderlyingType> result = entry->type_value;
            curr_entry->children[key] = std::move(entry);

            return result;
        }
    }

    return std::nullopt;
}

TypeSymbolTable create_type_symbol_table() {
    return TypeSymbolTable();
}
```

compiler/backend/src/create_type_symbol_table.hpp

```
#pragma once

#include <map>
#include <optional>
#include <string>
#include <vector>
#include <memory>
#include "t_types.hpp"
#include "dependencies/json.hpp"
```

```
using json = nlohmann::json;

class TypeSymbolTable {
private:
    struct MapEntry {
        std::map<std::string, std::unique_ptr<MapEntry>> children;
        std::optional<UnderlyingType> type_value;
    };

    std::unique_ptr<MapEntry> _root;

    std::vector<std::string> _split_trail(const std::string& trail);

public:
    TypeSymbolTable();

    void set(const std::string& trail, const UnderlyingType& value);

    std::optional<UnderlyingType> get(const std::string& trail);
};

TypeSymbolTable create_type_symbol_table();
```

compiler/backend/src/create_value_symbol_table.cpp

```
#include "create_value_symbol_table.hpp"
#include <optional>

static std::string to_scope_id(int value) {
    const std::string alphabet = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    const int alphabet_size = static_cast<int>(alphabet.size());
    const int scope_length = 6;
    std::string scope_id(scope_length, 'a');
    int remaining = value;

    for (int i = scope_length - 1; i >= 0; i--) {
        int digit = remaining % alphabet_size;
        remaining = remaining / alphabet_size;
        scope_id[i] = alphabet[static_cast<size_t>(digit)];
    }

    return scope_id;
}

static std::string next_scope_id() {
    static int scope_id = 0;
    return to_scope_id(scope_id++);
}

ValueSymbolTable::ValueSymbolTable(ValueSymbolTable* parent)
    : _parent(parent),
    _scope_id(next_scope_id()) {
}

void ValueSymbolTable::set(const std::string& name, const ValueSymbolTableEntry& entry)
    ↪ {
    _values[name] = entry;
}

std::optional<ValueSymbolTableEntry> ValueSymbolTable::get(const std::string& name) {
    auto it = _values.find(name);

    if (it == _values.end()) {
        if (_parent == nullptr) {
            return std::nullopt;
        }
```

```
        return _parent->get(name);
    }

    return it->second;
}

const std::string& ValueSymbolTable::scope_id() const {
    return _scope_id;
}

ValueSymbolTable create_value_symbol_table() {
    return ValueSymbolTable(nullptr);
}

ValueSymbolTable create_value_symbol_table(ValueSymbolTable* parent) {
    return ValueSymbolTable(parent);
}
```

compiler/backend/src/create_value_symbol_table.hpp

```
#pragma once

#include <map>
#include <string>
#include <optional>
#include "llvm/IR/Value.h"
#include "llvm/IR/Type.h"
#include "t_types_fwd.hpp"

struct ValueSymbolTableEntry {
    llvm::Value* alloca_ptr;
    llvm::Type* ir_type;
    Type type;
    bool is_mut;
};

class ValueSymbolTable {
private:
    std::map<std::string, ValueSymbolTableEntry> _values;
    ValueSymbolTable* _parent;
    std::string _scope_id;

public:
    ValueSymbolTable(ValueSymbolTable* parent);

    void set(const std::string& name, const ValueSymbolTableEntry& entry);

    std::optional<ValueSymbolTableEntry> get(const std::string& name);
    const std::string& scope_id() const;
};

ValueSymbolTable create_value_symbol_table();
ValueSymbolTable create_value_symbol_table(ValueSymbolTable* parent);
```

compiler/backend/src/demote_underlying.cpp

```
#include "demote_underlying.hpp"

Type demote_underlying(const UnderlyingType& type) {
    return std::visit([](auto&& v) -> Type { return v; }, type);
}
```

compiler/backend/src/demote_underlying.hpp

```
#pragma once

#include "t_types_fwd.hpp"

Type demote_underlying(const UnderlyingType& type);
```

compiler/backend/src/destroy_dummy_igc.cpp

```
#include "destroy_dummy_igc.hpp"

void destroy_dummy_igc(DummyIgc& dummy) {
    dummy.dummy_func->eraseFromParent();
}
```

compiler/backend/src/destroy_dummy_igc.hpp

```
#pragma once

#include "create_dummy_igc.hpp"

void destroy_dummy_igc(DummyIgc& dummy);
```

compiler/backend/src/evaluate_hardval.cpp

```
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <variant>
#include <string>
#include "evaluate_hardval.hpp"
#include "t_hardval.hpp"
#include "t_types.hpp"
#include "t_smooth.hpp"
#include "get_underlying_type.hpp"
#include "llvm_to_smooth.hpp"
#include "llvm/ADT/APInt.h"
#include "llvm/ADT/APFloat.h"
#include "llvm/IR/Constants.h"

Smooth evaluate_hardval(
    IrGenCtx& igc,
    const Hardval& hardval,
    Type type
) {
    Type underlying = get_underlying_type(type);

    std::string type_str = "";

    if (auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&underlying)) {
        type_str = (*p_v_rotten)->type_str;
    }

    if (std::holds_alternative<std::shared_ptr<HardvalInteger>>(hardval)) {
        const auto& p_v_int = std::get<std::shared_ptr<HardvalInteger>>(hardval);
        const std::string& value_str = p_v_int->value;

        int bits_needed = 0;

        if (!type_str.empty()) {
```

```
        if (type_str[0] != 'i' && type_str[0] != 'u') {
            fprintf(stderr, "This received .type is not currently implemented, got %
↪ s\n", type_str.c_str());
            exit(1);
        }

        bits_needed = std::stoi(type_str.substr(1));
    } else {
        bool is_negative = !value_str.empty() && value_str[0] == '-';
        std::string digits = is_negative ? value_str.substr(1) : value_str;

        // BCD encoding reminds us that 4 bits is conservative for each digit.
        uint32_t scratchy_bits = (uint32_t)(digits.size() * 4) + 1;

        if (is_negative) {
            llvm::APInt ap_value(scratchy_bits, digits.c_str(), 10);
            bits_needed = ap_value.getActiveBits() + 1;
        } else {
            llvm::APInt ap_value(scratchy_bits, digits.c_str(), 10);
            bits_needed = ap_value.getActiveBits();

            if (bits_needed == 0) {
                bits_needed = 1;
            }
        }
    }

    llvm::APInt ap_int(bits_needed, value_str.c_str(), 10);
    llvm::Value* const_value = llvm::ConstantInt::get(igc.context, ap_int);

    return llvm_to_smooth(igc, type, const_value);
}

if (std::holds_alternative<std::shared_ptr<HardvalFloat>>(hardval)) {
    const auto& p_v_float = std::get<std::shared_ptr<HardvalFloat>>(hardval);

    llvm::Type* float_type;

    if (!type_str.empty()) {
        if (type_str[0] != 'f') {
            fprintf(stderr, "This received .type is not currently implemented, got %
↪ s\n", type_str.c_str());
            exit(1);
        }

        int bit_width = std::stoi(type_str.substr(1));

        if (bit_width == 16) {
            float_type = llvm::Type::getHalfTy(igc.context);
        } else if (bit_width == 32) {
            float_type = llvm::Type::getFloatTy(igc.context);
        } else if (bit_width == 64) {
            float_type = llvm::Type::getDoubleTy(igc.context);
        } else if (bit_width == 128) {
            float_type = llvm::Type::getFP128Ty(igc.context);
        } else {
            fprintf(stderr, "The float size is not supported, got %d\n", bit_width);
            exit(1);
        }
    } else {
        float_type = llvm::Type::getDoubleTy(igc.context);
    }

    llvm::APFloat ap_float(float_type->getFltSemantics(), p_v_float->value);
    llvm::Value* const_value = llvm::ConstantFP::get(igc.context, ap_float);

    return llvm_to_smooth(igc, type, const_value);
}

if (std::holds_alternative<std::shared_ptr<HardvalString>>(hardval)) {
```

```
        const auto& p_v_string = std::get<std::shared_ptr<HardvalString>>(hardval);
        llvm::Value* str_const = igc.builder.CreateGlobalStringPtr(p_v_string->value);
        return llvm_to_smooth(igc, type, str_const);
    }

    fprintf(stderr, "Unhandled evaluation logic for hardval\n");
    exit(1);
}
```

compiler/backend/src/evaluate_hardval.hpp

```
#pragma once

#include "t_hardval_fwd.hpp"
#include "t_types_fwd.hpp"
#include "t_smooth_fwd.hpp"
#include "t_ctx.hpp"

Smooth evaluate_hardval(
    IrGenCtx& igc,
    const Hardval& hardval,
    Type type
);
```

compiler/backend/src/evaluate_singletonish.cpp

```
#include <bit>
#include <algorithm>
#include <cstdio>
#include <cstdlib>
#include "llvm/IR/Constants.h"
#include "llvm/IR/DerivedTypes.h"
#include "evaluate_singletonish.hpp"
#include "get_underlying_type.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"
#include "t_hardval.hpp"
#include "preinstantiated_smooths.hpp"

Smooth evaluate_singletonish(IrGenCtx& igc, Type type) {
    type = get_underlying_type(type);

    if (auto p_v_void = std::get_if<std::shared_ptr<TypeVoid>>(&type)) {
        return smooth_void(igc, type);
    }

    if (auto p_v_enum = std::get_if<std::shared_ptr<TypeEnum>>(&type)) {
        auto v_enum = *p_v_enum;

        if (!v_enum->hardsym.has_value()) {
            fprintf(stderr, "Enum was noted as singletonish incorrectly - no hardsym.\n
    ↪ ");
            exit(1);
        }

        const std::string& hardsym = v_enum->hardsym.value();
        auto it = std::find(v_enum->syms.begin(), v_enum->syms.end(), hardsym);
        uint32_t enum_idx = (uint32_t) std::distance(v_enum->syms.begin(), it);
        uint32_t bit_width = (uint32_t) std::bit_width(v_enum->syms.size() - 1);
        llvm::Type* int_type = llvm::IntegerType::get(igc.context, bit_width);
        llvm::Constant* value_as_constant = llvm::ConstantInt::get(int_type, enum_idx);

        return std::make_shared<SmoothEnum>(SmoothEnum{
            type,
```

```
            value_as_constant,
        });
    }

    if (auto p_v_pointer = std::get_if<std::shared_ptr<TypePointer>>(&type)) {
        auto v_pointer = *p_v_pointer;

        if (!v_pointer->hardval.has_value()) {
            fprintf(stderr, "Pointer was incorrectly noted as singletonish when no
    ↪ hardval.\n");
            exit(1);
        }

        if (auto p_v_str = std::get_if<std::shared_ptr<HardvalString>>(&v_pointer->
    ↪ hardval.value())) {
            llvm::Constant* str_const = igc.builder.CreateGlobalStringPtr((*p_v_str)->
    ↪ value);
            return std::make_shared<SmoothPointer>(SmoothPointer{ type, str_const });
        }

        fprintf(stderr, "Pointer hardval not implemented for singletonish optimization.\
    ↪ n");
        exit(1);
    }

    fprintf(stderr, "Type didn't have any singletonishability.\n");
    exit(1);
}
```

compiler/backend/src/evaluate_singletonish.hpp

```
#pragma once

#include "t_types_fwd.hpp"
#include "t_smooth_fwd.hpp"
#include "t_ctx.hpp"

Smooth evaluate_singletonish(IrGenCtx& igc, Type type);
```

compiler/backend/src/evaluate_smooth.cpp

```
#include <algorithm>
#include <bit>
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <variant>
#include <string>
#include "llvm/IR/Constants.h"
#include "llvm/IR/DerivedTypes.h"
#include "evaluate_smooth.hpp"
#include "evaluate_hardval.hpp"
#include "t_hardval.hpp"
#include "t_types.hpp"
#include "t_smooth.hpp"
#include "t_smooth_fwd.hpp"
#include "llvm_value.hpp"
#include "extract_leaf.hpp"
#include "merge_smooth.hpp"
#include "create_value_symbol_table.hpp"
#include "process_map_body.hpp"
#include "get_underlying_type.hpp"
#include "is_subset_type.hpp"
#include "llvm_to_smooth_bool.hpp"
```

```
#include "structwrap.hpp"
#include "is_structwrappable.hpp"
#include "smooth_type.hpp"
#include "llvm_to_smooth.hpp"
#include "build_compare.hpp"
#include "rotten_int_info.hpp"
#include "rotten_float_info.hpp"
#include "extract_map_leaf.hpp"
#include "preinstantiated_smooths.hpp"
#include "access_variable.hpp"
#include "access_member.hpp"
#include "is_type_singletonish.hpp"
#include "evaluate_singletonish.hpp"
#include "produce_call_func.hpp"
#include "happy_smooth.hpp"

bool determine_has_leaf(const Type& type) { // soon this function should be deprecated
    ↪ because it is not relevant to non-maps anymore.
    if (auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&type)) {
        // return (*p_v_map)->leaf_type.has_value() || (*p_v_map)->leaf_hardval.
    ↪ has_value();
        return (*p_v_map)->leaf_type.has_value();
    }

    // what was I thinking here.
    // if (auto p_pointer = std::get_if<std::shared_ptr<TypePointer>>(&type)) {
    //  return true;
    // }

    return false;
}

// number one rule in this codebase: we never replace a type with its underlying, we
    ↪ keep type informatoin in tact. values respect the types, not the other way around.
// types are not constructed on the fly to match smooth, rather smooth must match
    ↪ underlying types, original types remain in tact.
// underlying types are not determined from smooth, always the other way around.

Smooth evaluate_smooth(
    IrGenCtx& igc,
    const Type& type
) {
    if (auto p_v_var_access = std::get_if<std::shared_ptr<TypeVarAccess>>(&type)) {
        return access_variable(igc, type);
    }

    if (auto p_v_merged = std::get_if<std::shared_ptr<TypeMerged>>(&type)) {
        const auto& merged = **p_v_merged;

        if (merged.types.empty()) {
            fprintf(stderr, "merging nothing?\n");
            exit(1);
        }

        Smooth result = evaluate_smooth(igc, merged.types[0]);

        for (size_t i = 1; i < merged.types.size(); i++) {
            Smooth next = evaluate_smooth(igc, merged.types[i]);
            result = merge_smooth(igc, result, next);
        }

        return result;
    }

    if (auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&type)) {
        const TypeMap& map = **p_v_map;

        llvm::Function* call_func = produce_call_func(igc, *p_v_map);

        std::shared_ptr<ValueSymbolTable> map_value_table = std::make_shared<
```

```
↪ ValueSymbolTable>(
        create_value_symbol_table(igc.value_table.get())
    );

    IrGenCtx map_igc = igc;
    map_igc.value_table = map_value_table;
    // map_igc.block_break = nullptr;

    process_map_body(map_igc, map);

    std::optional<Smooth> leaf = std::nullopt;
    std::vector<llvm::Type*> member_types;
    std::vector<llvm::Value*> member_values;
    std::vector<Smooth> field_smooths;

    if (map.leaf_hardval.has_value() && !map.leaf_type.has_value()) {
        fprintf(stderr, "Since refactor, this is a major invariant violation.\n");
        exit(1);
    }

    if (map.leaf_type.has_value()) {
        if (is_type_singletonish(map.leaf_type.value())) {
            leaf = evaluate_singletonish(igc, map.leaf_type.value());
        } else {
            leaf = evaluate_smooth(igc, map.leaf_type.value());

            if (map.leaf_hardval.has_value()) {
                // now consistently using the central merging system here for
↪ hardvals.

                Smooth hardval_smooth = evaluate_hardval(map_igc, map.leaf_hardval.
↪ value(), map.leaf_type.value());

                leaf = merge_smooth(igc, leaf.value(), hardval_smooth);
            }

            llvm::Value* leaf_value = llvm_value(leaf.value());
            member_types.push_back(leaf_value->getType());
            member_values.push_back(leaf_value);
            field_smooths.push_back(leaf.value());
        }
    }

    for (const auto& [sym_name, sym_type] : map.sym_inputs) {
        if (is_type_singletonish(*sym_type)) {
            continue;
        }

        std::string map_sym_var_name = "ms_" + sym_name;
        std::optional<ValueSymbolTableEntry> o_entry = map_igc.value_table->get(
↪ map_sym_var_name);

        if (!o_entry.has_value()) {
            fprintf(stderr, "Symbol as a variable %s was not really present in the
↪ value table\n", sym_name.c_str());
            exit(1);
        }

        ValueSymbolTableEntry entry = o_entry.value();

        llvm::Value* loaded = map_igc.builder.CreateLoad(
            entry.ir_type,
            entry.alloca_ptr
        );

        member_types.push_back(loaded->getType());
        member_values.push_back(loaded);
        field_smooths.push_back(llvm_to_smooth(map_igc, *sym_type, loaded));
    }
```

```
    llvm::StructType* struct_type = llvm::StructType::get(igc.context, member_types)
↪ ;
    llvm::Value* struct_value = llvm::UndefValue::get(struct_type);

    for (size_t i = 0; i < member_values.size(); i++) {
        struct_value = igc.builder.CreateInsertValue(struct_value, member_values[i],
↪  i);
    }

    return std::make_shared<SmoothStructval>(SmoothStructval{
        type,
        struct_value,
        determine_has_leaf(type),
        leaf,
        call_func,
        field_smooths,
    });
}

if (auto p_v_pointer = std::get_if<std::shared_ptr<TypePointer>>(&type)) {
    const TypePointer& pointer = **p_v_pointer;

    if (!pointer.hardval.has_value()) {
        // fprintf(stderr, "Evaluation cannot consist of a pointer that has no
↪ definitive value, for now (in future, would make sense to deal with pointer of
↪ variable, etc.)\n");
        // exit(1);
        return smooth_void(igc, type);
    }

    // might be thinking when on earth would a pointer be assigned to a hardval,
    // but don't forget string literals.
    Smooth ptr = evaluate_hardval(igc, pointer.hardval.value(), type);

    return std::make_shared<SmoothPointer>(SmoothPointer{
        type,
        llvm_value(ptr),
    });
}

if (auto p_v_log = std::get_if<std::shared_ptr<TypeLog>>(&type)) {
    const auto& v_log = *p_v_log;

    if (v_log->message == nullptr) {
        llvm::Value* log_str = igc.builder.CreateGlobalStringPtr("");
        igc.builder.CreateCall(igc.puts_func, { log_str });
    } else {
        Smooth message_smooth = evaluate_smooth(igc, *v_log->message);
        Smooth leaf_smooth = extract_leaf(igc, message_smooth, true);
        igc.builder.CreateCall(igc.puts_func, { llvm_value(leaf_smooth) });
    }

    return smooth_void(igc, type);
}

if (auto p_v_log_d = std::get_if<std::shared_ptr<TypeLogD>>(&type)) {
    const auto& v_log_d = *p_v_log_d;

    Smooth message_smooth = evaluate_smooth(igc, *v_log_d->message);

    llvm::Value* leaf;

    if (auto p_v_enum = std::get_if<std::shared_ptr<SmoothEnum>>(&message_smooth)) {
        leaf = (*p_v_enum)->value;
    } else {
        Smooth leaf_smooth = extract_leaf(igc, message_smooth, true);
        leaf = llvm_value(leaf_smooth);
    }

    llvm::Type* leaf_type = leaf->getType();
```

```
    uint64_t bit_width = leaf_type->getPrimitiveSizeInBits();

    if (bit_width == 0 && leaf_type->isPointerTy()) {
        bit_width = igc.module.getDataLayout().getPointerSizeInBits();
    }

    if (bit_width == 0) {
        llvm::Value* error_str = igc.builder.CreateGlobalStringPtr("[undeterminable
↪ size]");
        igc.builder.CreateCall(igc.puts_func, { error_str });

        return smooth_void(igc, type);
    }

    if (leaf_type->isPointerTy()) {
        leaf = igc.builder.CreatePtrToInt(leaf, llvm::IntegerType::get(igc.context,
↪ bit_width));
    } else if (!leaf_type->isIntegerTy()) { // interpret the value as raw bits
        leaf = igc.builder.CreateBitCast(leaf, llvm::IntegerType::get(igc.context,
↪ bit_width));
    }

    uint64_t num_chunks = (bit_width + 63) / 64;
    uint64_t total_bits = num_chunks * 64;

    llvm::Type* i8  = llvm::Type::getInt8Ty(igc.context);
    llvm::Type* i64 = llvm::Type::getInt64Ty(igc.context);

    llvm::Value* wide = (total_bits > bit_width) // zero-extend
        ? igc.builder.CreateZExt(leaf, llvm::IntegerType::get(igc.context,
↪ total_bits))
        : leaf;

    for (uint64_t i = 0; i < num_chunks; i++) {
        uint64_t shift = (num_chunks - 1 - i) * 64;
        llvm::Value* shifted = igc.builder.CreateLShr(wide, llvm::ConstantInt::get(
↪ wide->getType(), shift));
        llvm::Value* chunk = igc.builder.CreateTrunc(shifted, i64);

        llvm::Value* bl;
        llvm::Value* br;

        if (num_chunks == 1) {
            bl = llvm::ConstantInt::get(i8, '[');
            br = llvm::ConstantInt::get(i8, ']');
        } else if (i == 0) {
            bl = llvm::ConstantInt::get(i8, '[');
            br = llvm::ConstantInt::get(i8, '-');
        } else if (i == num_chunks - 1) {
            bl = llvm::ConstantInt::get(i8, '-');
            br = llvm::ConstantInt::get(i8, ']');
        } else {
            bl = llvm::ConstantInt::get(i8, '-');
            br = llvm::ConstantInt::get(i8, '-');
        }

        igc.builder.CreateCall(igc.log_data_func, { chunk, bl, br });
    }

    return smooth_void(igc, type);
}

if (auto p_v_log_dd = std::get_if<std::shared_ptr<TypeLogDd>>(&type)) {
    const auto& v_log_dd = *p_v_log_dd;

    Smooth message_smooth = evaluate_smooth(igc, *v_log_dd->message);
    Smooth leaf_smooth = extract_leaf(igc, message_smooth, true);
    llvm::Value* leaf = llvm_value(leaf_smooth);
    llvm::Type* leaf_type = leaf->getType();
```

```
    if (!leaf_type->isPointerTy()) {
        fprintf(stderr, "A pointer must be encoded in the leaf when logging in dd
↪ mode\n");
        exit(1);
    }

    llvm::Type* i8ptr = llvm::Type::getInt8PtrTy(igc.context);
    llvm::Type* i64 = llvm::Type::getInt64Ty(igc.context);

    llvm::Value* ptr = igc.builder.CreateBitCast(leaf, i8ptr);

    llvm::Value* byte_count;

    if (v_log_dd->is_nullterm) {
        byte_count = llvm::ConstantInt::get(i64, (uint64_t) -1);
    } else {
        Smooth count_smooth = evaluate_smooth(igc, *v_log_dd->byte_count);
        Smooth count_leaf_smooth = extract_leaf(igc, count_smooth, true);
        llvm::Value* count_leaf = llvm_value(count_leaf_smooth);

        if (!count_leaf->getType()->isIntegerTy()) {
            fprintf(stderr, "log_dd byte_count must be integeral.\n");
            exit(1);
        }

        byte_count = (count_leaf->getType() == i64)
            ? count_leaf
            : igc.builder.CreateZExt(count_leaf, i64);
    }

    igc.builder.CreateCall(igc.log_data_deref_func, { ptr, byte_count });

    return smooth_void(igc, type);
}

if (auto p_v_var_walrus = std::get_if<std::shared_ptr<TypeVarWalrus>>(&type)) {
    const auto& v_var_walrus = *p_v_var_walrus;

    std::string map_var_name = "m_" + v_var_walrus->name;
    std::string scoped_alloca_name = igc.value_table->scope_id() + "~" +
↪ map_var_name;

    Smooth smooth = evaluate_smooth(igc, *v_var_walrus->typeval);

    if (is_structwrappable(smooth)) {
        smooth = structwrap(igc, smooth);
    }

    llvm::Value* value = llvm_value(smooth);

    llvm::Value* alloca = igc.builder.CreateAlloca(value->getType(), nullptr,
↪ scoped_alloca_name);
    igc.builder.CreateStore(value, alloca);

    ValueSymbolTableEntry entry{
        alloca,
        value->getType(),
        smooth_type(smooth),
        v_var_walrus->is_mut,
    };

    igc.value_table->set(map_var_name, entry);

    return access_variable(igc, type);
}

if (auto p_v_var_assign = std::get_if<std::shared_ptr<TypeVarAssign>>(&type)) {
    const auto& v_var_assign = *p_v_var_assign;
```

```
    std::string map_var_name = "m_" + v_var_assign->name;
    std::optional<ValueSymbolTableEntry> o_entry = igc.value_table->get(map_var_name
↪ );

    if (!o_entry.has_value()) {
        fprintf(stderr, "So %s does not exist.\n", v_var_assign->name.c_str());
        exit(1);
    }

    const ValueSymbolTableEntry& existing = o_entry.value();

    if (!existing.is_mut) {
        fprintf(stderr, "Cannot mutate immutable variable %s, consider adding \"mut
↪ \" keyword like `mut x := 5;`\n", v_var_assign->name.c_str());
        exit(1);
    }

    Smooth smooth = evaluate_smooth(igc, *v_var_assign->typeval);

    llvm::Value* value;

    if (is_structwrappable(smooth)) {
        value = structwrap(igc, smooth)->value;
    } else {
        value = llvm_value(smooth);
    }

    if (!is_subset_type(smooth_type(smooth), existing.type)) {
        fprintf(stderr, "Value is not assignable to variable \"%s\" due to
↪ incompatibility (types are not subsets)\n", v_var_assign->name.c_str());
        exit(1);
    }

    igc.builder.CreateStore(value, existing.alloca_ptr);

    // as we are using stack allocations, we can mutate via stack address, bypassing
↪  immutable registers and phi logic.
    // llvm will optimize this to single-static-assignment (SSA) form for us, when
↪ possible.

    return access_variable(igc, type);
}

if (auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&type)) {
    llvm::Value* void_value = smooth_void(igc, type)->value;

    if (auto info = rotten_int_info(*p_v_rotten)) {
        llvm::Type* flexi_type = llvm::IntegerType::get(igc.context, info->bits);

        return std::make_shared<SmoothVoidInt>(SmoothVoidInt{
            type,
            flexi_type,
            void_value,
        });
    }

    if (auto info = rotten_float_info(*p_v_rotten)) {
        llvm::Type* flexi_type = nullptr;

        if (info->bits == 16) flexi_type = llvm::Type::getHalfTy(igc.context);
        else if (info->bits == 32) flexi_type = llvm::Type::getFloatTy(igc.context);
        else if (info->bits == 64) flexi_type = llvm::Type::getDoubleTy(igc.context)
↪ ;
        else if (info->bits == 128) flexi_type = llvm::Type::getFP128Ty(igc.context)
↪ ;

        return std::make_shared<SmoothVoidFloat>(SmoothVoidFloat{
            type,
            flexi_type,
            void_value,
```

```
        });
    }

    fprintf(stderr, "What sort of a rotten that is not intish or floatish is this.")
↪ ;
    exit(1);
}

if (auto p_v_enum = std::get_if<std::shared_ptr<TypeEnum>>(&type)) {
    const auto& v_enum = *p_v_enum;

    if (!v_enum->is_instantiated && !v_enum->hardsym.has_value()) {
        return std::make_shared<SmoothEnum>(SmoothEnum{ type, nullptr });
    }

    uint32_t bit_width = (uint32_t) std::bit_width(v_enum->syms.size() - 1);
    llvm::Type* int_type = llvm::IntegerType::get(igc.context, bit_width);

    // if (!v_enum->hardsym.has_value()) {
    //  fprintf(stderr, "Enum does not have any definitive symbol, could not reduce
↪ to one possibility.\n");
    //  exit(1);
    // }

    const std::string& hardsym = v_enum->hardsym.value();
    auto it = std::find(v_enum->syms.begin(), v_enum->syms.end(), hardsym);

    if (it == v_enum->syms.end()) {
        fprintf(stderr, "While attempting to instantiate \"%s\", it was not found as
↪  a possibility.\n", hardsym.c_str());
        exit(1);
    }

    uint32_t enum_idx = (uint32_t) std::distance(v_enum->syms.begin(), it);
    llvm::Value* value = llvm::ConstantInt::get(int_type, enum_idx);

    return std::make_shared<SmoothEnum>(SmoothEnum{
        type,
        value,
    });
}

if (auto p_v_call_with_sym = std::get_if<std::shared_ptr<TypeCallWithSym>>(&type)) {
    const auto& v_call_with_sym = *p_v_call_with_sym;
    Smooth target_smooth = evaluate_smooth(igc, *v_call_with_sym->target);

    if (!is_structwrappable(target_smooth)) {
        fprintf(stderr, "cannot be expected to call with a sym to a non structish
↪ object.\n");
        exit(1);
    }

    auto modern_target_smooth = structwrap(igc, target_smooth);

    return access_member(igc, modern_target_smooth, v_call_with_sym->sym);
}

if (auto p_v_call_with_dynamic = std::get_if<std::shared_ptr<TypeCallWithDynamic>>(&
↪ type)) {
    const auto& v_call_with_dynamic = *p_v_call_with_dynamic;

    Smooth target_smooth = evaluate_smooth(igc, *v_call_with_dynamic->target);
    Smooth data_smooth = evaluate_smooth(igc, *v_call_with_dynamic->call_data);

    if (!is_structwrappable(target_smooth)) {
        fprintf(stderr, "have to call on a structish thing.\n");
        exit(1);
    }

    if (auto p_smooth_enum = std::get_if<std::shared_ptr<SmoothEnum>>(&data_smooth))
```

```
↪  {
        const auto& smooth_enum = *p_smooth_enum;
        auto p_type_enum = std::get_if<std::shared_ptr<TypeEnum>>(&smooth_enum->type
↪ );

        // for now we cannot allow dynamic field selection because type is too
↪ variable.
        // i want to solve this down the road using comptime logic where the
↪ compiler knows the set of possible types, but one must match the input sym to
↪ determine which type is in effect.
        // each match statement would constraint the possibility and prove to the
↪ compiler what the type is.

        if (!p_type_enum || !(*p_type_enum)->hardsym.has_value()) {
            fprintf(stderr, "for now : hardsym must be statically provided for
↪ dynamic sym call. will be tackled down the road using my so-called \"description\"
↪  system.\n");
            exit(1);
        }

        const std::string& sym = (*p_type_enum)->hardsym.value();

        auto improved_smooth = structwrap(igc, target_smooth);

        return access_member(igc, improved_smooth, sym);
    } else if (std::get_if<std::shared_ptr<SmoothStructval>>(&data_smooth)) {
        auto actual_target = std::get_if<std::shared_ptr<SmoothStructval>>(&
↪ target_smooth);

        if (!actual_target || (*actual_target)->call_func == nullptr) {
            fprintf(stderr, "cannot do a map call on a target that doesn't support
↪ it. - map is not map-callable.");
            exit(1);
        }

        auto actual_target_type = std::get_if<std::shared_ptr<TypeMap>>(&(*
↪ actual_target)->type);

        if (!actual_target_type || (*actual_target_type)->call_output_type ==
↪ nullptr) {
            fprintf(stderr, "somehow target had no appropriate map information.\n");
            exit(1);
        }

        Smooth wrapped_up = structwrap(igc, data_smooth);
        Smooth upgraded = happy_smooth(igc, wrapped_up, Type((*actual_target_type)->
↪ call_input_type));
        llvm::Value* input_payload = llvm_value(upgraded);

        llvm::Value* output_value = igc.builder.CreateCall((*actual_target)->
↪ call_func, { input_payload });

        Type output_type = Type((*actual_target_type)->call_output_type);

        auto output_smooth = std::make_shared<SmoothStructval>(SmoothStructval{
            output_type,
            output_value,
            false,
            std::nullopt, // this is problematic.
            nullptr, // this line is problematic.
            {}, // this all needs to be sorted properly.
        });

        return Smooth(output_smooth);
    } else {
        fprintf(stderr, "currenty only sym calls and map calls are supported.\n");
        exit(1);
    }
}
```

```
if (auto p_v_expr_multi = std::get_if<std::shared_ptr<TypeExprMulti>>(&type)) {
    const auto& v_expr_multi = **p_v_expr_multi;
    llvm::Value* result = nullptr;
    std::optional<bool> is_float; // we will not allow combining int and float
↪ operations implicitely.
    char prefix = 'i';

    for (const auto& op_numeric : v_expr_multi.ops) {
        Smooth smooth = evaluate_smooth(igc, *op_numeric.operand);
        Smooth value_smooth = extract_leaf(igc, smooth, true);

        Type operand_underlying = get_underlying_type(extract_map_leaf(*op_numeric.
↪ operand, true));
        auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&
↪ operand_underlying);

        if (!p_v_rotten) {
            fprintf(stderr, "cannot multiply something non-rotten\n");
            fprintf(stderr, "Cannot do multiplication on things that aren't integer
↪ or float.\n");
            exit(1);
        }

        bool is_this_one_float;

        if (auto info = rotten_int_info(*p_v_rotten)) {
            if (is_float.has_value() && is_float.value()) {
                fprintf(stderr, "do not mix int and float like this (got int after
↪ float).\n");
                fprintf(stderr, "The compiler does not support implicit combining of
↪  ints and float operations.\n");
                exit(1);
            }

            is_float = false;
            prefix = info->prefix;
            is_this_one_float = false;
        } else if (auto info = rotten_float_info(*p_v_rotten)) {
            if (is_float.has_value() && !is_float.value()) {
                fprintf(stderr, "do not mix int and float (got float after int)\n");
                fprintf(stderr, "The compiler does not support implicit combining of
↪  ints and float operations.\n");
                exit(1);
            }

            is_float = true;
            is_this_one_float = true;
        } else {
            fprintf(stderr, "some other rotten type was used that is not numerical (
↪ not supporting multiplication)\n");
            exit(1);
        }

        llvm::Value* value = llvm_value(value_smooth);

        if (result == nullptr) {
            result = is_float.value() ? llvm::ConstantFP::get(value->getType(), 1.0)
↪  : llvm::ConstantInt::get(value->getType(), 1);
        }

        if (!is_float.value()) { // i don't care about floats for now.
            uint32_t result_bits = result->getType()->getIntegerBitWidth();
            uint32_t value_bits = value->getType()->getIntegerBitWidth();

            if (result_bits < value_bits) {
                result = igc.builder.CreateZExt(result, value->getType());
            } else if (value_bits < result_bits) {
                value = igc.builder.CreateZExt(value, result->getType());
            }
        }
```

```
        if (op_numeric.op == "*") {
            result = is_float.value() ? igc.builder.CreateFMul(result, value) : igc.
↪ builder.CreateMul(result, value);
        } else if (op_numeric.op == "/") {
            result = is_float.value() ? igc.builder.CreateFDiv(result, value) : igc.
↪ builder.CreateSDiv(result, value);
            // todo: is signed division appropriate in all cases? probably need to
↪ adjust this.
        } else {
            fprintf(stderr, "Some bizarre operator was encountered %s (
↪ multiplicative)\n", op_numeric.op.c_str());
            exit(1);
        }
    }

    if (result == nullptr) {
        fprintf(stderr, "multiplicative expression had no operations.\n");
        exit(1);
    }

    uint32_t actual_bits = result->getType()->getPrimitiveSizeInBits();
    auto rotten_outcome = std::make_shared<TypeRotten>();

    if (is_float.value()) {
        rotten_outcome->type_str = "f" + std::to_string(actual_bits);
    } else {
        rotten_outcome->type_str = std::string(1, prefix) + std::to_string(
↪ actual_bits);
    }

    (*p_v_expr_multi)->underlying_type = std::make_shared<Type>(Type(rotten_outcome)
↪ );

    if (is_float.value()) {
        return std::make_shared<SmoothFloat>(SmoothFloat{
            type,
            result,
        });
    }

    return std::make_shared<SmoothInt>(SmoothInt{
        type,
        result,
    });
}

if (auto p_v_expr_addit = std::get_if<std::shared_ptr<TypeExprAddit>>(&type)) {
    const auto& v_expr_addit = **p_v_expr_addit;
    llvm::Value* result = nullptr;
    std::optional<bool> is_float;
    char prefix = 'i';

    for (const auto& op_numeric : v_expr_addit.ops) {
        Smooth smooth = evaluate_smooth(igc, *op_numeric.operand);
        Smooth value_smooth = extract_leaf(igc, smooth, true);

        Type operand_underlying = get_underlying_type(extract_map_leaf(*op_numeric.
↪ operand, true));
        auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&
↪ operand_underlying);

        if (!p_v_rotten) {
            fprintf(stderr, "cannot add something non-rotten\n");
            fprintf(stderr, "Cannot do addition on data that isn't integer or float
↪ .\n");
            exit(1);
        }

        bool is_this_one_float;
```

```
        if (auto info = rotten_int_info(*p_v_rotten)) {
            if (is_float.has_value() && is_float.value()) {
                fprintf(stderr, "do not mix int and float like this (got int after
↪ float).\n");
                fprintf(stderr, "The compiler does not support implicit combining of
↪  ints and float operations.\n");
                exit(1);
            }

            is_float = false;
            prefix = info->prefix;
            is_this_one_float = false;
        } else if (auto info = rotten_float_info(*p_v_rotten)) {
            if (is_float.has_value() && !is_float.value()) {
                fprintf(stderr, "do not mix int and float (got float after int)\n");
                fprintf(stderr, "The compiler does not support implicit combining of
↪  ints and float operations.\n");
                exit(1);
            }

            is_float = true;
            is_this_one_float = true;
        } else {
            fprintf(stderr, "some other rotten type was used that is not numerical (
↪ not supporting addition)\n");
            exit(1);
        }

        llvm::Value* value = llvm_value(value_smooth);

        if (result == nullptr) {
            result = is_float.value() ? llvm::ConstantFP::get(value->getType(), 0.0)
↪  : llvm::ConstantInt::get(value->getType(), 0);
        }

        if (!is_float.value()) { // dont care about floats for now.
            uint32_t result_bits = result->getType()->getIntegerBitWidth();
            uint32_t value_bits = value->getType()->getIntegerBitWidth();

            if (result_bits < value_bits) {
                result = igc.builder.CreateZExt(result, value->getType());
            } else if (value_bits < result_bits) {
                value = igc.builder.CreateZExt(value, result->getType());
            }
        }

        if (op_numeric.op == "+") {
            result = is_float.value() ? igc.builder.CreateFAdd(result, value) : igc.
↪ builder.CreateAdd(result, value);
        } else if (op_numeric.op == "-") {
            result = is_float.value() ? igc.builder.CreateFSub(result, value) : igc.
↪ builder.CreateSub(result, value);
        } else {
            fprintf(stderr, "Some bizarre operator was encountered %s (additive)\n",
↪  op_numeric.op.c_str());
            exit(1);
        }
    }

    if (result == nullptr) {
        fprintf(stderr, "additive expression had no operations.\n");
        exit(1);
    }

    uint32_t actual_bits = result->getType()->getPrimitiveSizeInBits();
    auto rotten_outcome = std::make_shared<TypeRotten>();

    if (is_float.value()) {
        rotten_outcome->type_str = "f" + std::to_string(actual_bits);
```

```
    } else {
        rotten_outcome->type_str = std::string(1, prefix) + std::to_string(
↪ actual_bits);
    }

    (*p_v_expr_addit)->underlying_type = std::make_shared<Type>(Type(rotten_outcome)
↪ );

    if (is_float.value()) {
        return std::make_shared<SmoothFloat>(SmoothFloat{
            type,
            result,
        });
    }

    return std::make_shared<SmoothInt>(SmoothInt{
        type,
        result,
    });
}

if (auto p_v_compare = std::get_if<std::shared_ptr<TypeCompare>>(&type)) {
    const auto& v_compare = **p_v_compare;

    Smooth a_smooth = evaluate_smooth(igc, *v_compare.operand_a);
    llvm::Value* a = llvm_value(a_smooth);

    Smooth b_smooth = evaluate_smooth(igc, *v_compare.operand_b);
    llvm::Value* b = llvm_value(b_smooth);

    llvm::Value* cmp = build_compare(igc, a, b, v_compare.operator_);

    return llvm_to_smooth_bool(igc, cmp);
}

if (auto p_v_expr_logical_and = std::get_if<std::shared_ptr<TypeExprLogicalAnd>>(&
↪ type)) {
    const auto& v_expr_logical_and = **p_v_expr_logical_and;

    if (v_expr_logical_and.ops.empty()) {
        fprintf(stderr, "so why did we get no operatands from the frontend.\n");
        exit(1);
    }

    llvm::Type* i1 = llvm::Type::getInt1Ty(igc.context);
    llvm::Value* result = llvm::ConstantInt::get(i1, 1);

    for (const auto& op : v_expr_logical_and.ops) {
        Smooth operand_smooth = evaluate_smooth(igc, *op.operand);

        if (!std::get_if<std::shared_ptr<SmoothEnum>>(&operand_smooth)) {
            fprintf(stderr, "non-enusm not permitted for logical AND.\n");
            exit(1);
        }

        llvm::Value* actual_value = llvm_value(operand_smooth);

        if (!actual_value->getType()->isIntegerTy(1)) {
            fprintf(stderr, "Tried to perform logical AND on something that wasn't
↪ intish of size 1.\n");
            exit(1);
        }

        result = igc.builder.CreateAnd(result, actual_value);
    }

    return llvm_to_smooth_bool(igc, result);
}

if (auto p_v_expr_logical_or = std::get_if<std::shared_ptr<TypeExprLogicalOr>>(&type
```

```
    ↪ )) {
        const auto& v_expr_logical_or = **p_v_expr_logical_or;

        if (v_expr_logical_or.ops.empty()) {
            fprintf(stderr, "some operands missing from thefrontned.\n");
            exit(1);
        }

        llvm::Type* i1 = llvm::Type::getInt1Ty(igc.context);
        llvm::Value* result = llvm::ConstantInt::get(i1, 0);

        for (const auto& op : v_expr_logical_or.ops) {
            Smooth operand_smooth = evaluate_smooth(igc, *op.operand);

            if (!std::get_if<std::shared_ptr<SmoothEnum>>(&operand_smooth)) {
                fprintf(stderr, "non-enum not permitted for logical OR.\n");
                exit(1);
            }

            llvm::Value* actual_value = llvm_value(operand_smooth);

            if (!actual_value->getType()->isIntegerTy(1)) {
                fprintf(stderr, "Tried to perform logical OR on something that wasn't
    ↪ intish of size 1.\n");
                exit(1);
            }

            result = igc.builder.CreateOr(result, actual_value);
        }

        return llvm_to_smooth_bool(igc, result);
    }

    if (std::get_if<std::shared_ptr<TypeVoid>>(&type)) {
        return smooth_void(igc, type);
    }

    fprintf(stderr, "Unhandled scenario when handling evaluation\n");
    exit(1);
}
```

compiler/backend/src/evaluate_smooth.hpp

```
#pragma once

#include <string>
#include "llvm/IR/Value.h"
#include "t_types_fwd.hpp"
#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_smooth.hpp"

Smooth evaluate_smooth(
    IrGenCtx& igc,
    const Type& type
);
```

compiler/backend/src/extract_leaf.cpp

```
#include "extract_leaf.hpp"
#include "get_underlying_type.hpp"
#include "is_structwrappable.hpp"
#include "rotten_float_info.hpp"
#include "structwrap.hpp"
```

```
#include "t_smooth.hpp"
#include "t_types.hpp"

Smooth extract_leaf(IrGenCtx& igc, Smooth smooth, bool be_permissive) {
    if (auto p_v_structval = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth)) {
        const auto& v_structval = *p_v_structval;

        if (!v_structval->has_leaf) {
            fprintf(stderr, "Not good circumstances - no leaf.\n");
            fprintf(stderr, "Actually what we got is: ");
            v_structval->value->print(llvm::errs());

            fprintf(stderr, "Bad programmer - check for .has_leaf first!\n");
            exit(1);
        }

        if (v_structval->leaf.has_value()) {
            return v_structval->leaf.value();
        }

        // legacy system below in case something goes wrong - can possibly delete this
    ↪ code once i've confirmed the entire codebase has migrated to the new system.

        Type underlying = get_underlying_type(v_structval->type);
        auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&underlying);

        if (!p_v_map) {
            fprintf(stderr, "structval must interpret itself as a map after evaluation.\
    ↪ n");
            exit(1);
        }

        const auto& o_leaf_type = (*p_v_map)->leaf_type;
        const auto& o_leaf_hardval = (*p_v_map)->leaf_hardval;

        if (!o_leaf_type.has_value() && !o_leaf_hardval.has_value()) {
            fprintf(stderr, "invariant violation.\n");
            exit(1);
        }

        llvm::Value* value = igc.builder.CreateExtractValue(v_structval->value, 0);

        if (!o_leaf_type.has_value()) {
            auto v_rotten = std::make_shared<TypeRotten>();

            if (value->getType()->isFloatingPointTy()) {
                v_rotten->type_str = "f" + std::to_string(value->getType()->
    ↪ getPrimitiveSizeInBits());

                return std::make_shared<SmoothFloat>(SmoothFloat{
                    Type(v_rotten),
                    value,
                });
            }

            v_rotten->type_str = "i" + std::to_string(value->getType()->
    ↪ getIntegerBitWidth());

            return std::make_shared<SmoothInt>(SmoothInt{
                Type(v_rotten),
                value,
            });
        }

        const Type leaf_type = o_leaf_type.value();
        Type leaf_underlying = get_underlying_type(leaf_type);

        if (auto p_v_enum = std::get_if<std::shared_ptr<TypeEnum>>(&leaf_underlying)) {
            return std::make_shared<SmoothEnum>(SmoothEnum{
                leaf_type,
```

```
                value,
            });
        }

        if (std::get_if<std::shared_ptr<TypePointer>>(&leaf_underlying)) {
            return std::make_shared<SmoothPointer>(SmoothPointer{
                leaf_type,
                value,
            });
        }

        if (auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&leaf_underlying)
    ↪ ) {
            const std::string& type_str = (*p_v_rotten)->type_str;

            const bool is_float = (false
                || type_str == "f16"
                || type_str == "f32"
                || type_str == "f64"
                || type_str == "f128"
            );

            if (is_float) {
                return std::make_shared<SmoothFloat>(SmoothFloat{
                    leaf_type,
                    value,
                });
            }

            return std::make_shared<SmoothInt>(SmoothInt{
                leaf_type,
                value,
            });
        }

        fprintf(stderr, "Leaf was something we don't know how to deal with\n");
        exit(1);
    }

    if (!be_permissive) {
        fprintf(stderr, "smooth was not some scructval. either pass be_permissive (to
    ↪ auto-wrap before extraction) or ensure exact structval is passed.\n");
        exit(1);
    }

    if (!is_structwrappable(smooth)) {
        fprintf(stderr, "smooth was not structwrawppable. although be_permissive flag
    ↪ was set, we could not auto-wrap such an exoteic object.\n");
        exit(1);
    }

    return extract_leaf(igc, structwrap(igc, smooth));
}
```

compiler/backend/src/extract_leaf.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"

Smooth extract_leaf(IrGenCtx& igc, Smooth smooth, bool be_permissive = false);
```

compiler/backend/src/extract_map_leaf.cpp

```
#include "extract_map_leaf.hpp"
#include "get_underlying_type.hpp"
#include "is_mapwrappable.hpp"
#include "t_types.hpp"

Type extract_map_leaf(const Type& type, bool be_permissive) {
    Type underlying = get_underlying_type(type);
    auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&underlying);

    if (p_v_map) {
        if (!(*p_v_map)->leaf_type.has_value()) {
            fprintf(stderr, "there's no leaf type known for this map.\n");
            exit(1);
        }

        return (*p_v_map)->leaf_type.value();
    }

    if (!be_permissive) {
        fprintf(stderr, "be_permissive was not selected, but the passed type is not
    ↪ capable of holding a leaf.\n");
        exit(1);
    }

    if (!is_mapwrappable(type)) {
        fprintf(stderr, "While be_permissive was selected, the provided type could not
    ↪ be converted to something capable of holding a leaf.\n");
        exit(1);
    }

    return type;
}
```

compiler/backend/src/extract_map_leaf.hpp

```
#pragma once

#include "t_types_fwd.hpp"

Type extract_map_leaf(const Type& type, bool be_permissive = false);
```

compiler/backend/src/get_underlying_type.cpp

```
#include <cstdio>
#include <cstdlib>
#include <memory>
#include <variant>
#include "get_underlying_type.hpp"
#include "t_types.hpp"
#include "preinstantiated_types.hpp"

// there are three scenarios:
// a. the node is already the most underlying type: return the same
// b. the node tracks underlying type information via the previously assigned .
    ↪ underlying_type field.
// c. the node doesn't really need type information: return TypeVoid.

Type get_underlying_type(const Type& type) {
    if (std::holds_alternative<std::shared_ptr<TypeRotten>>(type)) {
        return type;
    }

    if (std::holds_alternative<std::shared_ptr<TypeMap>>(type)) {
        return type;
```

```
    }

    if (std::holds_alternative<std::shared_ptr<TypePointer>>(type)) {
        return type;
    }

    if (std::holds_alternative<std::shared_ptr<TypeEnum>>(type)) {
        return type;
    }

    if (auto p_v_merged = std::get_if<std::shared_ptr<TypeMerged>>(&type)) {
        const auto& v_merged = *p_v_merged;

        if (v_merged->underlying_type == nullptr) {
            fprintf(stderr, "Underlying type not populated (TypeMerged).\n");
            exit(1);
        }

        return get_underlying_type(*v_merged->underlying_type);
    }

    if (std::holds_alternative<std::shared_ptr<TypeVoid>>(type)) {
        return type;
    }

    if (std::holds_alternative<std::shared_ptr<TypeLog>>(type)) {
        return type_void;
    }

    if (std::holds_alternative<std::shared_ptr<TypeLogD>>(type)) {
        return type_void;
    }

    if (std::holds_alternative<std::shared_ptr<TypeLogDd>>(type)) {
        return type_void;
    }

    if (auto p_v_call_with_sym = std::get_if<std::shared_ptr<TypeCallWithSym>>(&type)) {
        const auto& v_call_with_sym = *p_v_call_with_sym;

        if (v_call_with_sym->underlying_type == nullptr) {
            fprintf(stderr, "Underlying type not populated (TypeCallWithSym).\n");
            exit(1);
        }

        return get_underlying_type(*v_call_with_sym->underlying_type);
    }

    if (auto p_v_call_with_dynamic = std::get_if<std::shared_ptr<TypeCallWithDynamic>>(&
    ↪ type)) {
        const auto& v_call_with_dynamic = *p_v_call_with_dynamic;

        if (v_call_with_dynamic->underlying_type == nullptr) {
            fprintf(stderr, "Underlying type not populated (TypeCallWithDynamic).\n");
            exit(1);
        }

        return get_underlying_type(*v_call_with_dynamic->underlying_type);
    }

    if (auto p_v_expr_multi = std::get_if<std::shared_ptr<TypeExprMulti>>(&type)) {
        const auto& v_expr_multi = *p_v_expr_multi;

        if (v_expr_multi->underlying_type == nullptr) {
            fprintf(stderr, "Underlying type not populated (TypeExprMulti).\n");
            exit(1);
        }

        return get_underlying_type(*v_expr_multi->underlying_type);
    }
```

```
if (auto p_v_expr_addit = std::get_if<std::shared_ptr<TypeExprAddit>>(&type)) {
    const auto& v_expr_addit = *p_v_expr_addit;

    if (v_expr_addit->underlying_type == nullptr) {
        fprintf(stderr, "Underlying type not populated (TypeExprAddit).\n");
        exit(1);
    }

    return get_underlying_type(*v_expr_addit->underlying_type);
}

if (auto p_v_expr_logical_and = std::get_if<std::shared_ptr<TypeExprLogicalAnd>>(&
↪ type)) {
    const auto& v_expr_logical_and = *p_v_expr_logical_and;

    if (v_expr_logical_and->underlying_type == nullptr) {
        fprintf(stderr, "Underlying type not populated (TypeExprLogicalAnd).\n");
        exit(1);
    }

    return get_underlying_type(*v_expr_logical_and->underlying_type);
}

if (auto p_v_expr_logical_or = std::get_if<std::shared_ptr<TypeExprLogicalOr>>(&type
↪ )) {
    const auto& v_expr_logical_or = *p_v_expr_logical_or;

    if (v_expr_logical_or->underlying_type == nullptr) {
        fprintf(stderr, "Underlying type not populated (TypeExprLogicalOr).\n");
        exit(1);
    }

    return get_underlying_type(*v_expr_logical_or->underlying_type);
}

if (auto p_v_var_access = std::get_if<std::shared_ptr<TypeVarAccess>>(&type)) {
    const auto& v_var_access = *p_v_var_access;

    if (v_var_access->underlying_type == nullptr) {
        fprintf(stderr, "Underlying type not populated (VarAccess): %s.\n",
↪ v_var_access->target_name.c_str());
        exit(1);
    }

    return get_underlying_type(*v_var_access->underlying_type);
}

if (auto p_v_walrus = std::get_if<std::shared_ptr<TypeVarWalrus>>(&type)) {
    const auto& v_walrus = *p_v_walrus;

    if (v_walrus->underlying_type == nullptr) {
        fprintf(stderr, "Underlying type not populated (VarWalrus): %s.\n", v_walrus
↪ ->name.c_str());
        exit(1);
    }

    return get_underlying_type(*v_walrus->underlying_type);
}

if (auto p_v_var_assign = std::get_if<std::shared_ptr<TypeVarAssign>>(&type)) {
    const auto& v_var_assign = *p_v_var_assign;

    if (v_var_assign->underlying_type == nullptr) {
        fprintf(stderr, "Underlying type not populated (VarAssign): %s.\n",
↪ v_var_assign->name.c_str());
        exit(1);
    }

    return get_underlying_type(*v_var_assign->underlying_type);
```

```
    }

    fprintf(stderr, "Currently no mechanism to determine the actual type of the
    ↪ expression.\n");
    exit(1);
}
```

compiler/backend/src/get_underlying_type.hpp

```
#pragma once

#include "t_types.hpp"

Type get_underlying_type(const Type& type);
```

compiler/backend/src/happy_smooth.cpp

```
#include "happy_smooth.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"
#include "get_underlying_type.hpp"
#include "rotten_int_info.hpp"
#include "rotten_float_info.hpp"
#include "is_type_singletonish.hpp"
#include "llvm_to_smooth.hpp"
#include "llvm_value.hpp"
#include "llvm/IR/DerivedTypes.h"
#include "llvm/IR/Type.h"

Smooth happy_smooth(IrGenCtx& igc, Smooth smooth, const Type& type) {
    Type underlying = get_underlying_type(type);

    if (auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&underlying)) {
        if (auto p_v_int = std::get_if<std::shared_ptr<SmoothInt>>(&smooth)) {
            if (!(*p_v_int)->value) {
                return smooth;
            }

            if (auto info = rotten_int_info(*p_v_rotten)) {
                uint32_t actual_bits = (*p_v_int)->value->getType()->getIntegerBitWidth
    ↪ ();

                if (actual_bits < info->bits) {
                    llvm::Value* extended = igc.builder.CreateZExt(
                        (*p_v_int)->value,
                        llvm::IntegerType::get(igc.context, info->bits)
                    );

                    return std::make_shared<SmoothInt>(SmoothInt{ (*p_v_int)->type,
    ↪ extended });
                }
            }

            return smooth;
        }

        if (auto p_v_float = std::get_if<std::shared_ptr<SmoothFloat>>(&smooth)) {
            if (!(*p_v_float)->value) {
                return smooth;
            }

            if (auto info = rotten_float_info(*p_v_rotten)) {
                llvm::Type* target_type = nullptr;
```

```
            if (info->bits == 16) target_type = llvm::Type::getHalfTy(igc.context);
            else if (info->bits == 32) target_type = llvm::Type::getFloatTy(igc.
↪ context);
            else if (info->bits == 64) target_type = llvm::Type::getDoubleTy(igc.
↪ context);
            else if (info->bits == 128) target_type = llvm::Type::getFP128Ty(igc.
↪ context);

            if (target_type && (*p_v_float)->value->getType() != target_type) {
                llvm::Value* extended = igc.builder.CreateFPExt(
                    (*p_v_float)->value,
                    target_type
                );

                return std::make_shared<SmoothFloat>(SmoothFloat{ (*p_v_float)->type
↪ , extended });
            }
        }

        return smooth;
    }

    return smooth;
} else if (auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&underlying)) {
    auto v_map = *p_v_map;

    auto p_v_structval = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth);

    if (!p_v_structval) {
        fprintf(stderr, "Glitch.");
        exit(1);
    }

    auto v_structval = *p_v_structval;

    llvm::Value* value = v_structval->value;

    unsigned member_idx = 0;

    std::vector<llvm::Type*> new_member_types;
    std::vector<llvm::Value*> new_member_values;

    if (true
        && v_map->leaf_type.has_value()
        && !is_type_singletonish(v_map->leaf_type.value())
    ) {
        llvm::Value* field = igc.builder.CreateExtractValue(value, member_idx);
        Smooth happy_field = happy_smooth(igc, llvm_to_smooth(igc, v_map->leaf_type.
↪ value(), field), v_map->leaf_type.value());
        llvm::Value* happy_field_value = llvm_value(happy_field);

        new_member_types.push_back(happy_field_value->getType());
        new_member_values.push_back(happy_field_value);

        member_idx++;
    }

    for (const auto& [sym_name, sym_type] : v_map->sym_inputs) {
        if (is_type_singletonish(*sym_type)) {
            continue;
        }

        llvm::Value* field = igc.builder.CreateExtractValue(value, member_idx);
        Smooth happy_field = happy_smooth(igc, llvm_to_smooth(igc, *sym_type, field)
↪ , *sym_type);
        llvm::Value* happy_field_value = llvm_value(happy_field);

        new_member_types.push_back(happy_field_value->getType());
        new_member_values.push_back(happy_field_value);
```

```
            member_idx++;
        }

        llvm::StructType* fancy_type = llvm::StructType::get(igc.context,
    ↪ new_member_types);
        llvm::Value* fancy_value = llvm::UndefValue::get(fancy_type);

        for (size_t i = 0; i < new_member_values.size(); i++) {
            fancy_value = igc.builder.CreateInsertValue(fancy_value, new_member_values[i
    ↪ ], (unsigned)i);
        }

        return std::make_shared<SmoothStructval>(SmoothStructval{
            v_structval->type,
            fancy_value,
            v_structval->has_leaf,
            v_structval->leaf,
            v_structval->call_func,
            v_structval->field_smooths, // todo: this is wrong and could cause glitches.
        });
    }

    return smooth;
}
```

compiler/backend/src/happy_smooth.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_types_fwd.hpp"

// the purpose behind this is keeping the smooth value happy, keeping it in line with
    ↪ the type information.
// the idea is that the underlying_type is the source of truth, but proper type
    ↪ information might be higher up the hierarchy.
// so every so often we run our smooths through happy_smooth if there is a risk of a
    ↪ mismatch.
// at the moment, the only risk of mismatch is during void type merging, so it is only
    ↪ called there.
Smooth happy_smooth(IrGenCtx& igc, Smooth smooth, const Type& type);
```

compiler/backend/src/is_eq_type_pointer.cpp

```
#include <cstdio>
#include <cstdlib>
#include <variant>
#include "is_eq_type_pointer.hpp"
#include "is_eq_type_rotten.hpp"
#include "t_types.hpp"

bool is_eq_type_pointer(
    const TypePointer& pointer_a,
    const TypePointer& pointer_b
) {
    if (pointer_a.target == nullptr || pointer_b.target == nullptr) {
        fprintf(stderr, "Pointer to nothing?\n");
        exit(1);
    }

    const Type& target_a = *pointer_a.target;
    const Type& target_b = *pointer_b.target;
```

```
    auto p_v_pointer_a = std::get_if<std::shared_ptr<TypePointer>>(&target_a);
    auto p_v_pointer_b = std::get_if<std::shared_ptr<TypePointer>>(&target_b);

    if (p_v_pointer_a && p_v_pointer_b) {
        return is_eq_type_pointer(**p_v_pointer_a, **p_v_pointer_b);
    }

    auto p_v_rotten_a = std::get_if<std::shared_ptr<TypeRotten>>(&target_a);
    auto p_v_rotten_b = std::get_if<std::shared_ptr<TypeRotten>>(&target_b);

    if (p_v_rotten_a && p_v_rotten_b) {
        return is_eq_type_rotten(**p_v_rotten_a, **p_v_rotten_b);
    }

    fprintf(stderr, "For now, pointers to non-rotten types are not implemented
    ↪ appropriately.\n");
    exit(1);
}
```

compiler/backend/src/is_eq_type_pointer.hpp

```
#pragma once

#include "t_types_fwd.hpp"

bool is_eq_type_pointer(
    const TypePointer& pointer_a,
    const TypePointer& pointer_b
);
```

compiler/backend/src/is_eq_type_rotten.cpp

```
#include "is_eq_type_rotten.hpp"
#include "t_types.hpp"

bool is_eq_type_rotten(
    const TypeRotten& rotten_a,
    const TypeRotten& rotten_b
) {
    return rotten_a.type_str == rotten_b.type_str;
}
```

compiler/backend/src/is_eq_type_rotten.hpp

```
#pragma once

#include "t_types_fwd.hpp"

bool is_eq_type_rotten(
    const TypeRotten& rotten_a,
    const TypeRotten& rotten_b
);
```

compiler/backend/src/is_leafable.cpp

```
#include "is_leafable.hpp"

#include "t_types.hpp"
```

```
bool is_leafable(const Type& type) {
    return (false
        || std::get_if<std::shared_ptr<TypePointer>>(&type) != nullptr
        || std::get_if<std::shared_ptr<TypeRotten>>(&type) != nullptr
    );
}
```

compiler/backend/src/is_leafable.hpp

```
#pragma once

#include "t_types.hpp"

bool is_leafable(const Type& type);
```

compiler/backend/src/is_mapwrappable.cpp

```
#include "is_mapwrappable.hpp"
#include "t_types_fwd.hpp"

bool is_mapwrappable(const Type& type) {
    return (false
        || std::holds_alternative<std::shared_ptr<TypeMap>>(type)
        || std::holds_alternative<std::shared_ptr<TypePointer>>(type)
        || std::holds_alternative<std::shared_ptr<TypeRotten>>(type)
    );
}
```

compiler/backend/src/is_mapwrappable.hpp

```
#pragma once

#include "t_types_fwd.hpp"

bool is_mapwrappable(const Type& type);
```

compiler/backend/src/is_structval_leaf_physical.cpp

```
#include "is_structval_leaf_physical.hpp"
#include "t_smooth.hpp"
#include "smooth_type.hpp"
#include "is_type_singletonish.hpp"

bool is_structval_leaf_physical(std::shared_ptr<SmoothStructval> structval) {
    return (true
        && structval->has_leaf
        && !is_type_singletonish(smooth_type(structval->leaf.value()))
    );
}
```

compiler/backend/src/is_structval_leaf_physical.hpp

```
#pragma once
```

```
#include <memory>
#include "t_smooth_fwd.hpp"

bool is_structval_leaf_physical(std::shared_ptr<SmoothStructval> structval);
```

compiler/backend/src/is_structwrappable.cpp

```
#include "is_structwrappable.hpp"
#include "t_smooth_fwd.hpp"

bool is_structwrappable(const Smooth& smooth) {
    return (false
        || std::holds_alternative<std::shared_ptr<SmoothStructval>>(smooth)
        || std::holds_alternative<std::shared_ptr<SmoothPointer>>(smooth)
        || std::holds_alternative<std::shared_ptr<SmoothInt>>(smooth)
        || std::holds_alternative<std::shared_ptr<SmoothFloat>>(smooth)
    );
}
```

compiler/backend/src/is_structwrappable.hpp

```
#pragma once

#include "t_smooth_fwd.hpp"

bool is_structwrappable(const Smooth& smooth);
```

compiler/backend/src/is_subset_type.cpp

```
#include "is_subset_type.hpp"
#include "is_subset_type_map.hpp"
#include "is_subset_type_enum.hpp"
#include "is_leafable.hpp"
#include "wrap_leafable.hpp"
#include "get_underlying_type.hpp"
#include "t_types.hpp"

bool is_subset_type(const Type& type_a, const Type& type_b) {
    const Type a = get_underlying_type(type_a);
    const Type b = get_underlying_type(type_b);

    if (std::get_if<std::shared_ptr<TypeMap>>(&a) && is_leafable(b)) {
        Type wrapped = std::make_shared<TypeMap>(wrap_leafable(b));
        return is_subset_type(a, wrapped);
    }

    if (std::get_if<std::shared_ptr<TypeMap>>(&b) && is_leafable(a)) {
        Type wrapped = std::make_shared<TypeMap>(wrap_leafable(a));
        return is_subset_type(wrapped, b);
    }

    if (a.index() != b.index()) {
        return false;
    }

    if (auto p_v_map_a = std::get_if<std::shared_ptr<TypeMap>>(&a)) {
        auto p_v_map_b = std::get_if<std::shared_ptr<TypeMap>>(&b);
        return is_subset_type_map(**p_v_map_a, **p_v_map_b);
    }

    if (auto p_v_ptr_a = std::get_if<std::shared_ptr<TypePointer>>(&a)) {
```

```
        auto p_v_ptr_b = std::get_if<std::shared_ptr<TypePointer>>(&b);
        return is_subset_type(*(*p_v_ptr_a)->target, *(*p_v_ptr_b)->target);
    }

    if (auto p_v_rotten_a = std::get_if<std::shared_ptr<TypeRotten>>(&a)) {
        auto p_v_rotten_b = std::get_if<std::shared_ptr<TypeRotten>>(&b);
        return (*p_v_rotten_a)->type_str == (*p_v_rotten_b)->type_str; // language will
    ↪ require explicit casting for now.
    }

    if (auto p_v_enum_a = std::get_if<std::shared_ptr<TypeEnum>>(&a)) {
        auto p_v_enum_b = std::get_if<std::shared_ptr<TypeEnum>>(&b);
        return is_subset_type_enum(**p_v_enum_a, **p_v_enum_b);
    }

    // for now we pretend everything else is incompatible.
    return false;
}
```

compiler/backend/src/is_subset_type.hpp

```
#pragma once

#include "t_types_fwd.hpp"

bool is_subset_type(const Type& type_a, const Type& type_b);
```

compiler/backend/src/is_subset_type_enum.cpp

```
#include "is_subset_type_enum.hpp"

bool is_subset_type_enum(const TypeEnum& enum_a, const TypeEnum& enum_b) {
    if (enum_a.hardsym.has_value()) {
        bool found = false;

        for (const auto& sym : enum_b.syms) {
            if (sym == enum_a.hardsym.value()) {
                found = true;
                break;
            }
        }

        if (!found) {
            return false;
        }
    }

    for (const auto& sym : enum_a.syms) {
        bool found = false;

        for (const auto& b_sym : enum_b.syms) {
            if (sym == b_sym) {
                found = true;
                break;
            }
        }

        if (!found) {
            return false;
        }
    }

    return true;
}
```

compiler/backend/src/is_subset_type_enum.hpp

```
#pragma once

#include "t_types.hpp"

bool is_subset_type_enum(const TypeEnum& enum_a, const TypeEnum& enum_b);
```

compiler/backend/src/is_subset_type_map.cpp

```
#include "is_subset_type_map.hpp"
#include "is_subset_type.hpp"

bool is_subset_type_map(const TypeMap& map_a, const TypeMap& map_b) {
    bool a_has_leaf = map_a.leaf_type.has_value() || map_a.leaf_hardval.has_value();
    bool a_needs_leaf = map_b.leaf_type.has_value() || map_b.leaf_hardval.has_value();

    if (a_needs_leaf && !a_has_leaf) {
        return false;
    }

    for (const auto& [sym_name_b, sym_type_b] : map_b.sym_inputs) { // todo: not sure if
    ↪  i got this backwards.
        auto it = map_a.sym_inputs.find(sym_name_b);

        if (it == map_a.sym_inputs.end()) {
            return false;
        }

        if (!is_subset_type(*it->second, *sym_type_b)) {
            return false;
        }
    }

    return true;
}
```

compiler/backend/src/is_subset_type_map.hpp

```
#pragma once

#include "t_types.hpp"

bool is_subset_type_map(const TypeMap& map_a, const TypeMap& map_b);
```

compiler/backend/src/is_type_singletonish.cpp

```
#include "is_type_singletonish.hpp"
#include "get_underlying_type.hpp"
#include "t_types.hpp"

bool is_type_singletonish(const Type& type) { // is there only one runtime value that
    ↪ the type encodes? if so, can be made static.
    Type underlying = get_underlying_type(type);

    if (auto p_v_pointer = std::get_if<std::shared_ptr<TypePointer>>(&underlying)) {
        // previously hardval determined whether pointer was singletonish, however this
    ↪ is wrong.
        // hardval means the underlying data is singletonish, but this has nothing to do
    ↪  with the pointer itself, which may be mutated (e.g. if in a variable).
```

```
        // we kinda just assume pointers can take on any value, and if it genuinely has
    ↪ constant data,
        // llvm would be good enough at optimizing this (probably fairly common
    ↪ optimization).

        return false;
    }

    if (auto p_v_enum = std::get_if<std::shared_ptr<TypeEnum>>(&underlying)) {
        // really we need a syms_in_use in the future, because we might have only one "
    ↪ sym in use", but the sym is still encoded as a non-undefined number.
        return (*p_v_enum)->syms.size() == 1;
    }

    if (auto p_v_void = std::get_if<std::shared_ptr<TypeVoid>>(&underlying)) {
        return true;
    }

    if (auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&underlying)) {
        return false; // unnecessary optimization, as llvm should optimize an empty
    ↪ struct automatically.

        const TypeMap& map = **p_v_map;

        if (map.leaf_type.has_value()) {
            if (!is_type_singletonish(map.leaf_type.value())) {
                return false;
            }
        }

        for (const auto& [_sym_name, sym_type_ptr] : map.sym_inputs) {
            if (!is_type_singletonish(*sym_type_ptr)) {
                return false;
            }
        }

        return true;
    }

    return false;
}
```

compiler/backend/src/is_type_singletonish.hpp

```
#pragma once

#include "t_types_fwd.hpp"

bool is_type_singletonish(const Type& type);
```

compiler/backend/src/llvm_flexi_type.cpp

```
#include "llvm_flexi_type.hpp"
#include "llvm_value.hpp"
#include "t_smooth.hpp"
#include "llvm/IR/DerivedTypes.h"

// this is a definitive record of what the type would be if we were genuinely storing a
    ↪ value.
// it bypasses the value is poison (i.e. unpopulated) and thus having a void type or
    ↪ similar.

llvm::Type* llvm_flexi_type(const Smooth smooth) {
    if (auto p_v_structval = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth)) {
```

```
        const auto& field_smooths = (*p_v_structval)->field_smooths;

        if (field_smooths.empty()) {
            return llvm_value(smooth)->getType();
        }

        std::vector<llvm::Type*> field_types;
        field_types.reserve(field_smooths.size());

        for (const auto& field_smooth : field_smooths) {
            field_types.push_back(llvm_flexi_type(field_smooth));
        }

        return llvm::StructType::get(field_types[0]->getContext(), field_types);
    }

    if (auto p_v_void_int = std::get_if<std::shared_ptr<SmoothVoidInt>>(&smooth)) {
        return (*p_v_void_int)->flexi_type;
    }

    if (auto p_v_void_float = std::get_if<std::shared_ptr<SmoothVoidFloat>>(&smooth)) {
        return (*p_v_void_float)->flexi_type;
    }

    return llvm_value(smooth)->getType();
}
```

compiler/backend/src/llvm_flexi_type.hpp

```
#pragma once

#include "t_smooth_fwd.hpp"
#include "llvm/IR/Type.h"

llvm::Type* llvm_flexi_type(const Smooth smooth);
```

compiler/backend/src/llvm_to_smooth.cpp

```
#include "llvm_to_smooth.hpp"
#include "get_underlying_type.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"
#include "produce_call_func.hpp"
#include "is_type_singletonish.hpp"
#include "evaluate_singletonish.hpp"
#include "rotten_int_info.hpp"
#include "rotten_float_info.hpp"
#include "llvm/IR/DerivedTypes.h"

// this is such a hacky system and needs to be rid of at some point.

Smooth llvm_to_smooth(IrGenCtx& igc, const Type& type, llvm::Value* value) {
    Type underlying = get_underlying_type(type);

    if (auto p_v_rotten = std::get_if<std::shared_ptr<TypeRotten>>(&underlying)) {
        const std::string& type_str = (*p_v_rotten)->type_str;

        const bool is_float = (false
            || type_str == "f16"
            || type_str == "f32"
            || type_str == "f64"
            || type_str == "f128"
        );
```

```
    if (value->getType()->isStructTy()) { // really type information should be
↪ telling us this, but this hack will do for now.
        if (auto info = rotten_float_info(*p_v_rotten)) {
            llvm::Type* flexi_type = nullptr;

            if (info->bits == 16) flexi_type = llvm::Type::getHalfTy(value->
↪ getContext());
            else if (info->bits == 32) flexi_type = llvm::Type::getFloatTy(value->
↪ getContext());
            else if (info->bits == 64) flexi_type = llvm::Type::getDoubleTy(value->
↪ getContext());
            else if (info->bits == 128) flexi_type = llvm::Type::getFP128Ty(value->
↪ getContext());

            return std::make_shared<SmoothVoidFloat>(SmoothVoidFloat{
                type,
                flexi_type,
                value,
            });
        } else if (auto info = rotten_int_info(*p_v_rotten)) {
            llvm::Type* flexi_type = llvm::IntegerType::get(value->getContext(),
↪ info->bits);

            return std::make_shared<SmoothVoidInt>(SmoothVoidInt{
                type,
                flexi_type,
                value,
            });
        }

        fprintf(stderr, "Failed to extract info from rotten.");
        exit(1);
    }

    if (is_float) {
        return std::make_shared<SmoothFloat>(SmoothFloat{
            type,
            value,
        });
    }

    return std::make_shared<SmoothInt>(SmoothInt{
        type,
        value,
    });
}

if (auto p_v_enum = std::get_if<std::shared_ptr<TypeEnum>>(&underlying)) {
    return std::make_shared<SmoothEnum>(SmoothEnum{
        type,
        value,
    });
}

if (std::get_if<std::shared_ptr<TypePointer>>(&underlying)) {
    return std::make_shared<SmoothPointer>(SmoothPointer{
        type,
        value,
    });
}

if (auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&underlying)) {
    bool has_leaf = (*p_v_map)->leaf_type.has_value();

    if (true
        && (*p_v_map)->leaf_hardval.has_value()
        && !(*p_v_map)->leaf_type.has_value()
    ) {
        fprintf(stderr, "serious invariant violation - new refactor does not support
↪  hardval without type.\n");
```

```
            exit(1);
        }

        std::optional<Smooth> leaf = std::nullopt;
        std::vector<Smooth> field_smooths;
        unsigned int field_index = 0;

        if (has_leaf) {
            if (is_type_singletonish((*p_v_map)->leaf_type.value())) {
                leaf = evaluate_singletonish(igc, (*p_v_map)->leaf_type.value());
            } else {
                llvm::Value* leaf_value = igc.builder.CreateExtractValue(value,
    ↪ field_index);
                leaf = llvm_to_smooth(igc, (*p_v_map)->leaf_type.value(), leaf_value);
                field_smooths.push_back(leaf.value());
                field_index += 1;
            }
        }

        for (const auto& [sym_name, sym_type] : (*p_v_map)->sym_inputs) {
            if (is_type_singletonish(*sym_type)) {
                continue;
            }

            llvm::Value* sym_value = igc.builder.CreateExtractValue(value, field_index);
            field_smooths.push_back(llvm_to_smooth(igc, *sym_type, sym_value));

            field_index += 1;
        }

        return std::make_shared<SmoothStructval>(SmoothStructval{
            type,
            value,
            has_leaf,
            leaf,
            produce_call_func(igc, *p_v_map),
            field_smooths,
        });
    }

    const char* name = std::visit([](auto&& v) { return typeid(*v).name(); }, type);
    const char* name2 = std::visit([](auto&& v) { return typeid(*v).name(); },
    ↪ underlying);
    fprintf(stderr, "not handled llvm_to_smooth.\n");
    fprintf(stderr, "got %s underlying as %s\n", name, name2);
    exit(1);
}
```

compiler/backend/src/llvm_to_smooth.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_types_fwd.hpp"
#include "llvm/IR/Value.h"

Smooth llvm_to_smooth(IrGenCtx& igc, const Type& type, llvm::Value* raw);
```

compiler/backend/src/llvm_to_smooth_bool.cpp

```
#include "llvm_to_smooth_bool.hpp"
#include "evaluate_smooth.hpp"
#include "t_types.hpp"
```

```
#include "t_smooth.hpp"
#include "preinstantiated_types.hpp"

std::shared_ptr<SmoothEnum> llvm_to_smooth_bool(
    IrGenCtx& igc,
    llvm::Value* value
) {
    auto smooth_enum_false = std::get<std::shared_ptr<SmoothEnum>>(evaluate_smooth(igc,
    ↪ Type(type_false)));
    auto smooth_enum_true = std::get<std::shared_ptr<SmoothEnum>>(evaluate_smooth(igc,
    ↪ Type(type_true)));

    llvm::Value* of_interest = igc.builder.CreateSelect(value, smooth_enum_true->value,
    ↪ smooth_enum_false->value);

    return std::make_shared<SmoothEnum>(SmoothEnum{
        Type(type_bool),
        of_interest,
    });
}
```

compiler/backend/src/llvm_to_smooth_bool.hpp

```
#pragma once

#include "llvm/IR/Value.h"
#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"

std::shared_ptr<SmoothEnum> llvm_to_smooth_bool(
    IrGenCtx& igc,
    llvm::Value* i1_val
);
```

compiler/backend/src/llvm_value.cpp

```
#include "llvm_value.hpp"
#include "t_smooth.hpp"

llvm::Value* llvm_value(const Smooth& smooth) {
    if (auto p_v_structval = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth)) {
        return (*p_v_structval)->value;
    }

    if (auto p_v_pointer = std::get_if<std::shared_ptr<SmoothPointer>>(&smooth)) {
        return (*p_v_pointer)->value;
    }

    if (auto p_v_enum = std::get_if<std::shared_ptr<SmoothEnum>>(&smooth)) {
        return (*p_v_enum)->value;
    }

    if (auto p_v_int = std::get_if<std::shared_ptr<SmoothInt>>(&smooth)) {
        return (*p_v_int)->value;
    }

    if (auto p_v_float = std::get_if<std::shared_ptr<SmoothFloat>>(&smooth)) {
        return (*p_v_float)->value;
    }

    if (auto p_v_void = std::get_if<std::shared_ptr<SmoothVoid>>(&smooth)) {
        return (*p_v_void)->value;
    }
```

```
    if (auto p_v_void_int = std::get_if<std::shared_ptr<SmoothVoidInt>>(&smooth)) {
        return (*p_v_void_int)->value;
    }

    if (auto p_v_void_float = std::get_if<std::shared_ptr<SmoothVoidFloat>>(&smooth)) {
        return (*p_v_void_float)->value;
    }

    fprintf(stderr, "This kind of a smooth does not have llvm value associated with it
    ↪ .");
    exit(1);
}
```

compiler/backend/src/llvm_value.hpp

```
#pragma once

#include "llvm/IR/Value.h"
#include "t_smooth_fwd.hpp"

llvm::Value* llvm_value(const Smooth& smooth);
```

compiler/backend/src/main.cpp

```
#include <cstdio>
#include <fstream>
#include <string>
#include "dependencies/json.hpp"
#include "create_type_symbol_table.hpp"
#include "create_value_symbol_table.hpp"
#include "evaluate_hardval.hpp"
#include "evaluate_smooth.hpp"
#include "process_map_body.hpp"
#include "normalize_type.hpp"
#include "promote_to_underlying.hpp"
#include "t_hardval.hpp"
#include "t_instructions.hpp"
#include "t_ctx.hpp"

#include "llvm/IR/LLVMContext.h"
#include "llvm/IR/Module.h"
#include "llvm/IR/IRBuilder.h"
#include "llvm/IR/Verifier.h"
#include "llvm/IR/Constants.h"
#include "llvm/IR/DerivedTypes.h"
#include "llvm/ADT/APInt.h"
#include "llvm/ADT/APFloat.h"
#include "llvm/Support/raw_ostream.h"
#include "llvm/Support/FileSystem.h"
#include "llvm/Support/TargetSelect.h"
#include "llvm/Target/TargetMachine.h"
#include "llvm/Target/TargetOptions.h"
#include "llvm/MC/TargetRegistry.h"
#include "llvm/IR/LegacyPassManager.h"
#include "llvm/TargetParser/Host.h"

void gen_module_binary(std::shared_ptr<TypeOrchCtx> toc, const json& create_data) {
    // the amount of boilerplate is crazy lol
    printf("Generating module binary!!\n");

    llvm::LLVMContext context;
    llvm::Module module("foobar", context);
    llvm::IRBuilder<> builder(context);
```

```
auto target_triple = llvm::sys::getDefaultTargetTriple();
module.setTargetTriple(target_triple);

std::vector<llvm::Type*> puts_args = { llvm::Type::getInt8PtrTy(context) };

llvm::FunctionType* puts_type = llvm::FunctionType::get(
    llvm::Type::getInt32Ty(context),
    puts_args,
    false
);

llvm::FunctionCallee puts_func = module.getOrInsertFunction("puts", puts_type);

// Takes in (i64 chunk, i8 bl, i8 br).
llvm::Function* log_data_func = [&]() -> llvm::Function* {
    // there's only one worse thing than handwriting an LLVM IR function, and that's
↪  handwiriting C++ code that generates one lol.
    // but by relying on handrwritten functions just for debugging functions (like
↪ log_d), we don't depend on the language's interoperability layer to be in working
↪ condition, allowing interoparability to be developed at a later stage.

    llvm::Type* i8  = llvm::Type::getInt8Ty(context);
    llvm::Type* i32 = llvm::Type::getInt32Ty(context);
    llvm::Type* i64 = llvm::Type::getInt64Ty(context);

    llvm::FunctionType* fn_type = llvm::FunctionType::get(
        llvm::Type::getVoidTy(context),
        { i64, i8, i8 },
        false
    );

    llvm::Function* fn = llvm::Function::Create(
        fn_type,
        llvm::Function::PrivateLinkage,
        "__ec_log_data",
        module
    );

    fn->addFnAttr(llvm::Attribute::NoInline);

    llvm::Argument* arg_chunk = fn->getArg(0);
    llvm::Argument* arg_bl    = fn->getArg(1);
    llvm::Argument* arg_br    = fn->getArg(2);

    llvm::IRBuilder<> b(context);

    auto* blk = llvm::BasicBlock::Create(context, "entry", fn);
    b.SetInsertPoint(blk);

    // if i've counted right, 51 should be enough characters for a single row.
    // 2 (prefix) + 1 (space) + 16 (hex) + 1 (space) + 8 (ascii) + 1 (space) + log10
↪ (2^64) (decimal) + 1 (null terminator) = 50

    llvm::Value* buf = b.CreateAlloca(i8, b.getInt32(51), "buf");

    int pos = 0;

    auto store_valu = [&](llvm::Value* val, int at) {
        // get-element-pointer generates offset on pointer in platform-agnostic
↪ manner.
        b.CreateStore(val, b.CreateGEP(i8, buf, b.getInt32(at)));
    };

    auto store_char = [&](char ch) {
        store_valu(b.getInt8(ch), pos++);
    };

    // 1. PREFIX

    store_valu(arg_bl, pos++);
```

```
    store_valu(arg_br, pos++);
    store_char(' ');

    // 2. HEX

    for (int nibble = 15; nibble >= 0; nibble--) { // most significnant first
        int shift_amt = nibble * 4;
        llvm::Value* shifted = b.CreateLShr(arg_chunk, b.getInt64(shift_amt));
        llvm::Value* masked = b.CreateAnd(shifted, b.getInt64(0xF)); // grab 4 bits
↪ at a times.
        llvm::Value* truncated = b.CreateTrunc(masked, i8); // dealing with 8 bit
↪ characters
        llvm::Value* decimal  = b.CreateAdd(truncated, b.getInt8('0'));
        llvm::Value* hex  = b.CreateAdd(truncated, b.getInt8('a' - 10));
        llvm::Value* is_less_than_ten = b.CreateICmpULT(truncated, b.getInt8(10));
        llvm::Value* hex_char = b.CreateSelect(is_less_than_ten, decimal, hex);
        store_valu(hex_char, pos++);
    }

    store_char(' ');

    // 3. ASCII

    for (int byte_idx = 7; byte_idx >= 0; byte_idx--) { // msb first
        int shift_amt = byte_idx * 8;
        llvm::Value* shifted = b.CreateLShr(arg_chunk, b.getInt64(shift_amt));
        llvm::Value* truncated = b.CreateTrunc(shifted, i8); // dealing with 8 bit
↪ characters, and grabbing 8 bits at a time.
        // printable range is generally recognized as 0x20 to 0x7e
        // other chars may do funny business: move cursor, play beep, etc , so are
↪ replaced with '.'
        llvm::Value* high_enough = b.CreateICmpUGE(truncated, b.getInt8(0x20));
        llvm::Value* low_enough = b.CreateICmpULE(truncated, b.getInt8(0x7e));
        llvm::Value* is_printable = b.CreateAnd(high_enough, low_enough);
        llvm::Value* ascii_char = b.CreateSelect(is_printable, truncated, b.getInt8
↪ ('.'));
        store_valu(ascii_char, pos++);
    }

    store_char(' ');

    // 4. DECIMAL (unsigned only)

    int dec_tail = 48; // we are floating to the right and working our way to the
↪ left

    auto* block_dec_loop = llvm::BasicBlock::Create(context, "dec_loop", fn);
    auto* block_dec_done = llvm::BasicBlock::Create(context, "dec_done", fn);
    auto* block_fill_loop = llvm::BasicBlock::Create(context, "fill_loop", fn);
    auto* block_fill_done = llvm::BasicBlock::Create(context, "fill_done", fn);

    b.CreateStore(b.getInt8(0), b.CreateGEP(i8, buf, b.getInt32(dec_tail + 1))); //
↪ null terminator

    llvm::Value* alloca_idx = b.CreateAlloca(i32, nullptr, "idx");
    llvm::Value* alloca_dec_value = b.CreateAlloca(i64, nullptr, "dec_value");
    b.CreateStore(b.getInt32(dec_tail), alloca_idx);
    b.CreateStore(arg_chunk, alloca_dec_value);
    b.CreateBr(block_dec_loop);

    b.SetInsertPoint(block_dec_loop);
    llvm::Value* dec_value = b.CreateLoad(i64, alloca_dec_value);
    llvm::Value* digit_rem = b.CreateURem(dec_value, b.getInt64(10));
    llvm::Value* digit_truncated = b.CreateTrunc(digit_rem, i8);
    llvm::Value* digit_char = b.CreateAdd(digit_truncated, b.getInt8('0'));
    llvm::Value* idx = b.CreateLoad(i32, alloca_idx);
    b.CreateStore(digit_char, b.CreateGEP(i8, buf, idx));
    b.CreateStore(b.CreateSub(idx, b.getInt32(1)), alloca_idx);
    llvm::Value* dec_value_next = b.CreateUDiv(dec_value, b.getInt64(10));
    b.CreateStore(dec_value_next, alloca_dec_value);
```

```
    b.CreateCondBr(b.CreateICmpNE(dec_value_next, b.getInt64(0)), block_dec_loop,
↪ block_dec_done);

    b.SetInsertPoint(block_dec_done);
    b.CreateBr(block_fill_loop);

    b.SetInsertPoint(block_fill_loop);
    idx = b.CreateLoad(i32, alloca_idx);
    b.CreateStore(b.getInt8(' '), b.CreateGEP(i8, buf, idx));
    b.CreateStore(b.CreateSub(idx, b.getInt32(1)), alloca_idx);
    b.CreateCondBr(b.CreateICmpSGE(b.CreateLoad(i32, alloca_idx), b.getInt32(pos)),
↪ block_fill_loop, block_fill_done);

    b.SetInsertPoint(block_fill_done);
    b.CreateCall(puts_func, { buf });
    b.CreateRetVoid();

    return fn;
}();

// Takes in (i8* ptr, i64 byte_count).
// byte_count of -1 means null-terminated.
llvm::Function* log_data_deref_func = [&]() -> llvm::Function* {
    llvm::Type* i1 = llvm::Type::getInt1Ty(context);
    llvm::Type* i8 = llvm::Type::getInt8Ty(context);
    llvm::Type* i64 = llvm::Type::getInt64Ty(context);
    llvm::Type* i8ptr = llvm::Type::getInt8PtrTy(context);

    llvm::FunctionType* fn_type = llvm::FunctionType::get(
        llvm::Type::getVoidTy(context),
        { i8ptr, i64 },
        false
    );

    llvm::Function* fn = llvm::Function::Create(
        fn_type,
        llvm::Function::PrivateLinkage,
        "__ec_log_data_deref",
        module
    );

    fn->addFnAttr(llvm::Attribute::NoInline);

    llvm::Argument* arg_ptr = fn->getArg(0);
    llvm::Argument* arg_byte_count = fn->getArg(1);

    llvm::IRBuilder<> b(context);

    // uses an outer loop for blocks and inner loop to deal with potential partial
↪ blocks.
    // because of "[---]", we have to implement lookahead so keep track of both
↪ chunk and chunk_prev.

    auto* block_entry = llvm::BasicBlock::Create(context, "entry", fn);
    auto* block_outer_loop = llvm::BasicBlock::Create(context, "outer_loop", fn);
    auto* block_inner_setup = llvm::BasicBlock::Create(context, "inner_setup", fn);
    auto* block_inner_loop = llvm::BasicBlock::Create(context, "inner_loop", fn);
    auto* block_inner_body = llvm::BasicBlock::Create(context, "inner_body", fn);
    auto* block_inner_body3 = llvm::BasicBlock::Create(context, "inner_body3", fn);
    auto* block_inner_body2 = llvm::BasicBlock::Create(context, "inner_body2", fn);
    auto* block_render = llvm::BasicBlock::Create(context, "render", fn);
    auto* block_advance2 = llvm::BasicBlock::Create(context, "advance2", fn);
    auto* block_advance3 = llvm::BasicBlock::Create(context, "advance3", fn);
    auto* block_flush = llvm::BasicBlock::Create(context, "flush", fn);
    auto* block_flush_final = llvm::BasicBlock::Create(context, "flush_final", fn);
    auto* block_done = llvm::BasicBlock::Create(context, "done", fn);

    b.SetInsertPoint(block_entry);
    llvm::Value* alloca_outer_idx = b.CreateAlloca(i64, nullptr, "outer_idx");
    llvm::Value* alloca_inner_idx = b.CreateAlloca(i64, nullptr, "inner_idx");
```

```
    llvm::Value* alloca_chunk = b.CreateAlloca(i64, nullptr, "chunk");
    llvm::Value* alloca_chunk_prev = b.CreateAlloca(i64, nullptr, "chunk_prev");
    llvm::Value* alloca_has_prev = b.CreateAlloca(i1, nullptr, "has_prev");
    llvm::Value* alloca_is_first = b.CreateAlloca(i1, nullptr, "is_first");
    llvm::Value* alloca_is_game_over = b.CreateAlloca(i1, nullptr, "is_game_over");
    llvm::Value* is_nullterm = b.CreateICmpEQ(arg_byte_count, b.getInt64(-1), "
↪ is_nullterm");
    b.CreateStore(b.getInt64(0), alloca_outer_idx);
    b.CreateStore(b.getFalse(), alloca_has_prev);
    b.CreateStore(b.getFalse(), alloca_is_game_over);
    b.CreateBr(block_outer_loop);

    b.SetInsertPoint(block_outer_loop);
    llvm::Value* outer_idx = b.CreateLoad(i64, alloca_outer_idx);
    b.CreateCondBr(b.CreateLoad(i1, alloca_is_game_over), block_flush,
↪ block_inner_setup); // i feel like this won't handle zero rows properly.

    b.SetInsertPoint(block_inner_setup);
    b.CreateStore(b.getInt64(0), alloca_chunk);
    b.CreateStore(b.getInt64(0), alloca_inner_idx);
    b.CreateBr(block_inner_loop);

    b.SetInsertPoint(block_inner_loop);
    llvm::Value* inner_idx = b.CreateLoad(i64, alloca_inner_idx);
    llvm::Value* byte_idx = b.CreateAdd(outer_idx, inner_idx);
    llvm::Value* is_inner_done = b.CreateICmpEQ(inner_idx, b.getInt64(8));
    b.CreateCondBr(is_inner_done, block_render, block_inner_body);

    b.SetInsertPoint(block_inner_body);
    llvm::Value* byte_ptr = b.CreateGEP(i8, arg_ptr, byte_idx);
    llvm::Value* byte_value = b.CreateLoad(i8, byte_ptr);
    llvm::Value* is_null = b.CreateAnd(is_nullterm, b.CreateICmpEQ(byte_value, b.
↪ getInt8(0)));
    llvm::Value* is_ended = b.CreateAnd(b.CreateNot(is_nullterm), b.CreateICmpEQ(b.
↪ CreateAdd(byte_idx, b.getInt64(1)), arg_byte_count));
    llvm::Value* is_game_over = b.CreateOr(is_null, is_ended);
    b.CreateStore(is_game_over, alloca_is_game_over);
    llvm::Value* is_empty_chunk = b.CreateICmpEQ(inner_idx, b.getInt64(0));
    b.CreateCondBr(is_null, block_inner_body3, block_inner_body2);

    b.SetInsertPoint(block_inner_body3);
    b.CreateCondBr(is_empty_chunk, block_flush, block_render);

    b.SetInsertPoint(block_inner_body2);
    llvm::Value* extended = b.CreateZExt(byte_value, i64);
    llvm::Value* shift_amt = b.CreateSub(b.getInt64(56), b.CreateMul(inner_idx, b.
↪ getInt64(8))); // start from most significant side
    llvm::Value* chunk_original = b.CreateLoad(i64, alloca_chunk);
    b.CreateStore(b.CreateOr(chunk_original, b.CreateShl(extended, shift_amt)),
↪ alloca_chunk);
    b.CreateStore(b.CreateAdd(inner_idx, b.getInt64(1)), alloca_inner_idx);
    b.CreateCondBr(is_game_over, block_render, block_inner_loop);

    b.SetInsertPoint(block_render);
    llvm::Value* chunk = b.CreateLoad(i64, alloca_chunk);
    llvm::Value* has_prev = b.CreateLoad(i1, alloca_has_prev);
    b.CreateCondBr(has_prev, block_advance2, block_advance3);

    b.SetInsertPoint(block_advance2);
    llvm::Value* chunk_prev = b.CreateLoad(i64, alloca_chunk_prev);
    llvm::Value* is_first = b.CreateLoad(i1, alloca_is_first);
    llvm::Value* bl_of_interest = b.CreateSelect(is_first, b.getInt8('['), b.getInt8
↪ ('-'));
    b.CreateCall(log_data_func, { chunk_prev, bl_of_interest, b.getInt8('-') });
    b.CreateBr(block_advance3);

    b.SetInsertPoint(block_advance3);
    llvm::Value* was_first = b.CreateNot(b.CreateLoad(i1, alloca_has_prev));
    b.CreateStore(chunk, alloca_chunk_prev);
    b.CreateStore(was_first, alloca_is_first);
```

```
    b.CreateStore(b.getTrue(), alloca_has_prev);
    b.CreateStore(b.CreateAdd(outer_idx, b.getInt64(8)), alloca_outer_idx);
    b.CreateBr(block_outer_loop);

    b.SetInsertPoint(block_flush);
    b.CreateCondBr(b.CreateLoad(i1, alloca_has_prev), block_flush_final, block_done)
↪ ;

    b.SetInsertPoint(block_flush_final);
    llvm::Value* chunk_final = b.CreateLoad(i64, alloca_chunk_prev);
    b.CreateCall(log_data_func, { chunk_final, b.CreateSelect(b.CreateLoad(i1,
↪ alloca_is_first), b.getInt8('['), b.getInt8('-')), b.getInt8(']') });
    b.CreateBr(block_done);

    b.SetInsertPoint(block_done);
    b.CreateRetVoid();

    return fn;
}();

llvm::FunctionType* main_type = llvm::FunctionType::get(
    llvm::Type::getInt32Ty(context),
    false
);

llvm::Function* main_func = llvm::Function::Create(
    main_type,
    llvm::Function::ExternalLinkage,
    "main",
    module
);

llvm::BasicBlock* entry = llvm::BasicBlock::Create(context, "entry", main_func);
builder.SetInsertPoint(entry);

const char* message;

if (create_data.is_null()) {
    message = "No module entrypoint";
} else {
    message = "Module entrypoint found";
}

llvm::Value* message_str = builder.CreateGlobalStringPtr(message);
builder.CreateCall(puts_func, { message_str });

if (!create_data.is_null()) {
    if (!create_data.contains("description")) {
        fprintf(stderr, "Where is .description gone\n");
        exit(1);
    }

    const json& description = create_data["description"];

    Type normalized = normalize_type(*toc, description);
    auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&normalized);

    if (!p_v_map) {
        fprintf(stderr, "description of create must be map, not something else.\n");
        exit(1);
    }

    TypeMap v_map = **p_v_map;

    if (v_map.call_output_type == nullptr) {
        fprintf(stderr, "at the moment we need an output which I suppose makes sense
↪  for a create block\n");
        exit(1);
    }
```

```
    std::shared_ptr<ValueSymbolTable> value_table = std::make_shared<
↪ ValueSymbolTable>(create_value_symbol_table());

    IrGenCtx igc = {
        context,
        module,
        builder,
        value_table,
        puts_func,
        log_data_func,
        log_data_deref_func,
        nullptr,
        toc,
    };

    process_map_body(igc, *v_map.call_output_type);
}

builder.CreateRet(llvm::ConstantInt::get(llvm::Type::getInt32Ty(context), 0));

std::string verify_error;
llvm::raw_string_ostream verify_stream(verify_error);

if (llvm::verifyModule(module, &verify_stream)) {
    fprintf(stderr, "Verification of generated module did not go smoothly, because:%
↪ s\n", verify_error.c_str());
    exit(1);
}

printf("Module generation went fantastically.\n");

std::error_code error_code;
llvm::raw_fd_ostream ir_file("/app/out/hello.ll", error_code, llvm::sys::fs::OF_None
↪ );

if (error_code) {
    fprintf(stderr, "Opening file error: %s\n", error_code.message().c_str());
    exit(1);
}

module.print(ir_file, nullptr);
ir_file.close();

llvm::InitializeAllTargetInfos();
llvm::InitializeAllTargets();
llvm::InitializeAllTargetMCs();
llvm::InitializeAllAsmParsers();
llvm::InitializeAllAsmPrinters();

std::string error;
const llvm::Target* target = llvm::TargetRegistry::lookupTarget(target_triple, error
↪ );

if (!target) {
    fprintf(stderr, "Couldn't locate desired target because %s\n", error.c_str());
    exit(1);
}

llvm::TargetOptions opt;

llvm::TargetMachine* target_machine = target->createTargetMachine(
    target_triple,
    "generic",
    "",
    opt,
    llvm::Reloc::PIC_
);

module.setDataLayout(target_machine->createDataLayout());
```

```
    llvm::raw_fd_ostream obj_file("/app/out/hello.o", error_code, llvm::sys::fs::OF_None
    ↪ );

    if (error_code) {
        fprintf(stderr, "Opening file error because %s\n", error_code.message().c_str()
    ↪ ;
        delete target_machine;
        exit(1);
    }

    llvm::legacy::PassManager pass_manager;

    if (target_machine->addPassesToEmitFile(pass_manager, obj_file, nullptr, llvm::
    ↪ CodeGenFileType::CGFT_ObjectFile)) {
        fprintf(stderr, "Object file emission not supported for some reason.\n");
        delete target_machine;
        exit(1);
    }

    pass_manager.run(module);
    obj_file.close();

    printf("All done!");

    delete target_machine;
}

static void populate_type_symbol_table(std::shared_ptr<TypeOrchCtx> toc, const json&
    ↪ structure) {
    for (const auto& entry : structure) {
        if (!entry.contains("type") || entry["type"] != "type") {
            continue;
        }

        if (!entry.contains("trail")) {
            fprintf(stderr, "Missing .trail\n");
            exit(1);
        }

        if (!entry.contains("definition")) {
            fprintf(stderr, "Missing .definition\n");
            exit(1);
        }

        std::string trail = entry["trail"].get<std::string>();
        Type normalized = normalize_type(*toc, entry["definition"]);
        toc->type_table.set(trail, promote_to_underlying(normalized));
    }

    for (const auto& entry : structure) {
        if (!entry.contains("type") || entry["type"] != "top_extension") {
            continue;
        }

        if (!entry.contains("target_type")) {
            fprintf(stderr, "The .target_type is not present.\n");
            exit(1);
        }

        if (!entry.contains("case")) {
            fprintf(stderr, "The .case is also not present.\n");
            exit(1);
        }

        std::string target_type = entry["target_type"].get<std::string>();
        const json& ext_case = entry["case"];
        std::string ext_case_type = ext_case["type"].get<std::string>();

        if (ext_case_type == "extension_case_sym") {
            std::optional<UnderlyingType> existing = toc->type_table.get(target_type);
```

```
            if (!existing.has_value()) {
                fprintf(stderr, "Tried to equip a non-existent type alias with an extat.
    ↪  %s\n", target_type.c_str());
                exit(1);
            }

            auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&existing.value());

            if (!p_v_map) {
                fprintf(stderr, "Tried to extend a non-map type with an extat. %s\n",
    ↪ target_type.c_str());
                exit(1);
            }

            if (!ext_case.contains("trail") || !ext_case["trail"].is_array()) {
                fprintf(stderr, "Missing the .trail.\n");
                exit(1);
            }

            std::vector<std::string> trail;

            for (const auto& seg : ext_case["trail"]) {
                trail.push_back(seg.get<std::string>());
            }

            std::shared_ptr<TypeMap> the_existing = *p_v_map;

            for (size_t i = 0; i + 1 < trail.size(); i++) {
                const std::string segment = ":" + trail[i];

                if (the_existing->sym_inputs.find(segment) == the_existing->sym_inputs.
    ↪ end()) {
                    the_existing->sym_inputs[segment] = std::make_shared<Type>(std::
    ↪ make_shared<TypeMap>());
                }

                auto p_v_map2 = std::get_if<std::shared_ptr<TypeMap>>(the_existing->
    ↪ sym_inputs[segment].get());

                if (!p_v_map2) {
                    fprintf(stderr, "Nested trail check resulted in non-map %s\n",
    ↪ segment.c_str());
                    exit(1);
                }

                the_existing = *p_v_map2;
            }

            std::string leaf_name = ":" + trail.back();
            Type sym_type = normalize_type(*toc, ext_case["typeval"]);
            the_existing->sym_inputs[leaf_name] = std::make_shared<Type>(sym_type);

            auto v_sym = std::make_shared<InstructionSym>();
            v_sym->name = leaf_name;
            v_sym->typeval = the_existing->sym_inputs[leaf_name];
            the_existing->execution_sequence.push_back(v_sym);

            toc->type_table.set(target_type, promote_to_underlying(Type(*p_v_map)));
        } else {
            fprintf(stderr, "No more cases are implemented at this time %s\n",
    ↪ ext_case_type.c_str());
            exit(1);
        }
    }
}

int main(int argc, char* argv[]) {
    // disable out-of-order prints by just disabling buffering.
    setvbuf(stdout, NULL, _IONBF, 0);
```

```
    setvbuf(stderr, NULL, _IONBF, 0);

    std::string json_path = "/volume/in/frontend.out.json";

    printf("Obtaining json frontend result from %s\n", json_path.c_str());

    std::ifstream json_file(json_path);

    if (!json_file) {
        fprintf(stderr, "Error opening %s\n", json_path.c_str());
        return 1;
    }

    json frontend_data;

    try {
        json_file >> frontend_data;
    } catch (const json::parse_error& e) {
        fprintf(stderr, "JSON could not parse, because %s\n", e.what());
        return 1;
    }

    json_file.close();

    printf("json has been smoothly parsed.\n");

    auto& parse_output = frontend_data["parse_output"];

    auto toc = std::make_shared<TypeOrchCtx>();
    toc->type_table = create_type_symbol_table();

    if (!parse_output.contains("structure") || !parse_output["structure"].is_array()) {
        fprintf(stderr, "There is no .structure\n");
        exit(1);
    }

    populate_type_symbol_table(toc, parse_output["structure"]);

    json create_block = nullptr;
    if (parse_output.contains("create") && !parse_output["create"].is_null()) {
        create_block = parse_output["create"];
    }

    gen_module_binary(toc, create_block);

    auto& dir_node_translations = frontend_data["dir_node_translations"];

    for (auto& [uuid, node_data] : dir_node_translations.items()) {
        printf("%s -> %s\n", uuid.c_str(), node_data["path"].get<std::string>().c_str()
    ↪ ;
    }

    return 0;
}
```

compiler/backend/src/merge_smooth.cpp

```
#include <cstdio>
#include <cstdlib>
#include "merge_smooth.hpp"
#include "merge_smooth_value.hpp"
#include "merge_smooth_enum.hpp"
#include "merge_smooth_int.hpp"
#include "merge_smooth_float.hpp"
#include "t_smooth.hpp"

Smooth merge_smooth(IrGenCtx& igc, Smooth smooth_a, Smooth smooth_b) {
```

```
    // todo: i dunno how we are gonna handle merging the types of void

    if (auto p_v_void_a = std::get_if<std::shared_ptr<SmoothVoid>>(&smooth_a)) {
        return smooth_b;
    }

    if (auto p_v_void_b = std::get_if<std::shared_ptr<SmoothVoid>>(&smooth_b)) {
        return smooth_a;
    }

    if (std::get_if<std::shared_ptr<SmoothVoidInt>>(&smooth_a)) {
        return smooth_b;
    }

    if (std::get_if<std::shared_ptr<SmoothVoidInt>>(&smooth_b)) {
        return smooth_a;
    }

    if (std::get_if<std::shared_ptr<SmoothVoidFloat>>(&smooth_a)) {
        return smooth_b;
    }

    if (std::get_if<std::shared_ptr<SmoothVoidFloat>>(&smooth_b)) {
        return smooth_a;
    }

    auto p_v_structval_a = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth_a);
    auto p_v_structval_b = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth_b);

    if (p_v_structval_a && p_v_structval_b) {
        return merge_smooth_structval(igc, *p_v_structval_a, *p_v_structval_b);
    }

    auto p_v_enum_a = std::get_if<std::shared_ptr<SmoothEnum>>(&smooth_a);
    auto p_v_enum_b = std::get_if<std::shared_ptr<SmoothEnum>>(&smooth_b);

    if (p_v_enum_a && p_v_enum_b) {
        return merge_smooth_enum(igc, *p_v_enum_a, *p_v_enum_b);
    }

    auto p_v_int_a = std::get_if<std::shared_ptr<SmoothInt>>(&smooth_a);
    auto p_v_int_b = std::get_if<std::shared_ptr<SmoothInt>>(&smooth_b);

    if (p_v_int_a && p_v_int_b) {
        return merge_smooth_int(igc, *p_v_int_a, *p_v_int_b);
    }

    auto p_v_float_a = std::get_if<std::shared_ptr<SmoothFloat>>(&smooth_a);
    auto p_v_float_b = std::get_if<std::shared_ptr<SmoothFloat>>(&smooth_b);

    if (p_v_float_a && p_v_float_b) {
        return merge_smooth_float(igc, *p_v_float_a, *p_v_float_b);
    }

    fprintf(stderr, "Only structvals and enums can be merged (and ints and floats for
    ↪ zero extension purposes only), you must be merging something exotic which wouldn't
    ↪  make sense.\n");
    exit(1);
}
```

compiler/backend/src/merge_smooth.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
```

```
Smooth merge_smooth(IrGenCtx& igc, Smooth smooth_a, Smooth smooth_b);
```

compiler/backend/src/merge_smooth_enum.cpp

```
#include <bit>
#include <cstdio>
#include <cstdlib>
#include "llvm/IR/Constants.h"
#include "llvm/IR/DerivedTypes.h"
#include "merge_smooth_enum.hpp"
#include "merge_type_enum.hpp"
#include "get_underlying_type.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"

std::shared_ptr<SmoothEnum> merge_smooth_enum(
    IrGenCtx& igc,
    std::shared_ptr<SmoothEnum> enum_a,
    std::shared_ptr<SmoothEnum> enum_b
) {
    Type underlying_a = get_underlying_type(enum_a->type);
    Type underlying_b = get_underlying_type(enum_b->type);

    auto p_v_enum_a = std::get_if<std::shared_ptr<TypeEnum>>(&underlying_a);
    auto p_v_enum_b = std::get_if<std::shared_ptr<TypeEnum>>(&underlying_b);

    if (!p_v_enum_a || !p_v_enum_b) {
        fprintf(stderr, "smooth enum should resolve to an underlying enum!\n");
        exit(1);
    }

    std::shared_ptr<TypeEnum> v_enum_merged = merge_type_enum(*p_v_enum_a, *p_v_enum_b);

    llvm::Value* merged_value = nullptr;

    // as merging enums will cause indies to change, we have to recompute here.
    // really I need to find a unified place for producing values properly, but for now
    ↪ this will do.

    if (v_enum_merged->hardsym.has_value()) {
        const std::string& hardsym = v_enum_merged->hardsym.value();
        auto it = std::find(v_enum_merged->syms.begin(), v_enum_merged->syms.end(),
    ↪ hardsym);

        uint32_t enum_idx = (uint32_t) std::distance(v_enum_merged->syms.begin(), it);
        uint32_t bit_width = (uint32_t) std::bit_width(v_enum_merged->syms.size() - 1);
        llvm::Type* int_type = llvm::IntegerType::get(igc.context, bit_width);
        merged_value = llvm::ConstantInt::get(int_type, enum_idx);
    } else if (enum_a->value != nullptr && enum_b->value != nullptr) {
        fprintf(stderr, "Impossible to merge two enums that have different values that
    ↪ are unknown at compile-time. Only types can be merged like this.\n");
        exit(1);
    } else if (enum_a->value != nullptr) {
        merged_value = enum_a->value;
    } else if (enum_b->value != nullptr) {
        merged_value = enum_b->value;
    }

    return std::make_shared<SmoothEnum>(SmoothEnum{
        Type(v_enum_merged),
        merged_value,
    });
}
```

compiler/backend/src/merge_smooth_enum.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"

std::shared_ptr<SmoothEnum> merge_smooth_enum(
    IrGenCtx& igc,
    std::shared_ptr<SmoothEnum> enum_a,
    std::shared_ptr<SmoothEnum> enum_b
);
```

compiler/backend/src/merge_smooth_float.cpp

```
#include "merge_smooth_float.hpp"

#include <cstdio>
#include <cstdlib>
#include "get_underlying_type.hpp"
#include "rotten_float_info.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"

std::shared_ptr<SmoothFloat> merge_smooth_float(
    IrGenCtx& igc,
    std::shared_ptr<SmoothFloat> float_a,
    std::shared_ptr<SmoothFloat> float_b
) {
    Type underlying_a = get_underlying_type(float_a->type);
    Type underlying_b = get_underlying_type(float_b->type);

    auto p_v_rotten_a = std::get_if<std::shared_ptr<TypeRotten>>(&underlying_a);
    auto p_v_rotten_b = std::get_if<std::shared_ptr<TypeRotten>>(&underlying_b);

    if (!p_v_rotten_a || !p_v_rotten_b) {
        fprintf(stderr, "Bizzare situation.\n");
        exit(1);
    }

    auto info_a = rotten_float_info(*p_v_rotten_a);
    auto info_b = rotten_float_info(*p_v_rotten_b);

    if (!info_a || !info_b) {
        fprintf(stderr, "Bizzare situation\n");
        exit(1);
    }

    uint32_t bits_a = info_a->bits;
    uint32_t bits_b = info_b->bits;

    if (float_a->value != nullptr && float_b->value != nullptr) {
        fprintf(stderr, "More than one float encountuered while merging smooths.\n");
        exit(1);
    }

    if (float_a->value == nullptr && float_b->value == nullptr) {
        return bits_a >= bits_b ? float_a : float_b;
    }

    llvm::Value* value = float_a->value != nullptr ? float_a->value : float_b->value;

    return std::make_shared<SmoothFloat>(SmoothFloat{
        bits_a >= bits_b ? float_a->type : float_b->type,
        value,
    });
}
```

compiler/backend/src/merge_smooth_float.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"

std::shared_ptr<SmoothFloat> merge_smooth_float(
    IrGenCtx& igc,
    std::shared_ptr<SmoothFloat> float_a,
    std::shared_ptr<SmoothFloat> float_b
);
```

compiler/backend/src/merge_smooth_int.cpp

```
#include "merge_smooth_int.hpp"

#include <cstdio>
#include <cstdlib>
#include "get_underlying_type.hpp"
#include "rotten_int_info.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"
#include "llvm/IR/DerivedTypes.h"

std::shared_ptr<SmoothInt> merge_smooth_int(
    IrGenCtx& igc,
    std::shared_ptr<SmoothInt> int_a,
    std::shared_ptr<SmoothInt> int_b
) {
    Type underlying_a = get_underlying_type(int_a->type);
    Type underlying_b = get_underlying_type(int_b->type);

    auto p_v_rotten_a = std::get_if<std::shared_ptr<TypeRotten>>(&underlying_a);
    auto p_v_rotten_b = std::get_if<std::shared_ptr<TypeRotten>>(&underlying_b);

    if (!p_v_rotten_a || !p_v_rotten_b) {
        fprintf(stderr, "Invariant glitch.\n");
        exit(1);
    }

    auto info_a = rotten_int_info(*p_v_rotten_a);
    auto info_b = rotten_int_info(*p_v_rotten_b);

    if (!info_a || !info_b) {
        fprintf(stderr, "Another invariant glitch.\n");
        exit(1);
    }

    uint32_t bits_a = info_a->bits;
    uint32_t bits_b = info_b->bits;

    if (int_a->value != nullptr && int_b->value != nullptr) {
        fprintf(stderr, "Failed to merge two smooths because more than one int value.\n
    ↪ ");
        exit(1);
    }

    if (int_a->value == nullptr && int_b->value == nullptr) {
        return bits_a >= bits_b ? int_a : int_b;
    }

    llvm::Value* value = int_a->value != nullptr ? int_a->value : int_b->value;
    llvm::Type* type = llvm::IntegerType::get(igc.context, bits_a >= bits_b ? bits_a :
    ↪ bits_b);
    value = igc.builder.CreateZExt(value, type);
```

```
    return std::make_shared<SmoothInt>(SmoothInt{
        bits_a >= bits_b ? int_a->type : int_b->type,
        value,
    });
}
```

compiler/backend/src/merge_smooth_int.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"

std::shared_ptr<SmoothInt> merge_smooth_int(
    IrGenCtx& igc,
    std::shared_ptr<SmoothInt> int_a,
    std::shared_ptr<SmoothInt> int_b
);
```

compiler/backend/src/merge_smooth_value.cpp

```
#include <cstdio>
#include <cstdlib>
#include <variant>
#include "merge_smooth_value.hpp"
#include "merge_smooth.hpp"
#include "merge_type_map.hpp"
#include "llvm_value.hpp"
#include "t_types.hpp"
#include "t_smooth.hpp"
#include "llvm/IR/DerivedTypes.h"
#include "extract_leaf.hpp"
#include "smooth_type.hpp"
#include "get_underlying_type.hpp"
#include "is_type_singletonish.hpp"
#include "evaluate_singletonish.hpp"
#include "happy_smooth.hpp"

std::shared_ptr<SmoothStructval> merge_smooth_structval(
    IrGenCtx& igc,
    std::shared_ptr<SmoothStructval> structval_a,
    std::shared_ptr<SmoothStructval> structval_b
) {
    auto p_v_map_a = std::get_if<std::shared_ptr<TypeMap>>(&structval_a->type);
    auto p_v_map_b = std::get_if<std::shared_ptr<TypeMap>>(&structval_b->type);

    if (!p_v_map_a || !p_v_map_b) {
        fprintf(stderr, "only makes sense to merge two maps for now, really i don't
    ↪ think there should ever be non-map as a smooth_value.\n");
        exit(1);
    }

    std::shared_ptr<TypeMap> type_merged = merge_type_map(*p_v_map_a, *p_v_map_b);

    llvm::Value* value_merged = structval_b->value;
    std::optional<Smooth> smooth_leaf = std::nullopt;

    if (structval_a->has_leaf && structval_b->has_leaf) {
        if (is_type_singletonish(type_merged->leaf_type.value())) {
            smooth_leaf = evaluate_singletonish(igc, type_merged->leaf_type.value());
        } else {
            smooth_leaf = happy_smooth(
                igc,
```

```
                merge_smooth(igc, extract_leaf(igc, Smooth(structval_a)), extract_leaf(
    ↪ igc, Smooth(structval_b))),
                type_merged->leaf_type.value()
            );
        }

        auto underlying = get_underlying_type(smooth_type(smooth_leaf.value()));
        bool is_leaf_physical = !is_type_singletonish(underlying);

        llvm::StructType* struct_type_orig = llvm::cast<llvm::StructType>(structval_b->
    ↪ value->getType());
        std::vector<llvm::Type*> field_types(struct_type_orig->element_begin(),
    ↪ struct_type_orig->element_end());

        if (is_leaf_physical) {
            field_types[0] = llvm_value(smooth_leaf.value())->getType();
        }

        llvm::StructType* struct_type_new = llvm::StructType::get(igc.context,
    ↪ field_types);
        llvm::Value* replaced_value = llvm::UndefValue::get(struct_type_new);

        if (is_leaf_physical) {
            replaced_value = igc.builder.CreateInsertValue(replaced_value, llvm_value(
    ↪ smooth_leaf.value()), 0);
        }

        // currently only taking sym inputs from the b value, not properly merging this
    ↪ stuff yet.

        for (uint32_t i = is_leaf_physical ? 1 : 0; i < struct_type_orig->getNumElements
    ↪ (); i++) {
            llvm::Value* element = igc.builder.CreateExtractValue(structval_b->value, i)
    ↪ ;
            replaced_value = igc.builder.CreateInsertValue(replaced_value, element, i);
        }

        value_merged = replaced_value;
    }

    return std::make_shared<SmoothStructval>(SmoothStructval{
        type_merged,
        value_merged,
        structval_b->has_leaf,
        smooth_leaf,
        nullptr, // this is problematic.
    });
}
```

compiler/backend/src/merge_smooth_value.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth.hpp"

std::shared_ptr<SmoothStructval> merge_smooth_structval(
    IrGenCtx& igc,
    std::shared_ptr<SmoothStructval> structval_a,
    std::shared_ptr<SmoothStructval> structval_b
);
```

compiler/backend/src/merge_type_enum.cpp

```
#include <algorithm>
#include <bit>
#include <cstdio>
#include <cstdlib>
#include "merge_type_enum.hpp"
#include "t_types.hpp"

std::shared_ptr<TypeEnum> merge_type_enum(std::shared_ptr<TypeEnum> enum_a, std::
    ↪ shared_ptr<TypeEnum> enum_b) {
    auto merged = std::make_shared<TypeEnum>();

    for (const auto& sym : enum_a->syms) {
        merged->syms.push_back(sym);
    }

    for (const auto& sym : enum_b->syms) {
        bool already_present = false;

        for (const auto& existing : merged->syms) {
            if (existing == sym) {
                already_present = true;
                break;
            }
        }

        if (!already_present) {
            merged->syms.push_back(sym);
        }
    }

    if (enum_a->hardsym.has_value() && enum_b->hardsym.has_value()) {
        if (enum_a->hardsym.value() != enum_b->hardsym.value()) {
            fprintf(stderr, "no, cannot combine two enum syms simultaneously %s and %s.\
    ↪ n", enum_a->hardsym.value().c_str(), enum_b->hardsym.value().c_str());
            exit(1);
        }

        merged->hardsym = enum_a->hardsym;
        merged->is_instantiated = true;
    } else if (enum_a->hardsym.has_value()) {
        merged->hardsym = enum_a->hardsym;
        merged->is_instantiated = true;
    } else if (enum_b->hardsym.has_value()) {
        merged->hardsym = enum_b->hardsym;
        merged->is_instantiated = true;
    } else if (enum_a->is_instantiated || enum_b->is_instantiated) {
        merged->is_instantiated = true;
    }

    return merged;
}
```

compiler/backend/src/merge_type_enum.hpp

```
#pragma once

#include <memory>
#include "t_types_fwd.hpp"

std::shared_ptr<TypeEnum> merge_type_enum(std::shared_ptr<TypeEnum> enum_a, std::
    ↪ shared_ptr<TypeEnum> enum_b);
```

compiler/backend/src/merge_type_map.cpp

```
#include <cstdio>
#include <cstdlib>
#include "merge_type_map.hpp"
#include "merge_underlying_type.hpp"
#include "t_types.hpp"

std::shared_ptr<TypeMap> merge_type_map(std::shared_ptr<TypeMap> map_a, std::shared_ptr<
    ↪ TypeMap> map_b) {
    if (!map_a->execution_sequence.empty() || !map_b->execution_sequence.empty()) {
        fprintf(stderr, "Instructoins should not be present, expected to be processed
    ↪ already by this point.\n");
        exit(1);
    }

    if (map_a->call_input_type != nullptr && map_b->call_input_type != nullptr) {
        fprintf(stderr, "Merge not implemented.\n");
        exit(1);
    }

    if (map_a->call_output_type != nullptr && map_b->call_output_type != nullptr) {
        fprintf(stderr, "Merge not implemented.\n");
        exit(1);
    }

    if (!map_a->sym_inputs.empty() && !map_b->sym_inputs.empty()) {
        fprintf(stderr, "Merge not implemented!\n");
        exit(1);
    }

    auto merged = std::make_shared<TypeMap>(*map_b);

    if (map_a->leaf_type.has_value() && map_b->leaf_type.has_value()) {
        // not really sure if this is the best approach but essentially don't want to
    ↪ lose context (whether we need it later or not).
        Type modern_leaf_situation = merge_underlying_type(map_a->leaf_type.value(),
    ↪ map_b->leaf_type.value());
        auto new_merged_wrapper = std::make_shared<TypeMerged>();

        new_merged_wrapper->types.push_back(map_a->leaf_type.value());
        new_merged_wrapper->types.push_back(map_b->leaf_type.value());

        new_merged_wrapper->underlying_type = std::make_shared<Type>(
    ↪ modern_leaf_situation);

        merged->leaf_type = Type(new_merged_wrapper);
    } else if (map_a->leaf_type.has_value()) {
        merged->leaf_type = map_a->leaf_type;
    }

    if (map_a->call_input_type != nullptr) {
        merged->call_input_type = map_a->call_input_type;
    }

    if (map_a->call_output_type != nullptr) {
        merged->call_output_type = map_a->call_output_type;
    }

    if (!map_a->sym_inputs.empty()) {
        merged->sym_inputs = map_a->sym_inputs;
    }

    // todo: consider sym_inputs

    return merged;
}
```

compiler/backend/src/merge_type_map.hpp

```
#pragma once

#include <memory>
#include "t_types_fwd.hpp"

std::shared_ptr<TypeMap> merge_type_map(std::shared_ptr<TypeMap> map_a, std::shared_ptr<
    ↪ TypeMap> map_b);
```

compiler/backend/src/merge_underlying_type.cpp

```
#include <cstdio>
#include <cstdlib>
#include "merge_underlying_type.hpp"
#include "merge_type_map.hpp"
#include "merge_type_enum.hpp"
#include "get_underlying_type.hpp"
#include "is_eq_type_rotten.hpp"
#include "is_eq_type_pointer.hpp"
#include "rotten_float_info.hpp"
#include "rotten_int_info.hpp"
#include "t_types.hpp"

Type merge_underlying_type(Type type_a, Type type_b) {
    Type underlying_a = get_underlying_type(type_a);
    Type underlying_b = get_underlying_type(type_b);

    auto p_map_a = std::get_if<std::shared_ptr<TypeMap>>(&underlying_a);
    auto p_map_b = std::get_if<std::shared_ptr<TypeMap>>(&underlying_b);

    if (p_map_a && p_map_b) {
        return merge_type_map(*p_map_a, *p_map_b);
    }

    auto p_enum_a = std::get_if<std::shared_ptr<TypeEnum>>(&underlying_a);
    auto p_enum_b = std::get_if<std::shared_ptr<TypeEnum>>(&underlying_b);

    if (p_enum_a && p_enum_b) {
        return merge_type_enum(*p_enum_a, *p_enum_b);
    }

    auto p_rotten_a = std::get_if<std::shared_ptr<TypeRotten>>(&underlying_a);
    auto p_rotten_b = std::get_if<std::shared_ptr<TypeRotten>>(&underlying_b);

    if (p_rotten_a && p_rotten_b) {
        const TypeRotten& rotten_a = **p_rotten_a;
        const TypeRotten& rotten_b = **p_rotten_b;

        if (!is_eq_type_rotten(rotten_a, rotten_b)) {
            auto float_a = rotten_float_info(*p_rotten_a);
            auto float_b = rotten_float_info(*p_rotten_b);

            if (float_a && float_b) {
                return float_a->bits >= float_b->bits ? underlying_a : underlying_b;
            }

            auto int_a = rotten_int_info(*p_rotten_a);
            auto int_b = rotten_int_info(*p_rotten_b);

            if (int_a && int_b && int_a->prefix == int_b->prefix) {
                return int_a->bits >= int_b->bits ? underlying_a : underlying_b;
            }

            fprintf(stderr, "Rotten conflict could not be dealth with: %s -vs- %s\n",
    ↪ rotten_a.type_str.c_str(), rotten_b.type_str.c_str());
            exit(1);
        }
```

```
        return underlying_a;
    }

    auto p_pointer_a = std::get_if<std::shared_ptr<TypePointer>>(&underlying_a);
    auto p_pointer_b = std::get_if<std::shared_ptr<TypePointer>>(&underlying_b);

    if (p_pointer_a && p_pointer_b) {
        const TypePointer& pointer_a = **p_pointer_a;
        const TypePointer& pointer_b = **p_pointer_b;

        if (!is_eq_type_pointer(pointer_a, pointer_b)) {
            fprintf(stderr, "Pointer conflict, two pointers are not equivalent.\n");
            exit(1);
        }

        return underlying_a;
    }

    fprintf(stderr, "these types can not be merged statically.\n");
    exit(1);
}
```

compiler/backend/src/merge_underlying_type.hpp

```
#pragma once

#include "t_types_fwd.hpp"

Type merge_underlying_type(Type type_a, Type type_b);
```

compiler/backend/src/normalize_type.cpp

```
#include <algorithm>
#include <cstdio>
#include <cstdlib>
#include <optional>
#include <string>
#include <memory>
#include "normalize_type.hpp"
#include "demote_underlying.hpp"
#include "merge_underlying_type.hpp"
#include "get_underlying_type.hpp"
#include "t_types.hpp"
#include "t_instructions.hpp"
#include "t_hardval.hpp"
#include "t_bundles.hpp"
#include "preinstantiated_types.hpp"
#include "llvm/ADT/APInt.h"
#include "llvm/ADT/StringRef.h"

using json = nlohmann::json;

Type normalize_type(
    TypeOrchCtx& toc,
    const json& typeval
) {
    if (typeval.is_null()) {
        fprintf(stderr, "Got null for .type\n");
        exit(1);
    }

    if (!typeval.contains("type")) {
        fprintf(stderr, "Where is the .type field\n");
        exit(1);
```

```
}

std::string type = typeval["type"];

if (type == "type_named") {
    if (!typeval.contains("trail")) {
        fprintf(stderr, "Expected .trail");
        exit(1);
    }

    std::string trail = typeval["trail"];
    std::optional<UnderlyingType> found = toc.type_table.get(trail);

    if (!found.has_value()) {
        fprintf(stderr, "No type pre-given for this %s\n", trail.c_str());
        exit(1);
    }

    return demote_underlying(found.value());
}

if (type == "type_ptr") {
    if (!typeval.contains("target")) {
        fprintf(stderr, "Expected to see .target\n");
        exit(1);
    }

    Type target_type = normalize_type(toc, typeval["target"]);

    return std::make_shared<TypePointer>(TypePointer{
        std::make_shared<Type>(target_type),
        std::nullopt,
    });
}

if (type == "type_map") {
    TypeMap result;
    result.leaf_type = std::nullopt;
    result.leaf_hardval = std::nullopt;

    if (typeval.contains("leaf_type") && !typeval["leaf_type"].is_null()) {
        const auto& leaf_type_data = typeval["leaf_type"];

        if (!leaf_type_data.contains("type")) {
            fprintf(stderr, "Expected a .type\n");
            exit(1);
        }

        std::string leaf_type_type = leaf_type_data["type"];

        if (leaf_type_type == "type_named") {
            if (!leaf_type_data.contains("trail")) {
                fprintf(stderr, "Expected some .trail\n");
                exit(1);
            }

            auto v_rotten = std::make_shared<TypeRotten>();
            v_rotten->type_str = leaf_type_data["trail"];
            result.leaf_type = Type(v_rotten);
        } else {
            fprintf(stderr, "Type given was not handled, got %s\n", leaf_type_type.
↪ c_str());
            exit(1);
        }
    }

    if (typeval.contains("leaf_hardval") && !typeval["leaf_hardval"].is_null()) {
        const auto& hardval_data = typeval["leaf_hardval"];

        if (!hardval_data.contains("type")) {
```

```
            fprintf(stderr, "Expected to get some .type\n");
            exit(1);
        }

        std::string hardval_type = hardval_data["type"];

        if (hardval_type == "hardval_integer") {
            if (!hardval_data.contains("value")) {
                fprintf(stderr, "Expected .value\n");
                exit(1);
            }

            if (!result.leaf_type.has_value()) { // this is duplicating logic, but
↪ it's too difficult to get the code working without forcing the leaf_type early on.
                const std::string& value_str = hardval_data["value"].get_ref<const
↪ std::string&>();
                bool is_negative = !value_str.empty() && value_str[0] == '-';
                std::string digits = is_negative ? value_str.substr(1) : value_str;

                uint32_t scratchy_bits = (uint32_t)(digits.size() * 4) + 1;
                llvm::APInt ap_value(scratchy_bits, digits.c_str(), 10);

                int bits_needed = 0;

                if (is_negative) {
                    bits_needed = ap_value.getActiveBits() + 1; // i'm not entirely
↪ sure if this is correct.
                } else {
                    bits_needed = ap_value.getActiveBits();

                    if (bits_needed == 0) {
                        bits_needed = 1;
                    }
                }

                auto v_rotten = std::make_shared<TypeRotten>();
                v_rotten->type_str = (is_negative ? "i" : "u") + std::to_string(
↪ bits_needed);
                result.leaf_type = Type(v_rotten);
            }

            auto v_int = std::make_shared<HardvalInteger>();
            v_int->value = hardval_data["value"].get<std::string>();
            result.leaf_hardval = Hardval(v_int);
        } else if (hardval_type == "hardval_float") {
            if (!hardval_data.contains("value")) {
                fprintf(stderr, "Expected .value\n");
                exit(1);
            }

            if (!result.leaf_type.has_value()) {
                auto v_rotten = std::make_shared<TypeRotten>();
                v_rotten->type_str = "f64"; // I don't have time to sort this,
↪ assuming all rottens are 64-bit floats which is fine for now.
                result.leaf_type = Type(v_rotten);
            }

            auto v_float = std::make_shared<HardvalFloat>();
            v_float->value = hardval_data["value"].get<std::string>();
            result.leaf_hardval = Hardval(v_float);
        } else if (hardval_type == "hardval_string") {
            if (!hardval_data.contains("value")) {
                fprintf(stderr, "Expected .value\n");
                exit(1);
            }

            if (!result.leaf_type.has_value()) {
                auto v_rotten = std::make_shared<TypeRotten>();
                v_rotten->type_str = "u8";
```

```
                auto v_pointer = std::make_shared<TypePointer>();
                v_pointer->target = std::make_shared<Type>(Type(v_rotten));
                v_pointer->hardval = std::nullopt;

                result.leaf_type = Type(v_pointer);
            }

            auto v_string = std::make_shared<HardvalString>();
            v_string->value = hardval_data["value"].get<std::string>();
            result.leaf_hardval = Hardval(v_string);
        } else {
            fprintf(stderr, "Unhandled situation %s\n", hardval_type.c_str());
            exit(1);
        }
    }

    result.call_input_identifier = (false
        || !typeval.contains("call_input_identifier")
        || typeval["call_input_identifier"].is_null()
    ) ? std::nullopt : std::optional<std::string>(typeval["call_input_identifier"].
↪ get<std::string>());

    result.call_input_type = (false
        || !typeval.contains("call_input_type")
        || typeval["call_input_type"].is_null()
    ) ? nullptr : [&]() {
        Type t = normalize_type(toc, typeval["call_input_type"]);
        auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&t);

        if (!p_v_map) {
            fprintf(stderr, "the input of a callable map must itself be a map\n");
            exit(1);
        }

        return std::make_unique<TypeMap>(**p_v_map);
    }();

    result.call_output_type = (false
        || !typeval.contains("call_output_type")
        || typeval["call_output_type"].is_null()
    ) ? nullptr : [&]() {
        Type t = normalize_type(toc, typeval["call_output_type"]);
        auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&t);

        if (!p_v_map) {
            fprintf(stderr, "the output of a callable map must itself be a map\n");
            exit(1);
        }

        return std::make_unique<TypeMap>(**p_v_map);
    }();

    if (true
        && result.call_input_type != nullptr
        && result.call_output_type != nullptr
    ) {
        result.bundle_id = toc.bundle_registry.install(
            std::make_shared<BundleMap>(BundleMap{ nullptr }
        ));
    }

    if (typeval.contains("sym_inputs")) {
        if (!typeval["sym_inputs"].is_object()) {
            fprintf(stderr, "Expected .sym_inputs as object\n");
            exit(1);
        }

        // i think populating this later is ok.
        // for (auto& [key, value] : typeval["sym_inputs"].items()) { // don't
↪ really care about this for now
```

```
        //  result.sym_inputs[key] = std::make_shared<Type>(normalize_type(value,
↪ symbol_table));
        // }
    }
    if (typeval.contains("instructions")) {
        if (!typeval["instructions"].is_array()) {
            fprintf(stderr, "Expected .instructions as array\n");
            exit(1);
        }
        for (const auto& instruction_data : typeval["instructions"]) {
            if (!instruction_data.contains("type")) {
                fprintf(stderr, "expected .type");
                exit(1);
            }

            std::string instruction_type = instruction_data["type"];

            if (instruction_type == "map_entry_expr") {
                auto v_expr = std::make_shared<InstructionExpr>();

                if (!instruction_data.contains("expr")) {
                    fprintf(stderr, "Expected an actual .expr\n");
                    exit(1);
                }

                Type actually_normalized = normalize_type(toc, instruction_data["
↪ expr"]);
                v_expr->expr = std::make_shared<Type>(actually_normalized);

                result.execution_sequence.push_back(v_expr);
                continue;
            }

            if (instruction_type == "map_entry_sym") {
                auto v_sym = std::make_shared<InstructionSym>();

                if (!instruction_data.contains("name")) {
                    fprintf(stderr, "Expected .name\n");
                    exit(1);
                }

                v_sym->name = instruction_data["name"].get<std::string>();

                if (!instruction_data.contains("typeval")) {
                    fprintf(stderr, "Expected .typeval\n");
                    exit(1);
                }

                Type normalized_type = normalize_type(toc, instruction_data["typeval
↪ "]);
                v_sym->typeval = std::make_shared<Type>(normalized_type);

                result.sym_inputs[v_sym->name] = v_sym->typeval;

                result.execution_sequence.push_back(v_sym);
                continue;
            }

            if (instruction_type == "instruction_for") {
                auto v_for = std::make_shared<InstructionFor>();

                if (!instruction_data.contains("body")) {
                    fprintf(stderr, "Expecting .body\n");
                    exit(1);
                }

                Type body = normalize_type(toc, instruction_data["body"]);
                auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&body);

                if (!p_v_map) {
```

```
                    fprintf(stderr, "Somehow parser allowed non-map body for my loop
↪ .\n");
                    exit(1);
                }

                v_for->body = *p_v_map;
                result.execution_sequence.push_back(v_for);

                continue;
            }

            if (instruction_type == "instruction_if") {
                auto v_if = std::make_shared<InstructionIf>();

                if (!instruction_data.contains("branches") || !instruction_data["
↪ branches"].is_array()) {
                    fprintf(stderr, ".branches missing.\n");
                    exit(1);
                }

                for (const auto& branch : instruction_data["branches"]) {
                    InstructionIf_Branch actual_branch;

                    if (!branch.contains("body")) {
                        fprintf(stderr, "Branch lacks a .body.\n");
                        exit(1);
                    }

                    if (branch.contains("condition") && !branch["condition"].is_null
↪ ()) {
                        Type condition = normalize_type(toc, branch["condition"]);
                        actual_branch.condition = std::make_shared<Type>(condition);
                    }

                    Type body = normalize_type(toc, branch["body"]);
                    auto p_v_map = std::get_if<std::shared_ptr<TypeMap>>(&body);

                    if (!p_v_map) {
                        fprintf(stderr, "frontend did not produce a map here.\n");
                        exit(1);
                    }

                    actual_branch.body = *p_v_map;
                    v_if->branches.push_back(actual_branch);
                }

                result.execution_sequence.push_back(v_if);

                continue;
            }

            if (instruction_type == "instruction_break") {
                auto v_break = std::make_shared<InstructionBreak>();

                result.execution_sequence.push_back(v_break);

                continue;
            }

            fprintf(stderr, "Not recognized instructoin type: %s\n",
↪ instruction_type.c_str());
            exit(1);
        }
    }

    return std::make_shared<TypeMap>(result);
}

if (type == "type_var_access") {
    if (!typeval.contains("target_name")) {
```

```
            fprintf(stderr, "Expected .target_name\n");
            exit(1);
        }

        std::string target_name = typeval["target_name"];
        auto v_var_access = std::make_shared<TypeVarAccess>();

        v_var_access->target_name = target_name;

        return v_var_access;
    }

    if (type == "expr_var_walrus") {
        auto v_var_walrus = std::make_shared<TypeVarWalrus>();

        if (!typeval.contains("name")) {
            fprintf(stderr, "expected .name\n");
            exit(1);
        }

        v_var_walrus->name = typeval["name"].get<std::string>();
        v_var_walrus->is_mut = typeval.value("is_mut", false);

        if (!typeval.contains("typeval")) {
            fprintf(stderr, "expected .typeval\n");
            exit(1);
        }

        Type delicious_type = normalize_type(toc, typeval["typeval"]);
        v_var_walrus->typeval = std::make_shared<Type>(delicious_type);

        return v_var_walrus;
    }

    if (type == "expr_var_assign") {
        auto v_var_assign = std::make_shared<TypeVarAssign>();

        if (!typeval.contains("name")) {
            fprintf(stderr, "expected .name\n");
            exit(1);
        }

        v_var_assign->name = typeval["name"].get<std::string>();

        if (!typeval.contains("typeval")) {
            fprintf(stderr, "expectred .typevla\n");
            exit(1);
        }

        Type delicious_type = normalize_type(toc, typeval["typeval"]);
        v_var_assign->typeval = std::make_shared<Type>(delicious_type);

        return v_var_assign;
    }

    if (type == "expr_log") {
        auto v_log = std::make_shared<TypeLog>();

        v_log->message = (false
            || !typeval.contains("message")
            || typeval["message"].is_null()
        ) ? nullptr : std::make_shared<Type>(normalize_type(toc, typeval["message"]));

        return v_log;
    }

    if (type == "expr_log_d") {
        auto v_log_d = std::make_shared<TypeLogD>();

        if (!typeval.contains("message") || typeval["message"].is_null()) {
```

```
        fprintf(stderr, "you have to actually log something when using log_d, not
↪ provide no arguments\n");
        exit(1);
    }

    v_log_d->message = std::make_shared<Type>(normalize_type(toc, typeval["message
↪ "]));

    return v_log_d;
}

if (type == "expr_log_dd") {
    auto v_log_dd = std::make_shared<TypeLogDd>();

    if (!typeval.contains("message") || typeval["message"].is_null()) {
        fprintf(stderr, "have to provide a message to log_dd\n");
        exit(1);
    }

    if (!typeval.contains("byte_count") || typeval["byte_count"].is_null()) {
        fprintf(stderr, "have to provide a byte_count to log_dd or specify null-term
↪ \n");
        exit(1);
    }

    v_log_dd->message = std::make_shared<Type>(normalize_type(toc, typeval["message
↪ "]));

    const auto& byte_count = typeval["byte_count"];
    const std::string byte_count_type = byte_count["type"].get<std::string>();

    if (byte_count_type == "nullterm") {
        v_log_dd->is_nullterm = true;
        v_log_dd->byte_count = nullptr;
    } else if (byte_count_type == "byte_count") {
        v_log_dd->is_nullterm = false;
        v_log_dd->byte_count = std::make_shared<Type>(normalize_type(toc, byte_count
↪ ["count"]));
    } else {
        fprintf(stderr, "bizarre type for byte_count %s\n", byte_count_type.c_str()
↪ ;
        exit(1);
    }

    return v_log_dd;
}

if (type == "type_enum") {
    if (!typeval.contains("hardsym") && !typeval.contains("syms")) {
        fprintf(stderr, "A definitive enum instantiation requires an actual symbol
↪ or a list of acceptable possibilities.\n");
        exit(1);
    }

    auto v_enum = std::make_shared<TypeEnum>();

    if (typeval.contains("syms") && typeval["syms"].is_array()) {
        for (const auto& sym_json : typeval["syms"]) {
            v_enum->syms.push_back(sym_json.get<std::string>());
        }
    }

    if (typeval.contains("hardsym")) {
        std::string hardsym = typeval["hardsym"].get<std::string>();
        v_enum->hardsym = hardsym;
        v_enum->is_instantiated = true;
        v_enum->syms.push_back(hardsym);
    }

    return v_enum;
```

```
}

if (type == "expr_call_with_sym") {
    if (!typeval.contains("target")) {
        fprintf(stderr, "Where is this .target gone!\n");
        exit(1);
    }

    if (!typeval.contains("sym")) {
        fprintf(stderr, "Where is the .sym gone!\n");
        exit(1);
    }

    Type target = normalize_type(toc, typeval["target"]);
    std::string sym = typeval["sym"].get<std::string>();

    if (auto p_v_enum = std::get_if<std::shared_ptr<TypeEnum>>(&target)) {
        if (!(*p_v_enum)->hardsym.has_value()) {
            // the syntax is not actually a call in this case, but rather an
↪ instantiation.

            bool was_found = std::find((*p_v_enum)->syms.begin(), (*p_v_enum)->syms.
↪ end(), sym) != (*p_v_enum)->syms.end();

            if (!was_found) {
                fprintf(stderr, "Cannot instantiate %s as it is not one of the
↪ predefined possibilitirets.\n", sym.c_str());
                exit(1);
            }

            auto v_enum_with_hardsym = std::make_shared<TypeEnum>();
            v_enum_with_hardsym->hardsym = sym;
            v_enum_with_hardsym->is_instantiated = true;
            v_enum_with_hardsym->syms.push_back(sym);

            auto v_merged = std::make_shared<TypeMerged>();
            v_merged->types.push_back(target);
            v_merged->types.push_back(v_enum_with_hardsym);
            v_merged->underlying_type = std::make_shared<Type>(merge_underlying_type
↪ (target, Type(v_enum_with_hardsym)));

            return v_merged;
        }
    }

    auto v_call_with_sym = std::make_shared<TypeCallWithSym>();
    v_call_with_sym->target = std::make_shared<Type>(target);
    v_call_with_sym->sym = sym;

    return v_call_with_sym;
}

if (type == "expr_call_with_dynamic") {
    if (!typeval.contains("target")) {
        fprintf(stderr, "No .target was provoided.\n");
        exit(1);
    }

    if (!typeval.contains("call_data")) {
        fprintf(stderr, "No .call_data was provided.\n");
        exit(1);
    }

    Type target = normalize_type(toc, typeval["target"]);
    Type call_data = normalize_type(toc, typeval["call_data"]);

    auto v_call_with_dynamic = std::make_shared<TypeCallWithDynamic>();
    v_call_with_dynamic->target = std::make_shared<Type>(target);
    v_call_with_dynamic->call_data = std::make_shared<Type>(call_data);
```

```
    return v_call_with_dynamic;
}

if (type == "expr_multi") {
    auto v_expr_multi = std::make_shared<TypeExprMulti>();

    for (const auto& op_data : typeval["ops"]) {
        OpNumeric op;
        op.op = op_data["op"].get<std::string>();
        op.operand = std::make_shared<Type>(normalize_type(toc, op_data["operand"]))
↪ ;
        v_expr_multi->ops.push_back(op);
    }

    if (v_expr_multi->ops.empty()) {
        fprintf(stderr, "this is quite bizrree.\n");
        exit(1);
    }

    return v_expr_multi;
}

if (type == "expr_addit") {
    auto v_expr_addit = std::make_shared<TypeExprAddit>();

    for (const auto& op_data : typeval["ops"]) {
        OpNumeric op;
        op.op = op_data["op"].get<std::string>();
        op.operand = std::make_shared<Type>(normalize_type(toc, op_data["operand"]))
↪ ;
        v_expr_addit->ops.push_back(op);
    }

    if (v_expr_addit->ops.empty()) {
        fprintf(stderr, "this is not good outcome.\n");
        exit(1);
    }

    return v_expr_addit;
}

if (type == "expr_logical_and") {
    auto v_expr_logical_and = std::make_shared<TypeExprLogicalAnd>();

    for (const auto& op_data : typeval["ops"]) {
        OpLogical op;
        op.operand = std::make_shared<Type>(normalize_type(toc, op_data["operand"]))
↪ ;
        v_expr_logical_and->ops.push_back(op);
    }

    v_expr_logical_and->underlying_type = std::make_shared<Type>(Type(type_bool));

    return v_expr_logical_and;
}

if (type == "expr_logical_or") {
    auto v_expr_logical_or = std::make_shared<TypeExprLogicalOr>();

    for (const auto& op_data : typeval["ops"]) {
        OpLogical op;
        op.operand = std::make_shared<Type>(normalize_type(toc, op_data["operand"]))
↪ ;
        v_expr_logical_or->ops.push_back(op);
    }

    v_expr_logical_or->underlying_type = std::make_shared<Type>(Type(type_bool));

    return v_expr_logical_or;
}
```

```
if (type == "expr_compare") {
    auto v_compare = std::make_shared<TypeCompare>();

    v_compare->operator_ = typeval["operator"].get<std::string>();
    v_compare->operand_a = std::make_shared<Type>(normalize_type(toc, typeval["
↪ operand_a"]));
    v_compare->operand_b = std::make_shared<Type>(normalize_type(toc, typeval["
↪ operand_b"]));
    v_compare->underlying_type = std::make_shared<Type>(Type(type_bool));

    return v_compare;
}

if (type == "type_constrained") {
    if (false
        || !typeval.contains("constraints")
        || !typeval["constraints"].is_array()
    ) {
        fprintf(stderr, "bad type_constrained, expected .constraints\n");
        exit(1);
    }

    const auto& constraints = typeval["constraints"];

    if (constraints.size() < 2) {
        fprintf(stderr, "useless constraints, only one or less given\n");
        exit(1);
    }

    if (constraints.size() > 2) {
        fprintf(stderr, "useful constraints, but more than 2 not handled yet\n");
        exit(1);
    }

    Type constraint_1 = normalize_type(toc, constraints[0]);
    Type constraint_2 = normalize_type(toc, constraints[1]);

    if (auto p_v_1 = std::get_if<std::shared_ptr<TypePointer>>(&constraint_1)) {
        auto p_v_2 = std::get_if<std::shared_ptr<TypeMap>>(&constraint_2);

        if (!p_v_2) {
            fprintf(stderr, "In case that first constraint is pointer, for now only
↪ second constraint as map is supported.\n");
            exit(1);
        }

        TypeMap& v_2 = **p_v_2;

        if (!v_2.leaf_hardval.has_value()) {
            fprintf(stderr, "For now constrained pointers must have definitive value
↪  associated with them\n");
            exit(1);
        }

        auto p_pointer = std::make_shared<TypePointer>();

        p_pointer->target = (*p_v_1)->target;
        p_pointer->hardval = v_2.leaf_hardval;

        return p_pointer;
    }

    auto p_v_1 = std::get_if<std::shared_ptr<TypeMap>>(&constraint_1);
    auto p_v_2 = std::get_if<std::shared_ptr<TypeMap>>(&constraint_2);

    if (!p_v_1 || !p_v_2) {
        // fprintf(stderr, "In case that first constraint is not a pointer, expected
↪  both constrained types to be maps, for now.\n");
        // exit(1);
```

```
        }

        if (p_v_1 && p_v_2) {
            auto v_merged = std::make_shared<TypeMerged>();

            v_merged->types.push_back(constraint_1);
            v_merged->types.push_back(constraint_2);
            v_merged->underlying_type = std::make_shared<Type>(merge_underlying_type(
    ↪ constraint_1, constraint_2));

            return v_merged;
        }

        auto v_merged = std::make_shared<TypeMerged>();

        v_merged->types.push_back(constraint_1);
        v_merged->types.push_back(constraint_2);
        v_merged->underlying_type = std::make_shared<Type>(merge_underlying_type(
    ↪ constraint_1, constraint_2));

        return v_merged;
    }

    fprintf(stderr, "unhandled type, got %s\n", type.c_str());
    exit(1);
}
```

compiler/backend/src/normalize_type.hpp

```
#pragma once

#include "dependencies/json.hpp"
#include "t_ctx.hpp"
#include "t_types.hpp"

using json = nlohmann::json;

// This function both parses and normalizes the JSON into a nice programmatic structure.

Type normalize_type(
    TypeOrchCtx& toc,
    const json& typeval
);
```

compiler/backend/src/preinstantiated_smooths.cpp

```
#include "preinstantiated_smooths.hpp"

#include "llvm/IR/Constants.h"
#include "llvm/IR/DerivedTypes.h"
#include "t_smooth.hpp"

std::shared_ptr<SmoothVoid> smooth_void(IrGenCtx& igc, std::optional<Type> type) {
    llvm::StructType* struct_type = llvm::StructType::get(igc.context, {});
    llvm::Value* value = llvm::UndefValue::get(struct_type);

    return std::make_shared<SmoothVoid>(SmoothVoid{
        type,
        value,
    });
}
```

compiler/backend/src/preinstantiated_smooths.hpp

```
#pragma once

#include <memory>
#include <optional>
#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_types_fwd.hpp"

std::shared_ptr<SmoothVoid> smooth_void(IrGenCtx& igc, std::optional<Type> type = std::
    ↪ nullopt);
```

compiler/backend/src/preinstantiated_types.cpp

```
#include "preinstantiated_types.hpp"

const std::shared_ptr<TypeEnum> type_bool = [] {
    auto v_enum = std::make_shared<TypeEnum>();
    v_enum->syms = {"false", "true"};
    v_enum->is_instantiated = true;
    return v_enum;
}();

const std::shared_ptr<TypeEnum> type_false = [] {
    auto v_enum = std::make_shared<TypeEnum>();
    v_enum->syms = {"false", "true"};
    v_enum->hardsym = "false";
    return v_enum;
}();

const std::shared_ptr<TypeEnum> type_true = [] {
    auto v_enum = std::make_shared<TypeEnum>();
    v_enum->syms = {"false", "true"};
    v_enum->hardsym = "true";
    return v_enum;
}();

const std::shared_ptr<TypeVoid> type_void = std::make_shared<TypeVoid>();
```

compiler/backend/src/preinstantiated_types.hpp

```
#pragma once

#include <memory>
#include "t_types.hpp"

extern const std::shared_ptr<TypeEnum> type_bool;
extern const std::shared_ptr<TypeEnum> type_false;
extern const std::shared_ptr<TypeEnum> type_true;
extern const std::shared_ptr<TypeVoid> type_void;
```

compiler/backend/src/process_map_body.cpp

```
#include <cstdio>
#include <cstdlib>
#include <variant>
#include <memory>
#include "llvm/IR/BasicBlock.h"
#include "process_map_body.hpp"
#include "evaluate_smooth.hpp"
```

```
#include "t_instructions.hpp"
#include "t_smooth.hpp"
#include "llvm_value.hpp"
#include "llvm_flexi_type.hpp"
#include "smooth_type.hpp"
#include "create_value_symbol_table.hpp"
#include "is_type_singletonish.hpp"

void process_map_body(
    IrGenCtx& igc,
    const TypeMap& body
) {
    for (const auto& instruction : body.execution_sequence) {
        if (auto p_v_expr = std::get_if<std::shared_ptr<InstructionExpr>>(&instruction))
    ↪   {
            const auto& v_expr = *p_v_expr;
            evaluate_smooth(igc, *v_expr->expr);
        }

        if (auto p_v_sym = std::get_if<std::shared_ptr<InstructionSym>>(&instruction)) {
            const auto& v_sym = *p_v_sym;

            std::string map_sym_var_name = "ms_" + v_sym->name;

            // we need to evaluate early here to ensure underlying_types are populated.
            // this may produce dead-code if the singletonish optimization is applied
    ↪ later, but llvm should eliminate this dead-code for us.
            Smooth smooth = evaluate_smooth(igc, *v_sym->typeval);
            Type discovered_type = smooth_type(smooth);

            ValueSymbolTableEntry entry = [&]() {
                if (is_type_singletonish(discovered_type)) {
                    return ValueSymbolTableEntry{
                        nullptr,
                        nullptr,
                        discovered_type,
                        false, // syms, when treated as variables, are always immutable.
                    };
                }

                std::string scoped_alloca_name = igc.value_table->scope_id() + "~" +
    ↪ map_sym_var_name;
                llvm::Value* value = llvm_value(smooth);
                llvm::Value* alloca = igc.builder.CreateAlloca(value->getType(), nullptr
    ↪ , scoped_alloca_name);
                igc.builder.CreateStore(value, alloca);

                return ValueSymbolTableEntry{
                    alloca,
                    value->getType(),
                    discovered_type, // maybe in the future we'll change how this works
                    false, // syms, when treated as variables, are always immutable.
                };
            }();

            igc.value_table->set(map_sym_var_name, entry);
        }

        if (auto p_v_for = std::get_if<std::shared_ptr<InstructionFor>>(&instruction)) {
            const auto& v_for = *p_v_for;

            llvm::Function* current_func = igc.builder.GetInsertBlock()->getParent();
            llvm::BasicBlock* block_start = llvm::BasicBlock::Create(igc.context, "
    ↪ for_START", current_func);
            llvm::BasicBlock* block_end = llvm::BasicBlock::Create(igc.context, "for_END
    ↪ ", current_func);

            igc.builder.CreateBr(block_start);
            igc.builder.SetInsertPoint(block_start);
```

```
        std::shared_ptr<ValueSymbolTable> new_value_table = std::make_shared<
↪ ValueSymbolTable>(
            create_value_symbol_table(igc.value_table.get())
        );

        IrGenCtx new_igc = igc;
        new_igc.value_table = new_value_table;
        new_igc.block_break = block_end;

        process_map_body(new_igc, *v_for->body);

        new_igc.builder.CreateBr(block_start);
        igc.builder.SetInsertPoint(block_end);
    }

    if (auto p_v_if = std::get_if<std::shared_ptr<InstructionIf>>(&instruction)) {
        const auto& v_if = *p_v_if;

        llvm::Function* current_func = igc.builder.GetInsertBlock()->getParent();
        llvm::BasicBlock* block_end = llvm::BasicBlock::Create(igc.context, "if_END
↪ ", current_func);

        bool fallthrough_done = false;

        for (size_t i = 0; i < v_if->branches.size(); i++) {
            const auto& branch = v_if->branches[i];

            llvm::BasicBlock* block_then = llvm::BasicBlock::Create(igc.context, "
↪ if_BRANCH", current_func);

            if (branch.condition != nullptr) { // exact condition given (each
↪ condition is a fallthrough so this may be an "if" or an "else if")
                Smooth condition_smooth = evaluate_smooth(igc, *branch.condition);

                if (!std::get_if<std::shared_ptr<SmoothEnum>>(&condition_smooth)) {
                    fprintf(stderr, "the if statement must take in an enum, please
↪ convert your data to a bool!\n");
                    exit(1);
                }

                llvm::Value* condition_value = llvm_value(condition_smooth);

                if (!condition_value->getType()->isIntegerTy(1)) {
                    fprintf(stderr, "\"if\" must take an enum of 1 bit of size (a
↪ bool), not some other enum.\n");
                    exit(1);
                }

                llvm::BasicBlock* block_next = llvm::BasicBlock::Create(igc.context,
↪  "if_PROGRESS", current_func);
                igc.builder.CreateCondBr(condition_value, block_then, block_next);
                igc.builder.SetInsertPoint(block_then);

                std::shared_ptr<ValueSymbolTable> inner_value_table = std::
↪ make_shared<ValueSymbolTable>(
                    create_value_symbol_table(igc.value_table.get())
                );

                IrGenCtx fresh_igc = igc;
                fresh_igc.value_table = inner_value_table;

                process_map_body(fresh_igc, *branch.body);

                fresh_igc.builder.CreateBr(block_end);
                igc.builder.SetInsertPoint(block_next);
            } else { // no condition means fallthrough, effectively means "else" or
↪ "if (true)" in the context of "if" statements.
                igc.builder.CreateBr(block_then);
                igc.builder.SetInsertPoint(block_then);
```

```
                    std::shared_ptr<ValueSymbolTable> brand_new_value_table = std::
    ↪ make_shared<ValueSymbolTable>(
                        create_value_symbol_table(igc.value_table.get())
                    );

                    IrGenCtx inner_igc = igc;
                    inner_igc.value_table = brand_new_value_table;

                    process_map_body(inner_igc, *branch.body);

                    inner_igc.builder.CreateBr(block_end);

                    fallthrough_done = true;

                    break;
                }
            }

            if (!fallthrough_done) {
                igc.builder.CreateBr(block_end);
            }

            igc.builder.SetInsertPoint(block_end);
        }

        if (auto p_v_break = std::get_if<std::shared_ptr<InstructionBreak>>(&instruction
    ↪ )) {
            if (igc.block_break == nullptr) {
                fprintf(stderr, "You are only permitted to use break when a break target
    ↪  is known (i.e. inside a for loop)\n");
                exit(1);
            }

            igc.builder.CreateBr(igc.block_break);

            llvm::Function* current_func = igc.builder.GetInsertBlock()->getParent();
            llvm::BasicBlock* deadblock = llvm::BasicBlock::Create(igc.context, "
    ↪ __DeadBlock__", current_func); // the idea behind this is to have a block in case
    ↪ the user writes some code after the break. but the code would be dead code and
    ↪ eliminator by the optimizer.

            igc.builder.SetInsertPoint(deadblock);

            return;
        }
    }
}
```

compiler/backend/src/process_map_body.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_types.hpp"

void process_map_body(
    IrGenCtx& igc,
    const TypeMap& body
);
```

compiler/backend/src/produce_call_func.cpp

```
#include <cstdio>
#include <cstdlib>
```

```
#include <string>
#include "llvm/IR/BasicBlock.h"
#include "llvm/IR/Function.h"
#include "llvm/IR/IRBuilder.h"
#include "produce_call_func.hpp"
#include "t_ctx.hpp"
#include "t_types.hpp"
#include "t_smooth.hpp"
#include "t_bundles.hpp"
#include "evaluate_smooth.hpp"
#include "create_value_symbol_table.hpp"
#include "create_dummy_igc.hpp"
#include "destroy_dummy_igc.hpp"
#include "llvm_value.hpp"
#include "llvm_flexi_type.hpp"

llvm::Function* produce_call_func(
    IrGenCtx& igc,
    std::shared_ptr<TypeMap> map
) {
    if (false
        || map->call_input_type == nullptr
        || map->call_output_type == nullptr
    ) {
        return nullptr;
    }

    if (!map->bundle_id.has_value()) {
        fprintf(stderr, "Callable maps require a designated bundle.\n");
        exit(1);
    }

    Bundle* bundle = igc.toc->bundle_registry.get(map->bundle_id.value());

    auto p_bundle_map = std::get_if<std::shared_ptr<BundleMap>>(bundle);

    if (!p_bundle_map) {
        fprintf(stderr, "Bundle not from map.");
        exit(1);
    }

    if ((*p_bundle_map)->call_func != nullptr) {
        return (*p_bundle_map)->call_func;
    }

    DummyIgc dummy1 = create_dummy_igc(igc);
    Smooth input_smooth = evaluate_smooth(dummy1.igc, Type(map->call_input_type));
    llvm::StructType* input_struct_type = llvm::cast<llvm::StructType>(llvm_flexi_type(
    ↪ input_smooth));
    destroy_dummy_igc(dummy1);

    DummyIgc dummy2 = create_dummy_igc(igc);
    TypeMap output_shell = *map->call_output_type;
    output_shell.execution_sequence.clear();
    auto v_map = std::make_shared<TypeMap>(output_shell);
    Smooth output_smooth_probe = evaluate_smooth(dummy2.igc, Type(v_map)); // if this
    ↪ works, what a lifehack
    llvm::StructType* output_struct_type = llvm::cast<llvm::StructType>(llvm_flexi_type(
    ↪ output_smooth_probe));
    destroy_dummy_igc(dummy2);

    llvm::FunctionType* func_type = llvm::FunctionType::get(
        output_struct_type,
        { input_struct_type },
        false
    );

    static int next_callable_id = 0;
    std::string func_name = "__ec_callable_" + std::to_string(next_callable_id++);
```

```
    llvm::Function* func = llvm::Function::Create(
        func_type,
        llvm::Function::PrivateLinkage,
        func_name,
        igc.module
    );

    llvm::BasicBlock* func_entry = llvm::BasicBlock::Create(igc.context, "entry", func);
    llvm::IRBuilder<> func_builder(igc.context);
    func_builder.SetInsertPoint(func_entry);

    std::shared_ptr<ValueSymbolTable> new_value_table = std::make_shared<
    ↪ ValueSymbolTable>( // currently no closure ability implemented.
        create_value_symbol_table()
    );

    IrGenCtx enhanced_igc = {
        igc.context,
        igc.module,
        func_builder,
        new_value_table,
        igc.puts_func,
        igc.log_data_func,
        igc.log_data_deref_func,
        nullptr,
        igc.toc,
    };

    llvm::Argument* single_argument = func->getArg(0);
    single_argument->setName("InputMap");

    llvm::Value* input_alloca = func_builder.CreateAlloca(input_struct_type, nullptr, "
    ↪ input_struct");
    func_builder.CreateStore(single_argument, input_alloca);

    {
        if (!map->call_input_identifier.has_value()) {
            fprintf(stderr, "must take in input using entire destructure for now.\n");
            exit(1);
        }

        std::string input_var_name = "m_" + map->call_input_identifier.value();

        ValueSymbolTableEntry input_entry{
            input_alloca,
            input_struct_type,
            Type(map->call_input_type),
            false,
        };

        new_value_table->set(input_var_name, input_entry);
    }

    Smooth output_smooth = evaluate_smooth(enhanced_igc, Type(map->call_output_type));
    func_builder.CreateRet(llvm_value(output_smooth));

    (*p_bundle_map)->call_func = func;

    return func;
}
```

compiler/backend/src/produce_call_func.hpp

```
#pragma once

#include "llvm/IR/Function.h"
#include "t_ctx.hpp"
```

```
#include "t_types.hpp"

llvm::Function* produce_call_func(
    IrGenCtx& igc,
    std::shared_ptr<TypeMap> map
);
```

compiler/backend/src/promote_to_underlying.cpp

```
#include <cstdio>
#include <cstdlib>
#include <typeinfo>
#include "promote_to_underlying.hpp"

UnderlyingType promote_to_underlying(const Type& type) {
    if (auto p = std::get_if<std::shared_ptr<TypeMap>>(&type)) {
        return *p;
    }

    if (auto p = std::get_if<std::shared_ptr<TypePointer>>(&type)) {
        return *p;
    }

    if (auto p = std::get_if<std::shared_ptr<TypeMerged>>(&type)) {
        return *p;
    }

    if (auto p = std::get_if<std::shared_ptr<TypeRotten>>(&type)) {
        return *p;
    }

    if (auto p = std::get_if<std::shared_ptr<TypeEnum>>(&type)) {
        return *p;
    }

    const char* name = std::visit([](auto&& v) { return typeid(*v).name(); }, type);
    fprintf(stderr, "Did not encounter map/pointer/merged/rotten during promotion to
    ↪ underlying, received %s\n", name);
    exit(1);
}
```

compiler/backend/src/promote_to_underlying.hpp

```
#pragma once

#include "t_types_fwd.hpp"
#include "t_types.hpp"

UnderlyingType promote_to_underlying(const Type& type);
```

compiler/backend/src/rotten_float_info.cpp

```
#include "rotten_float_info.hpp"
#include "t_types.hpp"

std::optional<RottenFloatInfo> rotten_float_info(std::shared_ptr<TypeRotten> rotten) {
    if (rotten->type_str.empty() || rotten->type_str[0] != 'f') {
        return std::nullopt;
    }

    return RottenFloatInfo{
```

```
        (uint32_t) std::stoul(rotten->type_str.substr(1)),
    };
}
```

compiler/backend/src/rotten_float_info.hpp

```
#pragma once

#include <cstdint>
#include <optional>
#include <string>
#include "t_types_fwd.hpp"

struct RottenFloatInfo {
    uint32_t bits;
};

std::optional<RottenFloatInfo> rotten_float_info(std::shared_ptr<TypeRotten> rotten);
```

compiler/backend/src/rotten_int_info.cpp

```
#include "rotten_int_info.hpp"
#include "t_types.hpp"

std::optional<RottenIntInfo> rotten_int_info(std::shared_ptr<TypeRotten> rotten) {
    if ((rotten->type_str).empty()) {
        return std::nullopt;
    }

    const char prefix = (rotten->type_str)[0];

    if (prefix != 'i' && prefix != 'u') {
        return std::nullopt;
    }

    return RottenIntInfo{
        prefix,
        (uint32_t) std::stoul((rotten->type_str).substr(1)),
    };
}
```

compiler/backend/src/rotten_int_info.hpp

```
#pragma once

#include <cstdint>
#include <optional>
#include <string>
#include "t_types_fwd.hpp"

struct RottenIntInfo {
    char prefix;
    uint32_t bits;
};

std::optional<RottenIntInfo> rotten_int_info(std::shared_ptr<TypeRotten> rotten);
```

compiler/backend/src/smooth_type.cpp

```
#include "smooth_type.hpp"
#include "preinstantiated_types.hpp"
#include "t_smooth.hpp"
#include "t_types.hpp"

Type smooth_type(Smooth smooth) {
    if (auto p_v_structval = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth)) {
        return (*p_v_structval)->type;
    }

    if (auto p_v_pointer = std::get_if<std::shared_ptr<SmoothPointer>>(&smooth)) {
        return (*p_v_pointer)->type;
    }

    if (auto p_v_enum = std::get_if<std::shared_ptr<SmoothEnum>>(&smooth)) {
        return (*p_v_enum)->type;
    }

    if (auto p_v_int = std::get_if<std::shared_ptr<SmoothInt>>(&smooth)) {
        return (*p_v_int)->type;
    }

    if (auto p_v_float = std::get_if<std::shared_ptr<SmoothFloat>>(&smooth)) {
        return (*p_v_float)->type;
    }

    if (auto p_v_void = std::get_if<std::shared_ptr<SmoothVoid>>(&smooth)) {
        if ((*p_v_void)->type.has_value()) {
            return (*p_v_void)->type.value();
        }

        return Type(type_void);
    }

    if (auto p_v_void_int = std::get_if<std::shared_ptr<SmoothVoidInt>>(&smooth)) {
        return (*p_v_void_int)->type;
    }

    if (auto p_v_void_float = std::get_if<std::shared_ptr<SmoothVoidFloat>>(&smooth)) {
        return (*p_v_void_float)->type;
    }

    fprintf(stderr, "The particular smooth does not have a type associated with it (
    ↪ programmer bug?)\n");
    exit(1);
}
```

compiler/backend/src/smooth_type.hpp

```
#pragma once

#include "t_smooth_fwd.hpp"
#include "t_types_fwd.hpp"

Type smooth_type(Smooth smooth);
```

compiler/backend/src/structwrap.cpp

```
#include <bit>
#include <cstdlib>
#include "llvm/IR/Constants.h"
#include "llvm/IR/DerivedTypes.h"
#include "structwrap.hpp"
#include "t_smooth.hpp"
```

```
#include "t_types.hpp"

std::shared_ptr<SmoothStructval> structwrap(IrGenCtx& igc, const Smooth& smooth) {
    if (auto p_v_structval = std::get_if<std::shared_ptr<SmoothStructval>>(&smooth)) {
        return *p_v_structval;
    }

    if (auto p_v_pointer = std::get_if<std::shared_ptr<SmoothPointer>>(&smooth)) {
        const auto& v_pointer = *p_v_pointer;

        llvm::StructType* struct_type = llvm::StructType::get(igc.context, llvm::
    ↪ ArrayRef<llvm::Type*>{ v_pointer->value->getType() });
        llvm::Value* struct_value = llvm::UndefValue::get(struct_type);
        struct_value = igc.builder.CreateInsertValue(struct_value, v_pointer->value, 0);

        auto v_map = std::make_shared<TypeMap>();
        v_map->leaf_type = v_pointer->type;

        return std::make_shared<SmoothStructval>(SmoothStructval{
            Type(v_map),
            struct_value,
            true,
            std::make_shared<SmoothPointer>(SmoothPointer{
                v_pointer->type, // todo: is this the right type.
                igc.builder.CreateExtractValue(struct_value, 0),
            }),
            nullptr,
        });
    }

    if (auto p_v_int = std::get_if<std::shared_ptr<SmoothInt>>(&smooth)) {
        const auto& v_int = *p_v_int;

        llvm::StructType* struct_type = llvm::StructType::get(igc.context, llvm::
    ↪ ArrayRef<llvm::Type*>{ v_int->value->getType() });
        llvm::Value* struct_value = llvm::UndefValue::get(struct_type);
        struct_value = igc.builder.CreateInsertValue(struct_value, v_int->value, 0);

        auto v_map = std::make_shared<TypeMap>();
        v_map->leaf_type = v_int->type;

        return std::make_shared<SmoothStructval>(SmoothStructval{
            Type(v_map),
            struct_value,
            true,
            std::make_shared<SmoothInt>(SmoothInt{
                v_int->type,
                igc.builder.CreateExtractValue(struct_value, 0),
            }),
            nullptr,
        });
    }

    if (auto p_v_float = std::get_if<std::shared_ptr<SmoothFloat>>(&smooth)) {
        const auto& v_float = *p_v_float;

        llvm::StructType* struct_type = llvm::StructType::get(igc.context, llvm::
    ↪ ArrayRef<llvm::Type*>{ v_float->value->getType() });
        llvm::Value* struct_value = llvm::UndefValue::get(struct_type);
        struct_value = igc.builder.CreateInsertValue(struct_value, v_float->value, 0);

        auto v_map = std::make_shared<TypeMap>();
        v_map->leaf_type = v_float->type;

        return std::make_shared<SmoothStructval>(SmoothStructval{
            Type(v_map),
            struct_value,
            true,
            std::make_shared<SmoothFloat>(SmoothFloat{
                v_float->type,
```

```
                igc.builder.CreateExtractValue(struct_value, 0),
            }),
            nullptr,
        });
    }

    fprintf(stderr, "the provided Smooth is not structwrappable.\n");
    exit(1);
}
```

compiler/backend/src/structwrap.hpp

```
#pragma once

#include "t_ctx.hpp"
#include "t_smooth_fwd.hpp"
#include "t_smooth.hpp"

std::shared_ptr<SmoothStructval> structwrap(IrGenCtx& igc, const Smooth& smooth);
```

compiler/backend/src/t_bundles.hpp

```
#pragma once

#include "t_bundles_fwd.hpp"
#include "llvm/IR/Function.h"

struct BundleMap {
    llvm::Function* call_func;
};
```

compiler/backend/src/t_bundles_fwd.hpp

```
#pragma once

#include <memory>
#include <variant>

struct BundleMap;

using Bundle = std::variant<
    std::shared_ptr<BundleMap>
>;
```

compiler/backend/src/t_ctx.hpp

```
#pragma once

#include <memory>
#include "llvm/IR/LLVMContext.h"
#include "llvm/IR/Module.h"
#include "llvm/IR/IRBuilder.h"
#include "llvm/IR/Function.h"
#include "llvm/IR/BasicBlock.h"
#include "create_type_symbol_table.hpp"
#include "create_value_symbol_table.hpp"
#include "create_bundle_registry.hpp"
```

```
struct TypeOrchCtx {
    TypeSymbolTable type_table;
    BundleRegistry bundle_registry;
};

struct IrGenCtx {
    llvm::LLVMContext& context;
    llvm::Module& module;
    llvm::IRBuilder<>& builder;
    std::shared_ptr<ValueSymbolTable> value_table;
    llvm::FunctionCallee& puts_func;
    llvm::Function* log_data_func;
    llvm::Function* log_data_deref_func;
    llvm::BasicBlock* block_break;
    std::shared_ptr<TypeOrchCtx> toc;
};
```

compiler/backend/src/t_hardval.hpp

```
#pragma once

#include <string>
#include "t_hardval_fwd.hpp"

struct HardvalInteger {
    std::string value;
};

struct HardvalFloat {
    std::string value;
};

struct HardvalString {
    std::string value;
};
```

compiler/backend/src/t_hardval_fwd.hpp

```
#pragma once

#include <memory>
#include <variant>

struct HardvalInteger;
struct HardvalFloat;
struct HardvalString;

// Forward declaration impossible without pointer indirection, typical C++.

using Hardval = std::variant<
    std::shared_ptr<HardvalInteger>,
    std::shared_ptr<HardvalFloat>,
    std::shared_ptr<HardvalString>
>;
```

compiler/backend/src/t_instructions.hpp

```
#pragma once

#include <string>
#include <memory>
```

```
#include <vector>
#include "t_instructions_fwd.hpp"
#include "t_types_fwd.hpp"

struct InstructionSym {
    std::string name;
    std::shared_ptr<Type> typeval;
};

struct InstructionExpr {
    std::shared_ptr<Type> expr;
};

struct InstructionFor {
    std::shared_ptr<TypeMap> body;
};

struct InstructionIf_Branch {
    std::shared_ptr<Type> condition;
    std::shared_ptr<TypeMap> body;
};

struct InstructionIf {
    std::vector<InstructionIf_Branch> branches;
};

struct InstructionBreak {
};
```

compiler/backend/src/t_instructions_fwd.hpp

```
#pragma once

#include <memory>
#include <variant>

struct InstructionSym;
struct InstructionExpr;
struct InstructionFor;
struct InstructionIf;
struct InstructionBreak;

// Forward declaration impossible without pointer indirection, typical C++.

using Instruction = std::variant<
    std::shared_ptr<InstructionSym>,
    std::shared_ptr<InstructionExpr>,
    std::shared_ptr<InstructionFor>,
    std::shared_ptr<InstructionIf>,
    std::shared_ptr<InstructionBreak>
>;
```

compiler/backend/src/t_smooth.hpp

```
#pragma once

#include <optional>
#include <vector>
#include "llvm/IR/Function.h"
#include "llvm/IR/Value.h"
#include "t_types_fwd.hpp"
#include "t_smooth_fwd.hpp"

struct SmoothStructval {
```

```
    Type type;
    llvm::Value* value;
    bool has_leaf;
    std::optional<Smooth> leaf;
    llvm::Function* call_func;
    std::vector<Smooth> field_smooths;
};

struct SmoothPointer {
    Type type;
    llvm::Value* value;
};

struct SmoothEnum {
    Type type;
    llvm::Value* value;
};

struct SmoothInt {
    Type type;
    llvm::Value* value;
};

struct SmoothFloat {
    Type type;
    llvm::Value* value;
};

struct SmoothVoid {
    std::optional<Type> type;
    llvm::Value* value;
};

struct SmoothVoidInt {
    Type type;
    llvm::Type* flexi_type;
    llvm::Value* value;
};

struct SmoothVoidFloat {
    Type type;
    llvm::Type* flexi_type;
    llvm::Value* value;
};
```

compiler/backend/src/t_smooth_fwd.hpp

```
#pragma once

#include <memory>
#include <variant>

struct SmoothStructval;
struct SmoothPointer;
struct SmoothEnum;
struct SmoothInt;
struct SmoothFloat;
struct SmoothVoid;
struct SmoothVoidInt;
struct SmoothVoidFloat;

// Forward declaration impossible without pointer indirection, typical C++.

using Smooth = std::variant<
    std::shared_ptr<SmoothStructval>,
    std::shared_ptr<SmoothPointer>,
    std::shared_ptr<SmoothEnum>,
```

```
    std::shared_ptr<SmoothInt>,
    std::shared_ptr<SmoothFloat>,
    std::shared_ptr<SmoothVoid>,
    std::shared_ptr<SmoothVoidInt>,
    std::shared_ptr<SmoothVoidFloat>
>;
```

compiler/backend/src/t_types.hpp

```
#pragma once

#include <cstdint>
#include <memory>
#include <map>
#include <optional>
#include <string>
#include <vector>
#include <variant>
#include "t_types_fwd.hpp"
#include "t_instructions_fwd.hpp"
#include "t_hardval_fwd.hpp"

struct TypeMap {
    std::optional<Type> leaf_type;
    std::optional<Hardval> leaf_hardval;
    std::optional<std::string> call_input_identifier;
    std::shared_ptr<TypeMap> call_input_type;
    std::shared_ptr<TypeMap> call_output_type;
    std::map<std::string, std::shared_ptr<Type>> sym_inputs;
    std::vector<Instruction> execution_sequence;
    std::optional<uint64_t> bundle_id;
};

struct TypePointer {
    std::shared_ptr<Type> target;
    std::optional<Hardval> hardval; // eventually this may need to be organized better,
    ↪ not sure how to best approach it.
};

struct TypeVarAccess {
    std::string target_name;
    std::shared_ptr<Type> underlying_type;
};

struct TypeMerged {
    std::vector<Type> types;
    std::shared_ptr<Type> underlying_type;
};

struct TypeRotten {
    std::string type_str;
};

struct TypeEnum {
    bool is_instantiated; // to distinguish between no hardsym vs unknown (runtime)
    ↪ hardsym.
    // wait, i should wrap in an overlying type instead of using an is_instantiated
    ↪ eventually (we'll see).
    // is_instantiated and hardsym are mutually exclusive.
    // hardsym - exact constant is known at compile-time;
    // is_instantiated - value is known at runtime;
    // neitehr hardsym nor is_instantiated - it is just an enum type "enum { ... }", not
    ↪  an actual value.
    std::optional<std::string> hardsym; // this is a restrictive constraint, when
    ↪ merging, we imagine any value that satisfies ALL.
    std::vector<std::string> syms; // this is a permissive constraint, when merging,
    ↪ imagine any value that satisfies ANY.
```

```
};

struct TypeVoid {
};

struct TypeLog {
    std::shared_ptr<Type> message;
};

struct TypeLogD {
    std::shared_ptr<Type> message;
};

struct TypeLogDd {
    std::shared_ptr<Type> message;
    std::shared_ptr<Type> byte_count;
    bool is_nullterm;
};

struct TypeVarWalrus {
    std::string name;
    bool is_mut;
    std::shared_ptr<Type> typeval;
    std::shared_ptr<Type> underlying_type; // note this is not the underlying type of
    ↪ the assignment, but rather the usage of assignment as an expression, which
    ↪ evaluates to a variable access - equivalent to underlying_type in TypeVarAccess.
};

struct TypeVarAssign {
    std::string name;
    std::shared_ptr<Type> typeval;
    std::shared_ptr<Type> underlying_type; // note this is similar to TypeVarWalrus
};

struct TypeCallWithSym {
    std::shared_ptr<Type> target;
    std::string sym;
    std::shared_ptr<Type> underlying_type;
};

struct TypeCallWithDynamic {
    std::shared_ptr<Type> target;
    std::shared_ptr<Type> call_data;
    std::shared_ptr<Type> underlying_type;
};

struct OpNumeric {
    std::string op;
    std::shared_ptr<Type> operand;
};

struct TypeExprMulti {
    std::vector<OpNumeric> ops;
    std::shared_ptr<Type> underlying_type;
};

struct TypeExprAddit {
    std::vector<OpNumeric> ops;
    std::shared_ptr<Type> underlying_type;
};

struct TypeCompare {
    std::string operator_;
    std::shared_ptr<Type> operand_a;
    std::shared_ptr<Type> operand_b;
    std::shared_ptr<Type> underlying_type;
};

struct OpLogical {
    std::shared_ptr<Type> operand;
```

```
};

struct TypeExprLogicalAnd {
    std::vector<OpLogical> ops;
    std::shared_ptr<Type> underlying_type;
};

struct TypeExprLogicalOr {
    std::vector<OpLogical> ops;
    std::shared_ptr<Type> underlying_type;
};
```

compiler/backend/src/t_types_fwd.hpp

```
#pragma once

#include <memory>
#include <variant>

// While these are called types, they can often be thought of as expressions (as the
    ↪ design has shifted over time).

// The following terms are used fairly interchangebly throughout this project, due to
    ↪ legacy naming:
// - expression
// - type
// - typeval
// - constraint

struct TypeMap;
struct TypePointer;
struct TypeVarAccess;
struct TypeMerged;
struct TypeRotten;
struct TypeEnum;
struct TypeLog;
struct TypeLogD;
struct TypeLogDd;
struct TypeVarWalrus;
struct TypeVarAssign;
struct TypeCallWithSym;
struct TypeCallWithDynamic;
struct TypeExprMulti;
struct TypeExprAddit;
struct TypeExprLogicalAnd;
struct TypeExprLogicalOr;
struct TypeCompare;
struct TypeVoid;

// Forward declaration impossible without pointer indirection, typical C++.

using Type = std::variant<
    std::shared_ptr<TypeMap>,
    std::shared_ptr<TypePointer>,
    std::shared_ptr<TypeVarAccess>,
    std::shared_ptr<TypeMerged>,
    std::shared_ptr<TypeRotten>,
    std::shared_ptr<TypeEnum>,
    std::shared_ptr<TypeLog>,
    std::shared_ptr<TypeLogD>,
    std::shared_ptr<TypeLogDd>,
    std::shared_ptr<TypeVarWalrus>,
    std::shared_ptr<TypeVarAssign>,
    std::shared_ptr<TypeCallWithSym>,
    std::shared_ptr<TypeCallWithDynamic>,
    std::shared_ptr<TypeExprMulti>,
    std::shared_ptr<TypeExprAddit>,
```

```
    std::shared_ptr<TypeExprLogicalAnd>,
    std::shared_ptr<TypeExprLogicalOr>,
    std::shared_ptr<TypeCompare>,
    std::shared_ptr<TypeVoid>
>;

using UnderlyingType = std::variant< // an underlying type is kind of the eventual type
    ↪ after evaluation.
    std::shared_ptr<TypeMap>,
    std::shared_ptr<TypePointer>,
    std::shared_ptr<TypeMerged>,
    std::shared_ptr<TypeRotten>,
    std::shared_ptr<TypeEnum>
>;
```

compiler/backend/src/total_needed_bits_addit.cpp

```
#include "total_needed_bits_addit.hpp"

#include <cmath>
#include <cstdint>
#include <vector>

// this uses a logarithmic approach (well, sort of, but by using hardware floats) to fit
    ↪  it into a double.
// this avoids needing a "BigInt" type in C++.
// idea inspired from LLM, see appendix.

uint32_t total_needed_bits_addit(const std::vector<uint32_t>& bits) {
    if (bits.empty()) {
        return 0;
    }

    double max_sum = 0.0;

    for (const uint32_t bit : bits) {
        max_sum += std::pow(2.0, (double) bit) - 1.0;
    }

    if (max_sum <= 0.0) {
        return 0;
    }

    double my_epsilon = 1e-9; // in case precision is lost, we need to be a tiny bit
    ↪ more conservative.

    return (uint32_t) std::floor(std::log2(max_sum) + my_epsilon) + 1;
}
```

compiler/backend/src/total_needed_bits_addit.hpp

```
#pragma once

#include <cstdint>
#include <vector>

uint32_t total_needed_bits_addit(const std::vector<uint32_t>& bits);
```

compiler/backend/src/total_needed_bits_multi.cpp

```
#include "total_needed_bits_multi.hpp"
```

```
#include <cstdint>
#include <numeric>
#include <vector>

// i think this implementation is right, but this is not a major concern for me.

uint32_t total_needed_bits_multi(const std::vector<uint32_t>& operand_bits) {
    return std::accumulate(operand_bits.begin(), operand_bits.end(), (uint32_t) 0);
}
```

compiler/backend/src/total_needed_bits_multi.hpp

```
#pragma once

#include <cstdint>
#include <vector>

uint32_t total_needed_bits_multi(const std::vector<uint32_t>& operand_bits);
```

compiler/backend/src/wrap_leafable.cpp

```
#include "wrap_leafable.hpp"
#include "t_types.hpp"

TypeMap wrap_leafable(const Type& leafable) {
    TypeMap result;
    result.leaf_type = leafable;
    result.leaf_hardval = std::nullopt;
    return result;
}
```

compiler/backend/src/wrap_leafable.hpp

```
#pragma once

#include "t_types.hpp"

TypeMap wrap_leafable(const Type& leafable);
```

compiler/ec.js

```
import { Command } from "commander";
import { writeFile } from "fs/promises";
import { join } from "path";
import { process as frontend_process } from "./frontend/process.js";
import { spawn } from "child_process";
import { create_promise } from "./frontend/uoe/create_promise.js";
import { throw_error } from "./frontend/uoe/throw_error.js";
import { error_internal } from "./frontend/uoe/error_internal.js";

const program = new Command();

const backend_process = async (result) => {
    console.log("Launching backend...");

    const [promise, resolve, reject] = create_promise();
```

```
    const backend_dir = join(import.meta.dirname, "backend");
    const args = [`${backend_dir}/host/run.sh`, result.module_path];

    const backend = spawn("bash", args, {
        stdio: "inherit",
        cwd: backend_dir,
    });

    backend.on("close", (code) => {
        if (code === 0) {
            resolve();
        } else {
            reject(throw_error(error_internal(`Backend exited with unsatisfactory code $
    ↪ {code}`)));
        }
    });

    backend.on("error", (err) => {
        reject(err);
    });

    return promise;
};

const compile_module = async (config) => {
    console.log("Compiling module located at:", config.src);
    const result = await frontend_process(config);
    const out_path = join(result.module_path, "frontend.out.json");

    const safe_result = {
        ...result,
        parse_output: {
            ...result.parse_output,
            source_scopes: undefined,
        },
    };

    await writeFile(out_path, JSON.stringify(safe_result, (_key, value) => {
        if (typeof value === "bigint") {
            return value.toString();
        }

        return value;
    }, "\t"));

    console.log("Frontend output dumped to", out_path);

    await backend_process(result);
};

program
    .name("ec")
    .description("Essence C compiler")
    .version("1.0.0");

program
    .command("compile <src> <dest>")
    .description("Compile a module located at <src> and output build artifacts to <dest
    ↪ >")
    .action(async (src, dest, _options) => {
        await compile_module({
            src,
            dest,
        });
    });

program.parse();
```

compiler/frontend/README.md

```
# Compiler Frontend

Language: JavaScript.
```

compiler/frontend/create_dir_node_translator.js

```
import { create_unsuspended_factory } from "./uoe/create_unsuspended_factory.js";
import { throw_error } from "./uoe/throw_error.js";
import { error_user_state } from "./uoe/error_user_state.js";

class DirNodeTranslator {
    _init(dependencies) {
        this._dependencies = dependencies;
        this._ino_to_uuid = new Map();
        this._uuid_to_path = new Map();
        this._uuid_to_ino = new Map();
        this._path_to_uuid = new Map();
    }

    async add(path) {
        const absolute_path = this._dependencies.path.resolve(path);

        const existing_uuid_by_path = this._path_to_uuid.get(absolute_path);

        if (existing_uuid_by_path !== undefined) {
            return existing_uuid_by_path;
        }

        try {
            var stats = await this._dependencies.fs.stat(absolute_path);
        } catch (error) {
            if (error.code === "ENOENT") {
                throw_error(error_user_state(`Directory non-existent on filesystem: ${
    ↪ absolute_path}`));
            }
            throw error;
        }

        const existing_uuid = this._ino_to_uuid.get(stats.ino);

        if (existing_uuid !== undefined) {
            return existing_uuid;
        }

        const uuid = this._dependencies.randomUUID();

        this._ino_to_uuid.set(stats.ino, uuid);
        this._uuid_to_path.set(uuid, absolute_path);
        this._uuid_to_ino.set(uuid, stats.ino);
        this._path_to_uuid.set(absolute_path, uuid);

        return uuid;
    }

    async add_abstract(uuid, rel_path) {
        const base_path = this._uuid_to_path.get(uuid);

        if (base_path === undefined) {
            throw_error(error_user_state(`UUID ${uuid} not recognized.`));
        }

        if (rel_path.includes("..")) {
            throw_error(error_user_state(`Safety: to prevent escaping directory of
    ↪ abstract path, ".." is disallowed. Todo: is this exhaustive?`));
            // todo: maybe node offers flag to force safety like Python does iirc
```

```
        }

        const combined_path = this._dependencies.path.join(base_path, rel_path);

        return await this.add(combined_path);
    }

    get(uuid) {
        return this._uuid_to_path.get(uuid);
    }

    dump_translations() {
        const translations = {};

        for (const [uuid, path] of this._uuid_to_path.entries()) {
            translations[uuid] = {
                path,
            };
        }

        return translations;
    }
}

export const create_dir_node_translator = create_unsuspended_factory(DirNodeTranslator);
```

compiler/frontend/create_fs_source_scope.js

```
import { create_unsuspended_factory } from "./uoe/create_unsuspended_factory.js";

class FsSourceScope {
    _init(dependencies, base_path) {
        this._dependencies = dependencies;
        this._base_path = base_path;
    }

    async resolve(path) {
        const directory = this._dependencies.path.join(this._base_path, path);
        return await this._dependencies.dir_node_translator.add(directory);
    }
}

export const create_fs_source_scope = create_unsuspended_factory(FsSourceScope);
```

compiler/frontend/create_source_extractor.js

```
import { create_unsuspended_factory } from "./uoe/create_unsuspended_factory.js";
import { error_user_state } from "./uoe/error_user_state.js";
import { throw_error } from "./uoe/throw_error.js";

class SourceExtractor {
    _init(dependencies) {
        this._dependencies = dependencies;
    }

    async extract(uuid) {
        // todo: add lru cache of like 1000 entries here.

        const directory = await this._dependencies.dir_node_translator.get(uuid);
        const files = await this._dependencies.fs.readdir(directory);

        if (files.some(file => file.endsWith(".ec"))) {
            return {
                type: "SOURCE MODULE",
```

```
                uuid,
                files: files.filter(file => file.endsWith(".ec")),
            };
        } else if (files.some(file => file.endsWith(".eh"))) {
            return {
                type: "INTERFACE MODULE",
                uuid,
                files: files.filter(file => file.endsWith(".eh")),
            };
        }

        throw_error(error_user_state(`Filesystem structure at ${uuid} does not align
    ↪ with a source or interface module.`));
    }
}

export const create_source_extractor = create_unsuspended_factory(SourceExtractor);
```

compiler/frontend/create_source_scope.js

```
import { create_unsuspended_factory } from "./uoe/create_unsuspended_factory.js";
import { throw_error } from "./uoe/throw_error.js";
import { error_user_payload } from "./uoe/error_user_payload.js";
import { error_user_state } from "./uoe/error_user_state.js";

const validate_redirect = (redirect) => {
    if (false
        || !Array.isArray(redirect)
        || typeof redirect[0] !== "string"
        || (true
            && redirect[1] !== undefined
            && typeof redirect[1] !== "function"
            && typeof redirect[1] !== "object"
        )
        || (true
            && redirect[2] !== undefined
            && typeof redirect[2] !== "string"
        )
    ) {
        return false;
    }

    return true;
};

const validate_redirects = (redirects) => {
    if (false
        || !Array.isArray(redirects)
        || redirects.some(redirect => !validate_redirect(redirect))
    ) {
        throw_error(error_user_payload("Validation error."));
    }
};

const validate_path = (path) => {
    if (false
        || typeof path !== "string"
        || !/^[a-z0-9_\/.]*$/.test(path)
        || path.includes("..") // prevent user from escaping sandbox.
    ) {
        throw_error(error_user_payload("Validation error."));
    }
};

class SourceScope {
    async _init(dependencies, redirects) {
        validate_redirects(redirects);
```

```
        this._dependencies = dependencies;

        const optimized_redirects = [];

        for (const [prefix, parent_scope, replacement] of redirects) {
            if (parent_scope === undefined) {
                optimized_redirects.push([prefix, undefined]);
            } else {
                const uuid = await parent_scope.resolve(replacement ?? prefix);
                optimized_redirects.push([prefix, uuid]);
            }
        }

        // the goal is to get the longer prefixes first to ensure correct precedence.
        // todo: is this exactly how I want it?
        this._redirects = optimized_redirects.sort((a, b) => b[0].length - a[0].length);
    }

    async resolve(path) {
        validate_path(path);

        for (const [prefix, uuid] of this._redirects) {
            if (path.startsWith(prefix)) {
                if (uuid === undefined) {
                    throw_error(error_user_state(`Path "${path}" has been reset,
    ↪ preventing you from escaping the predefined boundary. Did the creator of this
    ↪ scope forget to provide additional redirects (if applicable)?`));
                }

                const remainder = path.slice(prefix.length);
                return await this._dependencies.dir_node_translator.add_abstract(uuid,
    ↪ remainder);
            }
        }

        throw_error(error_user_state(`Path "${path}" does not play well with the
    ↪ predefined redirects. Did the creator of this scope forget to provide a catch-all
    ↪ (if applicable)?`));
    }
}

export const create_source_scope = create_unsuspended_factory(SourceScope);
```

compiler/frontend/process.js

```
import { mapData, join, opt, multi, opt_multi, or, declare, trace_print, trace_dump,
    ↪ rule, lookAhead } from "./uoe/ec/blurp.js";

import * as fs from "fs/promises";
import * as node__path from "path";
import { randomUUID } from "crypto";
import { create_fs_source_scope } from "./create_fs_source_scope.js";
import { create_source_scope as orig_create_source_scope } from "./create_source_scope.
    ↪ js";
import { create_source_extractor } from "./create_source_extractor.js";
import { create_dir_node_translator } from "./create_dir_node_translator.js";
import { throw_error } from "./uoe/throw_error.js";
import { error_internal } from "./uoe/error_internal.js";

// LEXICAL TOKENS

const WHITESPACE = rule("WHITESPACE", /^\s+/);
const REALSPACE = rule("REALSPACE", /^[ \t]+/);
const SINGLELINE_COMMENT = rule("SINGLELINE_COMMENT", mapData(/^\s*;(.*?)(\n|$)\s*/,
    ↪ data => data.groups[0]));
const SKIPPERS = rule("SKIPPERS", opt(multi(or(SINGLELINE_COMMENT, WHITESPACE))));
```

```
const CORE_SKIPPERS = rule("CORE_SKIPPERS", opt(multi(REALSPACE)));
const NEWLINE = rule("NEWLINE", /^\n/);

const withSkippers = (p) => mapData(join(SKIPPERS, p, SKIPPERS), data => data[1]);
const withCarefulSkippers = (p) => mapData(join(SKIPPERS, p, CORE_SKIPPERS), data =>
    ↪ data[1]);
const withLeftSkippers = (p) => mapData(join(SKIPPERS, p), data => data[1]);
const withRightSkippers = (p) => mapData(join(p, SKIPPERS), data => data[0]);
const withBareboneSkippers = (p) => mapData(join(CORE_SKIPPERS, p, CORE_SKIPPERS), data
    ↪ => data[1]);

const MANDATORY_NEWLINE = rule("MANDATORY_NEWLINE", join(CORE_SKIPPERS, or(
    ↪ SINGLELINE_COMMENT, NEWLINE)));

const KW_FORWARDING = rule("KW_FORWARDING", withCarefulSkippers("forwarding"));
const KW_RESET = rule("KW_RESET", withCarefulSkippers("reset"));
const KW_TYPE = rule("KW_TYPE", withCarefulSkippers("type"));
const KW_TYPES = rule("KW_TYPES", withCarefulSkippers("types"));
const KW_MAP = rule("KW_MAP", withCarefulSkippers("map"));
const KW_IMPORT = rule("KW_IMPORT", withCarefulSkippers("import"));
const KW_MOD = rule("KW_MOD", withCarefulSkippers("mod"));
const KW_CREATE = rule("KW_CREATE", withCarefulSkippers("create"));
const KW_LOG = rule("KW_LOG", withCarefulSkippers("log"));
const KW_LOG_D = rule("KW_LOG_D", withCarefulSkippers("log_d"));
const KW_LOG_DD = rule("KW_LOG_DD", withCarefulSkippers("log_dd"));
const KW_NULLTERM = rule("KW_NULLTERM", withCarefulSkippers("null-term"));
const KW_MUT = rule("KW_MUT", withCarefulSkippers("mut"));
const KW_FOR = rule("KW_FOR", withCarefulSkippers("for"));
const KW_ENUM = rule("KW_ENUM", withCarefulSkippers("enum"));
const KW_IF = rule("KW_IF", withCarefulSkippers("if"));
const KW_ELSE = rule("KW_ELSE", withCarefulSkippers("else"));
const KW_BREAK = rule("KW_BREAK", withCarefulSkippers("break"));
const LBRACE = rule("LBRACE", withCarefulSkippers("{"));
const LBRACE_BB = rule("LBRACE_BB", withBareboneSkippers("{"));
const RBRACE = rule("RBRACE", withCarefulSkippers("}"));
const LPAREN = rule("LPAREN", withCarefulSkippers("("));
const LPAREN_BB = rule("LPAREN_BB", withBareboneSkippers("("));
const RPAREN = rule("RPAREN", withCarefulSkippers(")"));
const ARROW = rule("ARROW", withCarefulSkippers("->"));
const ASTERISK = rule("ASTERISK", withCarefulSkippers("*"));
const COMMA = rule("COMMA", withCarefulSkippers(","));
const PATH_OUTER_ROOT = rule("PATH_OUTER_ROOT", withCarefulSkippers(mapData(/^\/\/[a-z0
    ↪ -9_\/\.]*/, data => data.groups.all)));
const PATH_MODULE_ROOT = rule("PATH_MODULE_ROOT", withCarefulSkippers(mapData(/^\/[a-z0
    ↪ -9_\/\.]*/, data => data.groups.all)));
const PATH = rule("PATH", or(PATH_OUTER_ROOT, PATH_MODULE_ROOT));
const OUTER_ROOT = rule("OUTER_ROOT", withCarefulSkippers("//"));
const SEMI = rule("SEMI", withRightSkippers(SINGLELINE_COMMENT));
// i need to go through all these again. why did I put "_" outside the []. why is
    ↪ underscore and numbers not matched in the first part?
const TYPE_IDENT = rule("TYPE_IDENT", withCarefulSkippers(mapData(/^(?:(?:([a-z]+(?:_[a-
    ↪ z0-9]+)*::)*[A-Z][a-zA-Z0-9]*)|(?:i[1-9][0-9]{0,4}|u[1-9][0-9]{0,4}|f16|f32|f64|
    ↪ f128|bool)(?![a-zA-Z0-9_]))/, data => data.groups.all))); // negative lookahead is
    ↪  required because there are no token boundaries in this parsing system.
const TYPES_IDENT = rule("TYPES_IDENT", withCarefulSkippers(mapData(/^(?:([a-z]+(?:_[a-
    ↪ z0-9]+)*::)*(?:[a-z]+(?:_[a-z0-9]+)*))/, data => data.groups.all)));
const MOD_IDENT = rule("MOD_IDENT", withCarefulSkippers(mapData(/^(?:([a-z]+(?:_[a-z0
    ↪ -9]+)*::)*(?:[a-z]+(?:_[a-z0-9]+)*))/, data => data.groups.all))); // identical to
    ↪  TYPES_IDENT for now.
const SYMBOL = rule("SYMBOL", withCarefulSkippers(mapData(/^:([a-zA-Z_][a-zA-Z0-9_]*)/,
    ↪ data => data.groups.all)));
const SYMBOL_BARE = rule("SYMBOL_BARE", withBareboneSkippers(mapData(/^:([a-zA-Z_][a-zA-
    ↪ Z0-9_]*)/, data => data.groups.all)));
const SYMBOL_TIGHT = rule("SYMBOL_TIGHT", mapData(/^:([a-zA-Z_][a-zA-Z0-9_]*)/, data =>
    ↪ data.groups.all));
const SYMBOL_GRAND = rule("SYMBOL_GRAND", withCarefulSkippers(mapData(
    multi(SYMBOL_TIGHT),
    (data) => data.map(s => s.substring(1)),
)));
const IDENT = rule("IDENT", withCarefulSkippers(mapData(/^([a-zA-Z_][a-zA-Z0-9_]*)/,
```

```
    ↪ data => data.groups.all)));
const IDENT_BB = rule("IDENT_BB", withBareboneSkippers(mapData(/^([a-zA-Z_][a-zA-Z0-9_
    ↪ ]*)/, data => data.groups[0])));
const WALRUS = rule("WALRUS", withCarefulSkippers(":="));
const EQUALS = rule("EQUALS", withCarefulSkippers("="));
const EXTAT = rule("EXTAT", withLeftSkippers("@"));
const INTEGER = rule("INTEGER", withCarefulSkippers(mapData(/^[0-9]+/, data => BigInt(
    ↪ data.groups.all))));
const INTEGER_BB = rule("INTEGER_BB", withBareboneSkippers(mapData(/^[0-9]+/, data =>
    ↪ BigInt(data.groups.all))));
const FLOAT = rule("FLOAT", withCarefulSkippers(mapData(/^[0-9]+\.[0-9]+/, data => data.
    ↪ groups.all)));
const FLOAT_BB = rule("FLOAT_BB", withBareboneSkippers(mapData(/^[0-9]+\.[0-9]+/, data
    ↪ => data.groups.all)));
const STRING = rule("STRING", withCarefulSkippers(mapData(/^"((?:[^"\\\r\n]|\\.)*)"/,
    ↪ data => data.groups[0])));
const STRING_BB = rule("STRING_BB", withBareboneSkippers(mapData(/^"((?:[^"\\\r\n]|\\.)
    ↪ *)"/,  data => data.groups[0])));
const PLUS = rule("PLUS", withCarefulSkippers("+"));
const MINUS = rule("MINUS", withCarefulSkippers("-"));
const DIV = rule("DIV", withCarefulSkippers("/"));
const AND = rule("AND", withCarefulSkippers("&&"));
const OR = rule("OR", withCarefulSkippers("||"));
const EQ = rule("EQ", withCarefulSkippers("=="));
const NEQ = rule("NEQ", withCarefulSkippers("!="));
const LTE = rule("LTE", withCarefulSkippers("<="));
const GTE = rule("GTE", withCarefulSkippers(">="));
const LT = rule("LT", withCarefulSkippers("<"));
const GT = rule("GT", withCarefulSkippers(">"));

// PARSER RULES

const hardval = declare();
const hardval_bb = declare();
const typeval = declare();
const typeval_atom = declare();
const type_callable = declare();
const map_entry_for = declare();
const map_entry_if = declare();
const map_entry_break = declare();
const map_entry_callable = declare();
const map_entry_semiless = declare();
const constraint_enum = declare();

const symbol_path = rule("symbol_path", mapData(
    multi(
        SYMBOL,
    ),
    (data) => ({
        type: "symbol_path",
        trail: data.map(entry => entry.substring(1)),
    }),
));

const expr_log = rule("expr_log", mapData(
    join(
        KW_LOG,
        LPAREN,
        opt(typeval),
        RPAREN,
    ),
    (data) => ({
        type: "expr_log",
        message: data[2],
    }),
));

const expr_log_d = rule("expr_log_d", mapData(
    join(
        KW_LOG_D,
```

```
        LPAREN,
        typeval,
        RPAREN,
    ),
    (data) => ({
        type: "expr_log_d",
        message: data[2],
    }),
));

const expr_log_dd = rule("expr_log_dd", mapData(
    join(
        KW_LOG_DD,
        LPAREN,
        typeval,
        COMMA,
        or(
            mapData(KW_NULLTERM, () => ({ type: "nullterm" })),
            mapData(typeval, (data) => ({ type: "byte_count", count: data })),
        ),
        RPAREN,
    ),
    (data) => ({
        type: "expr_log_dd",
        message: data[2],
        byte_count: data[4],
    }),
));

hardval.define(rule("hardval", or(
    mapData(
        FLOAT, // float must come before integer as integer is substring of float, due
    ↪ to lack of dedicated lexer.
        (data) => ({
            type: "type_map",
            leaf_type: undefined,
            leaf_hardval: {
                type: "hardval_float",
                value: data,
            },
            call_input_type: undefined,
            call_output_type: undefined,
            sym_inputs: {},
            instructions: [],
        }),
    ),
    mapData(
        INTEGER,
        (data) => ({
            type: "type_map",
            leaf_type: undefined,
            leaf_hardval: {
                type: "hardval_integer",
                value: data,
            },
            call_input_type: undefined,
            call_output_type: undefined,
            sym_inputs: {},
            instructions: [],
        }),
    ),
    mapData(
        STRING,
        (data) => ({
            type: "type_map",
            leaf_type: undefined,
            leaf_hardval: {
                type: "hardval_string",
                value: data,
            },
```

```
            call_input_type: undefined,
            call_output_type: undefined,
            sym_inputs: {},
            instructions: [],
        }),
    ),
    mapData(
        SYMBOL,
        (data) => ({
            type: "type_enum",
            hardsym: data.substring(1),
        }),
    ),
    mapData( // to be honest i'm not sure if variable access should be a hardval, it
    ↪ makes me reconsider whether pointers should also be normalized to maps.
        IDENT,
        (data) => ({
            type: "type_var_access",
            target_name: data,
        }),
    ),
)));

hardval_bb.define(rule("hardval_bb", or(
    mapData(
        FLOAT_BB, // float must come before integer as integer is substring of float,
    ↪ due to lack of dedicated lexer.
        (data) => ({
            type: "type_map",
            leaf_type: undefined,
            leaf_hardval: {
                type: "hardval_float",
                value: data,
            },
            call_input_type: undefined,
            call_output_type: undefined,
            sym_inputs: {},
            instructions: [],
        }),
    ),
    mapData(
        INTEGER_BB,
        (data) => ({
            type: "type_map",
            leaf_type: undefined,
            leaf_hardval: {
                type: "hardval_integer",
                value: data,
            },
            call_input_type: undefined,
            call_output_type: undefined,
            sym_inputs: {},
            instructions: [],
        }),
    ),
    mapData(
        STRING_BB,
        (data) => ({
            type: "type_map",
            leaf_type: undefined,
            leaf_hardval: {
                type: "hardval_string",
                value: data,
            },
            call_input_type: undefined,
            call_output_type: undefined,
            sym_inputs: {},
            instructions: [],
        }),
    ),
```

```
    mapData(
        IDENT_BB,
        (data) => ({
            type: "type_var_access",
            target_name: data,
        }),
    ),
)));

const expr_var_walrus = rule("expr_var_walrus", mapData(
    join(opt(KW_MUT), IDENT, WALRUS, typeval),
    (data) => ({
        type: "expr_var_walrus",
        is_mut: data[0] !== undefined,
        name: data[1],
        typeval: data[3],
    }),
));

const expr_var_assign = rule("expr_var_assign", mapData(
    join(IDENT, EQUALS, typeval),
    (data) => ({
        type: "expr_var_assign",
        name: data[0],
        typeval: data[2],
    }),
));

const map_entry_sym = rule("map_entry_sym", mapData(
    join(SYMBOL, typeval),
    (data) => ({
        type: "map_entry_sym",
        name: data[0],
        typeval: data[1],
    }),
));

const expr25 = rule("expr25", or(
    expr_log_dd,
    expr_log_d,
    expr_log,
    type_callable,
    typeval_atom,
));

const BAREBONES_AHEAD = rule("BAREBONES_AHEAD", /^(?![ \t\n]*[;+\-*\/&|])/);

const BAREBONES_AHEAD_with_debug = (input) => {
    const outcome = BAREBONES_AHEAD(input);

    if (outcome.success) {
        console.log("BAREBONES_AHEAD passed on:", JSON.stringify(input.substring(0, 32))
    ↪ );
    }

    return outcome;
};

const expr27 = rule("expr27", mapData(join(
    expr25,
    opt_multi(or(
        mapData(SYMBOL_BARE, (data) => ({ mode: "sym", data })),
        // no way , I forgot I had written a lookAhead function.
        // this has saved me.
        mapData(join(lookAhead(BAREBONES_AHEAD_with_debug), expr25), (data) => ({ mode:
    ↪ "dynamic", data: data[1] })),
    )),
), (data) => {
    let result = data[0];
```

```
    for (const item of data[1]) {
        if (item.mode === "sym") {
            result = {
                type: "expr_call_with_sym",
                target: result,
                sym: item.data.substring(1),
            };
        } else {
            result = {
                type: "expr_call_with_dynamic",
                target: result,
                call_data: item.data,
            };
        }
    }

    return result;
}));

const expr30 = rule("expr30", mapData( // multiplicative pistol
    join(expr27, opt_multi(join(or(ASTERISK, DIV), expr27))),
    (data) => {
        if (data[1].length === 0) {
            return data[0];
        }

        return ({
            type: "expr_multi",
            ops: [{ op: "*", operand: data[0] }, ...data[1].map(([op, operand]) => ({ op
    ↪ , operand }))],
        });
    },
));

const expr35 = rule("expr35", or( // multiplicative crystal
    mapData(
        // we demand >= 2 asterisk because to distinguish with pointer.
        join(join(or(ASTERISK, DIV), expr27), multi(join(or(ASTERISK, DIV), expr27))),
    ↪ // in unary case, we skip to expr27 to ignore binary case
        (data) => {
            const combined = [data[0], ...data[1]];

            return ({
                type: "expr_multi",
                ops: combined.map(([op, operand]) => ({ op, operand })),
            });
        },
    ),
    expr30,
));

const expr40 = rule("expr40", mapData( // additive pistol
    join(expr35, opt_multi(join(or(PLUS, MINUS), expr35))),
    (data) => {
        if (data[1].length === 0) {
            return data[0];
        }

        return ({
            type: "expr_addit",
            ops: [{ op: "+", operand: data[0] }, ...data[1].map(([op, operand]) => ({ op
    ↪ , operand }))],
        });
    },
));

const expr45 = rule("expr45", or( // additive crystal
    mapData(
        multi(join(or(PLUS, MINUS), expr35)), // in unary case, we skip to expr35 to
    ↪ ignore binary case
```

```
        (data) => {
            if (data.length === 1) {
                return data[0][1];
            }

            return ({
                type: "expr_addit",
                ops: data.map(([op, operand]) => ({ op, operand })),
            });
        },
    ),
    expr40,
));

const expr47 = rule("expr47", mapData(
    join(expr45, opt(join(or(EQ, NEQ, LTE, GTE, LT, GT), expr45))),
    (data) => {
        if (data[1] === undefined) {
            return data[0];
        }

        return ({
            type: "expr_compare",
            operator: data[1][0],
            operand_a: data[0],
            operand_b: data[1][1],
        });
    },
));

const expr50 = rule("expr50", mapData( // logical AND pistol
    join(expr47, opt_multi(join(AND, expr47))),
    (data) => {
        if (data[1].length === 0) {
            return data[0];
        }

        return ({
            type: "expr_logical_and",
            ops: [{ operand: data[0] }, ...data[1].map(([_op, operand]) => ({ operand })
    ↪ )],
        });
    },
));

const expr55 = rule("expr55", or( // logical AND crystal
    mapData(
        multi(join(AND, expr47)), // in unary case, we skip to expr47 to ignore binary
    ↪ case
        (data) => {
            if (data.length === 1) {
                return data[0][1];
            }

            return ({
                type: "expr_logical_and",
                ops: data.map(([_op, operand]) => ({ operand })),
            });
        },
    ),
    expr50,
));

const expr60 = rule("expr60", mapData( // logical OR pistol
    join(expr55, opt_multi(join(OR, expr55))),
    (data) => {
        if (data[1].length === 0) {
            return data[0];
        }
```

```
        return ({
            type: "expr_logical_or",
            ops: [{ operand: data[0] }, ...data[1].map(([_op, operand]) => ({ operand })
    ↪ )],
        });
    },
));

const expr65 = rule("expr65", or( // logical OR crystal
    mapData(
        multi(join(OR, expr55)), // in unary case, we skip to expr55 to ignore binary
    ↪ case
        (data) => {
            if (data.length === 1) {
                return data[0][1];
            }

            return ({
                type: "expr_logical_or",
                ops: data.map(([_op, operand]) => ({ operand })),
            });
        },
    ),
    expr60,
));

const expr75 = rule("expr75", expr65);

const expr85 = rule("expr85", or(
    expr_var_walrus,
    expr_var_assign,
    expr75
));

const expr = rule("expr", or(
    expr85,
));

const map_entry_expr = rule("map_entry_expr", mapData(
    typeval,
    (data) => ({
        type: "map_entry_expr",
        expr: data,
    }),
));

const map_entry = rule("map_entry", or(
    mapData(map_entry_sym, (data) => ({
        type: "instruction",
        data,
    })),
    mapData(map_entry_break, (data) => ({
        type: "instruction",
        data,
    })),
    mapData(map_entry_expr, (data) => ({
        type: "instruction",
        data,
    })),
));

const constraint_map_braced_multiline = rule("constraint_map_braced_multiline", mapData(
    join(
        LBRACE,
        MANDATORY_NEWLINE,
        opt_multi(
            or(
                mapData(
                    join(map_entry, SEMI),
                    (data) => data[0],
```

```
                ),
                map_entry_semiless,
            ),
        ),
        RBRACE,
    ),
    (data) => {
        const entries = data[2];

        const callable = entries.find(entry => entry.type === "callable");

        const instructions = (entries
            .filter(entry => entry.type === "instruction")
            .map(entry => entry.data)
        );

        const sym_inputs = Object.fromEntries(
            instructions
                .filter(instr => instr.type === "map_entry_sym")
                .map(instr => [instr.name, {}])
        );

        return {
            type: "type_map",
            call_input_identifier: callable?.data.callable.call_input_identifier,
            call_input_type: callable?.data.callable.call_input_type,
            call_output_type: callable?.data.callable.call_output_type,
            sym_inputs,
            instructions,
        };
    },
));

map_entry_for.define(rule("map_entry_for", mapData(
    join(KW_FOR, constraint_map_braced_multiline),
    (data) => ({
        type: "instruction_for",
        body: data[1],
    }),
)));

map_entry_if.define(rule("map_entry_if", mapData(
    join(
        KW_IF,
        LPAREN,
        typeval,
        RPAREN,
        constraint_map_braced_multiline,
        opt_multi(mapData(
            join(
                KW_ELSE,
                KW_IF,
                LPAREN,
                typeval,
                RPAREN,
                constraint_map_braced_multiline
            ),
            (data) => ({
                condition: data[3],
                body: data[5],
            }),
        )),
        opt(mapData(
            join(
                KW_ELSE,
                constraint_map_braced_multiline,
            ),
            (data) => data[1],
        )),
    ),
```

```
    (data) => ({
        type: "instruction_if",
        branches: [
            { condition: data[2], body: data[4] },
            ...data[5].map((branch) => ({ condition: branch.condition, body: branch.body
    ↪   })),
            ...(data[6] !== undefined ? [{ condition: undefined, body: data[6] }] : []),
        ],
    }),
)));

map_entry_break.define(rule("map_entry_break", mapData(
    KW_BREAK,
    () => ({
        type: "instruction_break",
    }),
)));

map_entry_callable.define(rule("map_entry_callable", mapData(
    type_callable,
    (data) => ({
        type: "map_entry_callable",
        callable: data,
    }),
)));

map_entry_semiless.define(rule("map_entry_semiless", or(
    mapData(map_entry_for, (data) => ({
        type: "instruction",
        data,
    })),
    mapData(map_entry_if, (data) => ({
        type: "instruction",
        data,
    })),
    mapData(map_entry_callable, (data) => ({
        type: "callable",
        data,
    })),
)));

const constraint_map_braced_multiline_bb = rule("constraint_map_braced_multiline_bb",
    ↪ mapData(
    join(
        LBRACE_BB,
        MANDATORY_NEWLINE,
        opt_multi(
            join(
                map_entry,
                SEMI,
            ),
        ),
        RBRACE,
    ),
    (data) => {
        const entries = data[2].map(entry => entry[0]);

        const instructions = (entries
            .filter(entry => entry.type === "instruction")
            .map(entry => entry.data)
        );

        const sym_inputs = Object.fromEntries(
            instructions
                .filter(instr => instr.type === "map_entry_sym")
                .map(instr => [instr.name, {}])
        );

        return {
            type: "type_map",
```

```
            sym_inputs,
            instructions,
        }
    },
));

const constraint_map_braced_singleline = rule("constraint_map_braced_singleline",
    ↪ mapData(
    join(
        LBRACE,
        opt(
            join(
                map_entry,
                opt_multi(join(COMMA, map_entry)),
            ),
        ),
        RBRACE,
    ),
    (data) => {
        const entries = [];

        if (data[1] !== undefined) {
            entries.push(data[1][0]);

            for (const entry of data[1][1]) {
                entries.push(entry[1]);
            }
        }

        const instructions = (entries
            .filter(entry => entry.type === "instruction")
            .map(entry => entry.data)
        );

        const sym_inputs = Object.fromEntries(
            instructions
                .filter(instr => instr.type === "map_entry_sym")
                .map(instr => [instr.name, {}])
        );

        return {
            type: "type_map",
            sym_inputs,
            instructions,
        };
    },
));

const constraint_map_braced_singleline_bb = rule("constraint_map_braced_singleline_bb",
    ↪ mapData(
    join(
        LBRACE_BB,
        opt(
            join(
                map_entry,
                opt_multi(join(COMMA, map_entry)),
            ),
        ),
        RBRACE,
    ),
    (data) => {
        const entries = [];

        if (data[1] !== undefined) {
            entries.push(data[1][0]);

            for (const entry of data[1][1]) {
                entries.push(entry[1]);
            }
        }
```

```
        const instructions = (entries
            .filter(entry => entry.type === "instruction")
            .map(entry => entry.data)
        );

        const sym_inputs = Object.fromEntries(
            instructions
                .filter(instr => instr.type === "map_entry_sym")
                .map(instr => [instr.name, {}])
        );

        return {
            type: "type_map",
            sym_inputs,
            instructions,
        };
    },
));

const constraint_map_braced = rule("constraint_map_braced", or(
    constraint_map_braced_multiline,
    constraint_map_braced_singleline,
));

const constraint_map_braced_bb = rule("constraint_map_braced_bb", or(
    constraint_map_braced_multiline_bb,
    constraint_map_braced_singleline_bb,
));

const constraint_enum_braced_multiline = rule("constraint_enum_braced_multiline",
    ↪ mapData(
    join(
        KW_ENUM,
        LBRACE,
        MANDATORY_NEWLINE,
        opt_multi(
            mapData(
                join(SYMBOL, SEMI),
                (data) => data[0],
            ),
        ),
        RBRACE,
    ),
    (data) => ({
        type: "type_enum",
        syms: data[3].map(sym => sym.substring(1)),
    }),
));

const constraint_enum_braced_singleline = rule("constraint_enum_braced_singleline",
    ↪ mapData(
    join(
        KW_ENUM,
        LBRACE,
        opt(
            join(
                SYMBOL,
                opt_multi(join(COMMA, SYMBOL)),
            ),
        ),
        RBRACE,
    ),
    (data) => {
        const syms = [];

        if (data[2] !== undefined) {
            syms.push(data[2][0].substring(1));

            for (const [, sym] of data[2][1]) {
```

```
                syms.push(sym.substring(1));
            }
        }

        return {
            type: "type_enum",
            syms,
        };
    },
));

const constraint_enum_braced = rule("constraint_enum_braced", or(
    constraint_enum_braced_multiline,
    constraint_enum_braced_singleline,
));

const constraint_map_tupled = rule("constraint_map_tupled", mapData(
    join(
        LPAREN,
        RPAREN,
    ),
    (data) => ({
        type: "type_map",
    }),
));

const constraint_map_tupled_bb = rule("constraint_map_tupled_bb", mapData(
    join(
        LPAREN_BB,
        RPAREN
    ),
    (data) => ({
        type: "type_map",
    }),
));

const constraint_map = rule("constraint_map", or(
    constraint_map_braced,
    constraint_map_tupled,
));

const constraint_map_bb = rule("constraint_map_bb", or(
    constraint_map_braced_bb,
    constraint_map_tupled_bb,
));

const constraint_integer = rule("constraint_integer", mapData(
    INTEGER,
    (data) => ({
        type: "constraint",
        mode: "constraint_integer",
        value: data,
    }),
));

const constraint_float = rule("constraint_float", mapData(
    FLOAT,
    (data) => ({
        type: "constraint",
        mode: "constraint_float",
        value: data,
    }),
));

const constraint_semiless = rule("constraint_semiless", or(
    constraint_map,
));

const constraint_semiful = rule("constraint_semiful", or(
    constraint_float,
```

```
    constraint_integer,
));

const constraint_maybesemi = rule("constraint_maybesemi", or(
    constraint_semiless,
    mapData(
        join(constraint_semiful, SEMI),
        (data) => data[0],
    ),
));

const forwarding = rule("forwarding", mapData(
    join(
        KW_FORWARDING,
        LBRACE,
        opt_multi(
            or(
                mapData(
                    // for now, only makes sense to reset a path in the outer root.
                    join(KW_RESET, or(PATH_OUTER_ROOT, OUTER_ROOT), SEMI),
                    (data) => ({ type: "reset", source: data[0] }),
                ),
                mapData(
                    join(PATH_OUTER_ROOT, PATH, SEMI),
                    (data) => ({ type: "redirect", source: data[0], destination: data[1]
    ↪  })
                ),
            )
        ),
        RBRACE,
    ),
    (data) => ({
        type: "forwarding",
        entries: data[2],
    }),
));

const top_forwarding = rule("top_forwarding", mapData(
    forwarding,
    (data) => ({
        ...data,
        type: "top_forwarding",
    }),
));

const type_reference = declare();
// todo: probably just declare everything at the top to avoid stress.

type_reference.define(rule("type_reference", or( // this rule is outdated and will be
    ↪ destroyed in a future update.
    mapData(
        KW_MAP,
        (_data) => ({
            type: "type_reference",
            mode: "anon_map",
            leaf_type: undefined,
            call_input_type: undefined,
            call_output_type: undefined,
        }),
    ),
    mapData(
        TYPE_IDENT,
        (data) => ({
            type: "type_reference",
            mode: "type_ident",
            trail: data.split("::"),
        }),
    ),
    mapData(
        join(ASTERISK, type_reference),
```

```
        (data) => ({
            type: "type_reference",
            mode: "pointer",
            uly: data[1],
        }),
    ),
)));

const type_named = rule("type_named", mapData(
    TYPE_IDENT,
    (data) => ({
        type: "type_named",
        trail: data,
    }),
));

const type_ptr_named = rule("type_ptr_named", mapData(
    join(multi(ASTERISK), type_named),
    (data) => {
        let result = data[1];

        for (let i = data[0].length - 1; i >= 0; i--) {
            result = {
                type: "type_ptr",
                target: result,
            };
        }

        return result;
    },
));

constraint_enum.define(rule("constraint_enum", or(
    constraint_enum_braced_multiline,
    constraint_enum_braced_singleline,
)));

const type_first = rule("type_first", or( // cannot include constraint_enum here due to
    ↪ precedence reasons
    type_ptr_named,
    type_named,
));

typeval_atom.define(rule("typeval_atom", or(
    mapData(
        join(type_first, hardval_bb),
        (data) => ({
            type: "type_constrained",
            constraints: [
                data[0],
                data[1],
            ],
        }),
    ),
    mapData(
        join(type_first, constraint_map_bb),
        (data) => ({
            type: "type_constrained",
            constraints: [
                data[0],
                data[1],
            ],
        }),
    ),
    type_first,
    constraint_enum,
    hardval,
    constraint_map,
    // mapData( // have no idea if grammar can accept this consistently
    //  join(LPAREN, typeval, RPAREN),
```

```
    //  (data) => data[1],
    // ),
)));

type_callable.define(rule("type_callable", mapData(
    join(opt(IDENT), typeval_atom, ARROW, typeval_atom),
    (data) => ({
        type: "type_map",
        leaf_type: undefined,
        call_input_identifier: data[0],
        call_input_type: data[1],
        call_output_type: data[3],
        sym_inputs: {},
    }),
)));

typeval.define(rule("typeval", or(
    expr,
)));

const type_map_body_braced = rule("type_map_body_braced", mapData(
    join(
        LBRACE,
        RBRACE,
    ),
    (_data) => ({
        type: "type_map_body",
    }),
));

const type_map_body_tupled = rule("type_map_body_tupled", mapData(
    join(
        LPAREN,
        RPAREN,
    ),
    (_data) => ({
        type: "type_map_body",
    }),
));

const type_map_body = rule("type_map_body", or(
    type_map_body_braced,
    type_map_body_tupled,
));

const type_map_callable = rule("type_map_callable", mapData( // outdated not using
    ↪ anymore probably.
    join(
        type_map_body,
        ARROW,
        opt(type_reference),
        // note: no need for third part for mixing type reference and map type, as map
    ↪ type can incorporate type reference leaf in future.
        constraint_map,
    ),
    (data) => ({
        type: "type_map_callable",
        call_input_type: data[0],
        call_output_type: data[2],
        call_output_constraint: data[3],
    }),
));

const top_type = rule("top_type", mapData(
    join(
        KW_TYPE,
        TYPE_IDENT,
        typeval,
        SEMI,
    ),
```

```
    (data) => ({
        type: "type",
        trail: data[1],
        definition: data[2],
    }),
));

// a value is just a stricter version of a type, but it's still a type from the compiler
    ↪ 's perspective. (value constraint perhaps i'll call it constraint instead of value
    ↪ ).

const top_types = rule("top_types", mapData(
    join(
        KW_TYPES,
        TYPES_IDENT,
        or(
            mapData(
                join(TYPES_IDENT, SEMI),
                (data) => ({
                    type: "types_reference",
                    mode: "types_ident",
                    trail: data[0],
                }),
            ),
            mapData(
                join(KW_IMPORT, PATH, SEMI),
                (data) => ({
                    type: "types_reference",
                    mode: "anon_import",
                    path: data[1],
                    forwarding: undefined,
                }),
            ),
            mapData(
                join(KW_IMPORT, PATH, forwarding),
                (data) => ({
                    type: "types_reference",
                    mode: "anon_import",
                    path: data[1],
                    forwarding: data[2],
                }),
            ),
        ),
    ),
    (data) => ({
        type: "top_types",
        trail: data[1],
        definition: data[2],
    }),
));

const top_mod = rule("top_mod", mapData(
    join(
        KW_MOD,
        MOD_IDENT,
        or(
            mapData(
                join(KW_IMPORT, PATH, SEMI),
                (data) => ({
                    type: "mod_reference",
                    mode: "anon_import",
                    path: data[1],
                    forwarding: undefined,
                }),
            ),
            mapData(
                join(KW_IMPORT, PATH, forwarding),
                (data) => ({
                    type: "mod_reference",
                    mode: "anon_import",
```

```
                    path: data[1],
                    forwarding: data[2],
                }),
            ),
        ),
    ),
    (data) => ({
        type: "top_mod",
        trail: data[1],
        definition: data[2],
    }),
));

const extension_case_sym = rule("extension_case_sym", mapData(
    join(SYMBOL_GRAND, typeval, SEMI),
    (data) => ({
        type: "extension_case_sym",
        trail: data[0],
        typeval: data[1],
    }),
));

const extension_case = rule("extension_case", or(
    extension_case_sym,
));

const top_extension = rule("top_extension", mapData(
    join(
        EXTAT,
        TYPE_IDENT,
        extension_case,
    ),
    (data) => ({
        type: "top_extension",
        target_type: data[1],
        case: data[2],
    }),
));

const top_create = rule("top_create", mapData(
    join(KW_CREATE, type_callable),
    (data) => ({
        type: "top_create",
        description: data[1],
    }),
));

const top_entry = rule("top_entry", or(
    top_forwarding,
    top_type,
    top_types,
    top_mod,
    top_extension,
    top_create,
    SEMI,
));

// i wonder if values and maps could be combined?
// but for now, value_map will just be optional after type.

const file_root = rule("file_root", mapData(join(opt_multi(top_entry), SKIPPERS), data
    ↪ => data[0]));

// SEMANTIC ANALYSIS

// COMPILER

// OUTER INTERFACE

// class ModuleLoader {
```

```
//  constructor() {

//  }

//  obtain_module_files(path) {

//  }


// }

// create_sources_scope([
//  ["//"], // reset
//  ["/", parent_scope, "/"],
//  ["/banana", parent_scope, "/prefix/on/parent/scope/"],
//  ["//octopus", parent_scope, "//prefix/on/parent/scope/"],

// ]);

const make_structured = (entries) => {
    const structured = {
        structure: entries,
        forwarding: [],
        mod_list: [],
        types_list: [],
        create: undefined,
    };

    for (const entry of entries) {
        if (entry.type === "top_forwarding") {
            structured.forwarding = [
                ...structured.forwarding,
                ...entry.entries,
            ];
        }

        if (entry.type === "top_mod") {
            structured.mod_list.push({
                ...entry,
            });
        }

        if (entry.type === "top_types") {
            structured.types_list.push({
                ...entry,
            });
        }

        if (entry.type === "top_create") {
            if (structured.create !== undefined) {
                throw_error(error_internal("Only one create block permitted per module
    ↪ ."));
            }

            structured.create = entry;
        }
    }

    return structured;
};

const augment = async (data, create_source_scope) => {
    console.log("scop", data.source_scopes, data);
    const module_source_scope = create_source_scope([
        // ["//", data.source_scopes.external, "//"],
        // Todo: i need to add the above back in. It would normally steal this from
    ↪ parent module, or if root then match internal scope. So we need to just pass
    ↪ relevant data through the system to know what mode we are in.
        ["//", data.source_scopes.internal, "/"], // temp solution
        ["/", data.source_scopes.internal, "/"],
```

```
    ]);

    const forwarding_source_scope = create_source_scope(data.forwarding.map(entry => {
        if (entry.type === "reset") {
            return [entry.source];
        } else if (entry.type === "redirect") {
            return [entry.source, module_source_scope, entry.destination];
        }

        console.warn("Entry:", entry);
        throw_error(error_internal("Assertion error: Unhandled case."));
    }));

    const result = {
        ...data,
        source_scopes: {
            ...data.source_scopes ?? [],
            module: module_source_scope,
            forwarding: forwarding_source_scope,
        },
    };

    for (const mod_entry of result.mod_list) {
        if (mod_entry.definition.mode === "anon_import") {
            mod_entry.definition.uuid = await module_source_scope.resolve(mod_entry.
    ↪ definition.path);
        }
    }

    for (const types_entry of result.types_list) {
        if (types_entry.definition.mode === "anon_import") {
            types_entry.definition.uuid = await module_source_scope.resolve(types_entry.
    ↪ definition.path);
        }
    }

    return result;
};

export const process = async (config) => {
    const path = config.src;
    const actual_path = node__path.resolve(path);
    console.log("[FE] Processing module located at path:", actual_path);

    const dir_node_translator = create_dir_node_translator({
        fs,
        path: node__path,
        randomUUID,
    });

    const root_uuid = await dir_node_translator.add(actual_path);

    const create_source_scope = (redirects) => orig_create_source_scope({
        dir_node_translator,
    }, redirects);

    const file_system_source_scope = create_fs_source_scope({
        fs,
        path: node__path,
        dir_node_translator,
    }, actual_path);

    const internal_source_scope = create_source_scope([
        ["//", file_system_source_scope, "./"], // todo: this will be made more
    ↪ exhaustive at a later stage.
        ["/", file_system_source_scope, "./"],
    ]);

    const source_extractor = create_source_extractor({
        fs,
```

```
        dir_node_translator,
    });

    const module_uuid = await internal_source_scope.resolve("/");
    const result = await source_extractor.extract(module_uuid);

    console.log(result);

    const parse_results = [];

    for (const file of result.files) {
        console.log("Parsing file", file);

        const text = (await fs.readFile(node__path.join(actual_path, file), "utf-8")).
    ↪ replaceAll("\r\n", "\n");
        const result = file_root(text);

        if (false
            || result.success === false
            || result.input !== ""
        ) {
            console.error(result);
            console.log("<start of trace>");
            // trace_print(console.log);
            console.log("<end of trace>");

            const json_path = node__path.join(actual_path, "frontend.trace.json");
            const text_path = node__path.join(actual_path, "frontend.trace.txt");

            // await trace_dump(async (content) => await fs.writeFile(json_path, content
    ↪ , "utf-8"));
            // await trace_print(async (line) => await fs.appendFile(text_path, line +
    ↪ "\n", "utf-8"))

            throw new Error(`Failed to parse.`);
        }

        parse_results.push(result.data);
    }

    console.log("Augmenting module data...");

    const combined = parse_results.flat();

    const structured = make_structured(combined);

    structured.module_path = actual_path;
    structured.module_uuid = root_uuid;

    structured.source_scopes = {
        internal: internal_source_scope,
    };

    const final = await augment(structured, create_source_scope);

    return {
        parse_output: final,
        dir_node_translations: await dir_node_translator.dump_translations(),
        module_path: actual_path,
    };
};
```

compiler/package.json

```
{
    "type": "module",
    "dependencies": {
```

```
        "commander": "^14.0.2"
    }
}
```

compiler/test/a.ec

```
forwarding {
    reset //;
}

forwarding {
    reset //;
    reset //foo/bar/baz;
    //banana/slop /foo/bop;
}

type foo::Bar f32;

; types foo::bar foo::bar;
type Banana f64;
type foo::bar::Baz Banana;
type foo::bar::Bink foo::Bar;
type foo::Coconut {};

; types egg::plant foo::bar;

; types banana import /foo/bar;

; types orange import /foo/bar forwarding {
;   reset //;
;   reset //foo/bar/baz;
;   //banana/slop /wanana/bop;
; }

; mod eggplant import //foo/plant;

; mod eggplant import //foo/plant forwarding {
;   //pear /foo;
; }

; todo why is eggplant2 not working?

; "Compiler" will be in-built type to make life easier

type foo::TestingSomePointers u8;

type TestingSomePointers u8;

type TesingPointers *u8;

type TestingPointersMore **u8;

type String *u8;

; types compiler import //std/compiler;

; @Mod:ct compiler::CompTime;

; create(:ct compiler::CompTime, :argc usize, :argv **u8) -> {
;   :ct ct;
; }

type MyService {};

@MyService:name "My Fantastic Service";

@MyService:print_foo input {
```

```
    :foo u64;
} -> {
    log_d(input:foo);
};

@Mod:name "My Module";

@Mod:do_something {} -> {
    log_d("Doing something...");
};

create() -> { ; this comment works
    test := 10;
    banana := u64 10;
    str := "Hello, Universe!";
    str2 := *u8 "Hello, Kingdom!";
    str3 := str2;
    str4 := String "Test";
    mut str5 := String "Mutable String";

    object := {
        name := "David";
        age := 23;
        :octopus "Test";
        :another_variable 23;
        :name2 name;
        name3 := name2;
        :nested {
            :octopus "This would be really cool if it worked.";
        };
        :star_sign enum {
            :aries;
            :taurus;
            :capricorn;
        } :capricorn;
    };

    foo := object:name2;
    bar := object:nested:octopus;

    log("Hello, World!");
    log();
    log("This log statement can now handle strings coming from variables.");
    log(str3);
    log(str4);
    log(foo);
    log(bar);
    log(access_during_assign := "This is really cool");

    log("Singletonish optimization test:");
    log_d(object:star_sign); // should be a constant
    log_d(object:another_variable); // should be a constant

    log("Dynamic sym call:");
    sym_of_interest := :star_sign;
    log_d(object sym_of_interest);

    log("Str 5 is:");
    log(str5);
    str5 = "New mutated value! I hope this formats well. abc";
    log("Str 5 is now:");
    log(str5);

    ; for {
    ;   log("Forever.");
    ; }

    advanced_calculation :=
        + 50
        + 70
```

```
    - 30
;

log_d(advanced_calculation);
log_d(156);
log_d(str5);
log("null-term");
log_dd(str5, null-term);
log("byte-count");
log_dd(str5, 20);
log("byte-count");
log_dd(str5, 5);

boolean := :true;
type_ := :foo_bar_baz;

log("Enum dump:");
log_d(boolean);
log_d(type_);

new_boolean := enum {
    :false;
    :true;
} :true;

log_d(new_boolean);

log("Boolean values:");

mut actual_boolean := bool :true;

log_d(actual_boolean);

actual_boolean = bool :false;

log_d(actual_boolean);

logical_test_a :=
    || actual_boolean
    || bool :true
;

logical_test_b :=
    && actual_boolean
    && bool :true
;

logical_test_c :=
    && bool :true
    && bool :true
;

log("Logical tests:");

log_d(logical_test_a);
log_d(logical_test_b);
log_d(logical_test_c);

if (bool :true) {
    log("Scenario 1A");
} else if (bool :false) {
    log("Scenario 1B");
} else {
    log("Scenario 1C");
}


if (bool :false) {
    log("Scenario 2A");
} else if (bool :true) {
```

```
        log("Scenario 2B");
    } else {
        log("Scenario 2C");
    }

    if (bool :false) {
        log("Scenario 3A");
    } else if (bool :false) {
        log("Scenario 3B");
    } else {
        log("Scenario 3C");
    }

    mut i := u64 0;

    for {
        log("Hello from a loop.");

        i = i + 1;

        log_d(i);

        if (i >= 10) {
            log("breaking...");
            break;
        }
    }

    actually_callable := {
        input {
            :foo u64;
        } -> {
            log_d(input:foo);
        }
    };

    actually_callable {
        :foo 123;
    };

    syntactically_shorter := input {
        :foo u64;
    } -> {
        log_d(input:foo);
    };

    syntactically_shorter {
        :foo 456;
    };

    syntactically_shorter {
        :foo 567;
    };

    x := MyService;

    log(x:name);

    x:print_foo {
        :foo 678;
    };

    log(mod:name);

    mod:do_something {};
}
```

compiler/test/b.ec

```
forwarding {
    //banana/slop /foo/bop;
    //banana/vegetable /foo/gravy;
}

type Bar {};
type Car {};

@Bar:test:blah27 f32;

type blah::Wah {};

@Bar:foo blah::Wah;

type NewType f32 {}
; type NewType map {}

type NewTypeAgain f32 5.0;
; type NewTypeAgain map 5;

type AnotherNewType f32 5.0;
type AnotherNewTypeAlright 5;

type Blah 5.5;

@Car:why_is_this_disappearing i32;
@Car:testing f32 5.0;
; @Car:testing 5;
```

compiler/test/foo/bar/placeholder.txt

compiler/test/foo/bop/placeholder.txt

compiler/test/foo/gravy/placeholder.txt

compiler/test/foo/plant/placeholder.txt

compiler/test2/main.ec

```
; test comment

create() -> {
    ; u64
}
```

extension/LICENSE.txt

```
no license given
```

extension/README.md

```
# Extension

This directory houses the VS Code extension for my language, which provides syntax
    ↪ highlighting to the programmer.

It is unlikely that any additional extension features would be worked on, as that would
    ↪ be out of the scope of this project.

## Setup

- Ensure `vsce` command is set up on your system.

## Build

- `vsce package`
- Alternatively: `pnpx @vscode/vsce package`

## Install

- Within VS Code, right click the generated `.vsix` file in the explorer tree.
- Select "Install Extension VSIX".
- You may need to reload your window to see the changes take effect.

## Structure

- The `language-configuration.json` file contains some fundamental syntax rules that VS
    ↪ Code needs to know about.
- The `syntaxes/essencec.tmLanguage.json` file contains the meat up the syntax and also
    ↪ assigns appropriate colors to the syntax.

## Autogenerated Files

The `yo code` command was used to generate the boilerplate for this directory.

Thus, some files were auto-generated by the CLI.

Some other files were scaffolded by the CLI but have since been modified significantly.
```

extension/language-configuration.json

```
{
    "comments": {
        // symbol used for single line comment. Remove this entry if your language does
    ↪ not support line comments
        "lineComment": ";",
    },
    // symbols used as brackets
    "brackets": [
        [
            "{",
            "}"
        ],
        [
            "[",
            "]"
        ],
        [
            "(",
            ")"
        ]
```

```
    ],
    // symbols that are auto closed when typing
    "autoClosingPairs": [
        [
            "{",
            "}"
        ],
        [
            "[",
            "]"
        ],
        [
            "(",
            ")"
        ],
        [
            "\"",
            "\""
        ],
        [
            "'",
            "'"
        ]
    ],
    // symbols that can be used to surround a selection
    "surroundingPairs": [
        [
            "{",
            "}"
        ],
        [
            "[",
            "]"
        ],
        [
            "(",
            ")"
        ],
        [
            "\"",
            "\""
        ],
        [
            "'",
            "'"
        ]
    ]
}
```

extension/package.json

```
{
  "publisher": "blabidy",
  "name": "essence-c",
  "displayName": "essence-c",
  "description": "",
  "version": "0.0.1",
  "engines": {
    "vscode": "^1.96.2"
  },
  "categories": [
    "Programming Languages"
  ],
  "contributes": {
    "languages": [
      {
        "id": "essencec",
```

```
        "aliases": [
          "Essence C",
          "essencec"
        ],
        "extensions": [
          ".ec",
          ".eh"
        ],
        "configuration": "./language-configuration.json"
      }
    ],
    "grammars": [
      {
        "language": "essencec",
        "scopeName": "source.essencec",
        "path": "./syntaxes/essencec.tmLanguage.json"
      }
    ]
  },
  "repository": "https://google.com/"
}
```

extension/syntaxes/essencec.tmLanguage.json

```
{
    "$schema": "https://raw.githubusercontent.com/martinring/tmlanguage/master/
    ↪ tmlanguage.json",
    "name": "Essence C",
    "patterns": [
        {
            "include": "#keywords"
        },
        {
            "include": "#strings"
        },
        {
            "name": "comment.line.content.essencec",
            "match": "^\\s*;(.*)"
        },
        {
            "match": "(;)(.*)",
            "captures": {
                "1": {
                    "name": "punctuation.separator.delimiter.essencec"
                },
                "2": {
                    "name": "comment.line.content.essencec"
                }
            }
        },
        {
            "name": "entity.name.qualified.custom",
            "begin": "\\b([a-z_][a-z0-9_]*)(::)",
            "beginCaptures": {
                "1": {
                    "name": "entity.name.namespace.component.custom"
                },
                "2": {
                    "name": "punctuation.separator.namespace.custom"
                }
            },
            "end": "(?!::[A-Za-z_][a-zA-Z0-9_]*)",
            "patterns": [
                {
                    "match": "[a-z][a-zA-Z0-9_]*",
                    "name": "entity.name.namespace.component.custom"
                },
```

```
            {
                "match": "[A-Z][a-zA-Z0-9_]*",
                "name": "entity.name.namespace.component.custom"
            },
            {
                "match": "::",
                "name": "punctuation.separator.namespace.custom"
            }
        ]
    },
    {
        "name": "storage.type.object.array.java",
        "match": "[A-Z]([a-z0-9_])*"
    },
    {
        "name": "storage.type.object.array.java",
        "match": "\\b(i[1-9][0-9]{0,4}|u[1-9][0-9]{0,4}|f16|f32|f64|f128|bool)\\b"
    },
    {
        "name": "entity.name.tag",
        "match": "\\b(this|mod)\\b"
    },
    {
        "name": "entity.name.function",
        "match": ":[a-z0-9_]+"
    },
    {
        "name": "entity.name.function",
        "match": ":\\s"
    },
    {
        "begin": "(\\/\\/)(([a-z0-9_\\.])+)",
        "beginCaptures": {
            "1": {
                "name": "keyword.control"
            },
            "2": {
                "name": "entity.name.function"
            }
        },
        "end": "(?![a-z0-9_\\/\\.])",
        "patterns": [
            {
                "name": "entity.name.function",
                "match": "[a-zA-Z0-9_\\.]+"
            },
            {
                "name": "keyword.control",
                "match": "\\/"
            }
        ]
    },
    {
        "begin": "(\\/)(([a-z0-9_\\.])+)",
        "beginCaptures": {
            "1": {
                "name": "storage.type.object.array.java"
            },
            "2": {
                "name": "entity.name.function"
            }
        },
        "end": "(?![a-z0-9_\\/\\.])",
        "patterns": [
            {
                "name": "entity.name.function",
                "match": "[a-zA-Z0-9_\\.]+"
            },
            {
                "name": "storage.type.object.array.java",
```

```
                    "match": "\\/"
                }
            ]
        },
        {
            "name": "keyword.control",
            "match": "\\/\\/"
        },
        {
            "name": "entity.name.tag",
            "match": "@"
        }
    ],
    "repository": {
        "keywords": {
            "patterns": [
                {
                    "name": "keyword.control.essencec",
                    "match": "\\b(in|if|else|for|return|import|forwarding|create|break)
    ↪ \\b"
                },
                {
                    "name": "keyword.modifier.essencec",
                    "match": "\\b(instantiate|reset|types|type|pub|map|enum|mut|null\\-
    ↪ term)\\b"
                }
            ]
        },
        "strings": {
            "name": "string.quoted.double.essencec",
            "begin": "\"",
            "end": "\"",
            "patterns": [
                {
                    "name": "constant.character.escape.essencec",
                    "match": "\\\\."
                }
            ]
        }
    },
    "scopeName": "source.essencec"
}
```

shell.nix

```
# As I have recently moved from Windows to NixOS after my laptop went kapoosh, this file
    ↪  sets up the necessary environment for development.

{ pkgs ? import <nixpkgs> {} }:

(pkgs.buildFHSEnv {
  name = "dev-environment";
  targetPkgs = pkgs: (with pkgs; [
    nodejs_20
    nodePackages.pnpm
    bashInteractive
  ]);

  runScript = "bash";
}).env

# We launch this environment by typing ‘nix-shell shell.nix‘.

# Note that it is my understanding that this setup mirrors the home directory of the
    ↪ system and the environment, so it pollutes the home directory.
# Not enough of a concern for me at this moment.
```

trace_explorer/README.md

```
# Trace Explorer

This useful web tool allows for debugging parser traces generated by the frontend, and
    ↪ outputted to ‘frontend.trace.json‘ file. So if something does not parse correctly,
    ↪  it can be tackled using this tool, to see where it went wrong.

This webapp uses my SolidStart template as detailed below, which I use in all my web
    ↪ applications.

The template can be found [here](https://github.com/davidcallanan/solidstart-template).

## Minimal SolidStart Template

Get started with this SolidStart template:

‘‘‘
npx degit davidcallanan/solidstart-template app
‘‘‘

Run ‘pnpm i‘ and ‘pnpm dev‘ to start the development server.

Included in this stack is:

 - Tailwind
 - Tanstack Router
 - TRPC
```

trace_explorer/app.config.ts

```
import { defineConfig } from "@solidjs/start/config";
import tailwindcss from "@tailwindcss/vite";
import { TanStackRouterVite } from "@tanstack/router-plugin/vite";

export default defineConfig({
    vite: {
        plugins: [
            tailwindcss(),
            TanStackRouterVite({
                target: "solid",
            }),
        ],
    },
});
```

trace_explorer/package.json

```
{
    "name": "with-tanstack-router",
    "type": "module",
    "scripts": {
        "dev": "vinxi dev",
        "build": "vinxi build",
        "start": "vinxi start",
        "version": "vinxi version"
    },
    "dependencies": {
        "@solidjs/meta": "^0.29.4",
        "@solidjs/router": "^0.15.0",
        "@solidjs/start": "^1.1.2",
        "@tanstack/router-plugin": "^1.112.0",
        "@tanstack/solid-router": "^1.114.1",
        "@tanstack/solid-router-devtools": "^1.114.1",
```

```
        "@trpc/client": "^10.45.2",
        "@trpc/server": "^10.45.2",
        "@typeschema/valibot": "^0.13.4",
        "clsx": "^2.1.1",
        "solid-js": "^1.9.5",
        "unsuspended-promise": "^1.0.1",
        "valibot": "^0.29.0",
        "vinxi": "^0.5.7"
    },
    "devDependencies": {
        "@tailwindcss/vite": "^4.1.11",
        "tailwindcss": "^4.1.11"
    },
    "engines": {
        "node": ">=22"
    }
}
```

trace_explorer/src/Keyed.js

```
import { createMemo } from "solid-js";

const Actual = (props) => {
    const keyed_children = createMemo(() => {
        return props.keyed_children?.();
    });

    return <>
        {keyed_children()}
    </>;
};

export const Keyed = (props) => {
    return <>
        <For each={[props.key()]}>
            {(key) => <>
                <Actual key={key} keyed_children={() => props.children} />
            </>}
        </For>
    </>;
};
```

trace_explorer/src/api.tsx

```
import {
    createTRPCProxyClient,
    httpBatchLink,
    loggerLink,
} from "@trpc/client";
import { Router } from "~/server/api/root";

const base_url = (() => {
    if (typeof window !== "undefined") {
        return "";
    }

    if (process.env.NODE_ENV === "production") {
        return "https://example.com";
    }

    return `http://localhost:${process.env.PORT ?? 3000}`;
})();

export const api = createTRPCProxyClient<Router>({
```

```
    links: [
        loggerLink(),
        httpBatchLink({ url: `${base_url}/api/trpc` })
    ],
});
```

trace_explorer/src/app.tsx

```
import { router } from "./router";
import { RouterProvider } from "@tanstack/solid-router";

import "./global.css";

export default () => {
    return <>
        <RouterProvider router={router} />
    </>;
};
```

trace_explorer/src/components/Devtools.tsx

```
import { TanStackRouterDevtools } from "@tanstack/solid-router-devtools";

export default TanStackRouterDevtools;
```

trace_explorer/src/components/Trace.jsx

```
import { For, Show } from "solid-js";
import { obtain_component } from "../obtain_component.js";

export const perform_tree_building = (trace) => {
    const root = {
        entry: { type: "enter", depth: -1, position: 0, rule_name: "root" },
        children: [],
        index: -1,
    };

    const stack = [root];

    for (let i = 0; i < trace.length; i++) {
        const entry = trace[i];

        if (entry.type === "enter") {
            const node = {
                entry,
                children: [],
                index: i,
            };

            stack[stack.length - 1].children.push(node);
            stack.push(node);
        } else {
            const node = stack.pop();

            if (node) {
                node.exit_entry = entry;
            }
        }
    }

    return root;
```

```
};

export const render_path_beautifully = (path) => {
    return path.length === 0 ? "(root)" : path.map((seg) => seg.rule_name).join(" → ");
};

const stringify_path = (path) => {
    return JSON.stringify(path.map((seg) => seg.index));
};

export const find_deepest_lonely_path = (root) => {
    const path = [];
    let current = root;

    while (current.children.length === 1) {
        const child = current.children[0];

        path.push({
            index: 0,
            rule_name: child.entry.rule_name ?? "UNTITLED"
        });

        current = child;
    }

    return path;
};

export const Trace = obtain_component(({ self, props }) => {
    const { tree, expanded_paths, current_path, toggle_expand } = props;

    const render_node = (node, path, depth, parent_child_count) => {
        const is_lonely = parent_child_count === 1;
        const success = node.exit_entry?.success;
        const failed = success === false;
        const consumed = node.exit_entry?.consumed || 0;

        const render_snippet = () => {
            const snippet = node.entry.snippet;
            if (!snippet) return null;

            const display_snippet = snippet.substring(0, 50);

            if (consumed === 0) {
                return <span class="text-gray-500 text-[10px] ml-1">{display_snippet
    ↪ }...</span>;
            }

            const consumed_part = display_snippet.substring(0, consumed);
            const remaining_part = display_snippet.substring(consumed);

            return (
                <span class="text-[10px] ml-1">
                    <span class="bg-green-200 font-semibold">{consumed_part}</span>
                    <span class="text-gray-500">{remaining_part}...</span>
                </span>
            );
        };

        return (
            <div class="ml-2 border-l-2 border-gray-400 pl-1">
                <div
                    class={`py-0.5 ${is_lonely ? "" : "cursor-pointer hover:bg-gray
    ↪ -200"} ${stringify_path(current_path()) === stringify_path(path) ? "bg-yellow-100"
    ↪  : ""}`}
                    onClick={(e) => {
                        if (!is_lonely) {
                            e.stopPropagation();
                            toggle_expand(path);
                        }
```

```
                    }}
                >
                    <span class="inline-block w-3 text-center text-[10px]">
                        {node.children.length > 0 ? ((expanded_paths().has(
    ↪ stringify_path(path)) || is_lonely) ? "▽" : "▷") : "•"}
                    </span>
                    <span class={`mono text-xs ${failed ? "text-red-600" : success ===
    ↪ true ? "text-green-600" : ""}`}>
                        {node.entry.rule_name || "unknown"}
                    </span>
                    <span class="text-gray-600 text-[10px] ml-1">
                        @{node.entry.position}
                    </span>
                    <Show when={node.exit_entry}>
                        <span class="text-gray-600 text-[10px] ml-1">
                            +{consumed}
                        </span>
                    </Show>
                    <Show when={node.entry.snippet}>
                        {render_snippet()}
                    </Show>
                </div>

                <Show when={(expanded_paths().has(stringify_path(path)) || is_lonely) &&
    ↪  node.children.length > 0}>
                    <For each={node.children}>
                        {(child, index) => {
                            const child_path = [...path, { index: index(), rule_name:
    ↪ child.entry.rule_name ?? "UNTITLED" }];
                            return render_node(child, child_path, depth + 1, node.
    ↪ children.length);
                        }}
                    </For>
                </Show>
            </div>
        );
    };

    return () => (
        <div class="bg-white border-2 border-gray-400 p-4 shadow-md overflow-auto" style
    ↪ ="max-height: 600px;">
            <h2 class="text-lg font-bold mb-3 text-gray-700 border-b border-gray-400 pb
    ↪ -1">Trace Tree:</h2>
            <Show when={tree()}>
                <For each={tree().children}>
                    {(child, index) => {
                        const child_path = [{ index: index(), rule_name: child.entry.
    ↪ rule_name ?? "UNTITLED" }];
                        return render_node(child, child_path, 0, tree().children.length)
    ↪ ;
                    }}
                </For>
            </Show>
        </div>
    );
});
```

trace_explorer/src/components/TraceLog.jsx

```
import { obtain_component } from "../obtain_component.js";

const stringify_path = (path) => {
    return JSON.stringify(path.map((seg) => seg.index));
};

const generate_log = (node, path, depth, expanded_paths) => {
    const log = [];
```

```
    const indentation = " ".repeat(depth);
    const position = `@${node.entry.position}`;
    const rule_name = node.entry.rule_name ?? "UNTITLED";
    const snippet = node.entry.snippet ?? "";

    log.push(`${indentation}${position} ${rule_name} ${snippet}`);

    const path_str = stringify_path(path);
    const is_expanded = expanded_paths.has(path_str);

    if (is_expanded && node.children.length > 0) {
        for (let i = 0; i < node.children.length; i++) {
            const child = node.children[i];

            const child_path = [
                ...path,
                {
                    index: i,
                    rule_name: child.entry.rule_name ?? "UNTITLED",
                },
            ];

            const child_lines = generate_log(
                child,
                child_path,
                depth + 1,
                expanded_paths
            );

            log.push(...child_lines);
        }
    } else if (!is_expanded && node.children.length > 0) {
        log.push(`${indentation} (omitted)`);
    }

    if (node.exit_entry) {
        const is_success = node.exit_entry.success;
        const amt_consumed = node.exit_entry.consumed || 0;

        if (is_success === false) {
            log.push(`${indentation}${position} FAIL`);
        } else if (is_success === true) {
            log.push(`${indentation}${position} +${amt_consumed}`);
        }
    }

    return log;
};

const produce_log = (tree, expanded_paths) => {
    if (!tree) {
        return "";
    }

    const log = [];

    for (let i = 0; i < tree.children.length; i++) {
        const child = tree.children[i];

        const child_path = [
            {
                index: i,
                rule_name: child.entry.rule_name || "unknown",
            },
        ];

        const child_lines = generate_log(
            child,
            child_path,
            0,
```

```
            expanded_paths
        );

        log.push(...child_lines);
    }

    return log.join("\n");
};

export const TraceLog = obtain_component(({ self, props }) => {
    const { tree, expanded_paths } = props;

    const log_text = () => produce_log(tree(), expanded_paths());

    const copy_outcome = self.create_signal("idle");

    const perform_copy = async () => {
        const text = log_text();
        try {
            await navigator.clipboard.writeText(text);
            copy_outcome.set("copied");
            setTimeout(() => copy_outcome.set("idle"), 2000);
        } catch (error) {
            copy_outcome.set("failed");
            setTimeout(() => copy_outcome.set("idle"), 2000);
        }
    };

    return () => (
        <div class="bg-white border-2 border-gray-400 p-4 shadow-md">
            <div class="flex items-center justify-between mb-3 border-b border-gray-400
    ↪ pb-1">
                <h2 class="text-lg font-bold text-gray-700">Trace Log:</h2>
                <button
                    onClick={perform_copy}
                    classList={{
                        "bg-blue-500 hover:bg-blue-600 border-blue-700": copy_outcome()
    ↪ === "idle" || copy_outcome() === "copied",
                        "bg-red-500 hover:bg-red-600 border-red-700": copy_outcome() ===
    ↪  "failed",
                    }}
                    class="text-white font-bold py-1 px-3 border-2 text-xs"
                >
                    {copy_outcome() === "failed" ? "Failed!" : copy_outcome() === "
    ↪ copied" ? "Copied!" : "Copy"}
                </button>
            </div>
            <pre class="mono text-xs bg-gray-50 p-3 border border-gray-300 overflow-auto
    ↪ " style="max-height: 600px;">
                {log_text()}
            </pre>
        </div>
    );
});
```

trace_explorer/src/create_self.js

```
import {
    createResource,
    createEffect,
    getOwner,
    runWithOwner,
} from "solid-js";

import { create_signal } from "./create_signal.js";

export const create_self = (owner) => {
```

```
    owner ??= getOwner();

    if (owner === undefined) {
        console.error("Cannot create_self outside of a component. Consider passing an
    ↪ owner if it cannot be inferred, such as within an asynchronous workflow, or, if
    ↪ possible, run create_self early on while the owner can be inferred.");
    }

    const wrap = (primitive) => {
        return (...args) => {
            return runWithOwner(owner, () => {
                return primitive(...args);
            });
        };
    };

    return {
        owner,
        create_signal: wrap(create_signal),
        create_resource: wrap(createResource),
        create_effect: wrap(createEffect),
    };
};
```

trace_explorer/src/create_signal.js

```
import { createSignal } from "solid-js";

export const create_signal = (initial) => {
    const [signal, set_signal] = createSignal({
        retrieve: typeof initial === "function" ? initial : () => initial,
    });

    const result = () => {
        return signal().retrieve();
    };

    result.set = (value) => {
        set_signal({
            retrieve: typeof value === "function" ? value : () => value,
        });
    };

    return result;
};
```

trace_explorer/src/entry-client.tsx

```
// @refresh reload
import { mount, StartClientTanstack } from "@solidjs/start/client";

mount(() => <StartClientTanstack />, document.getElementById("app")!);
```

trace_explorer/src/entry-server.tsx

```
// @refresh reload
import { createHandler, FetchEvent, StartServer } from "@solidjs/start/server";
import { createMemoryHistory } from "@tanstack/solid-router";
import { router } from "./router";

const routerLoad = async (event: FetchEvent) => {
```

```
    const url = new URL(event.request.url);
    const path = url.href.replace(url.origin, "");

    router.update({
        history: createMemoryHistory({
            initialEntries: [path],
        }),
    });

    await router.load();
};

export default createHandler(
    () => <>
        <StartServer
            document={({ assets, children, scripts }) => (
                <html lang="en">
                    <head>
                        <meta charset="utf-8" />
                        <meta name="viewport" content="width=device-width, initial-scale
    ↪ =1" />
                        <link rel="icon" href="/favicon.ico" />
                        {assets}
                    </head>
                    <body>
                        <div id="app">{children}</div>
                        {scripts}
                    </body>
                </html>
            )}
        />
    </>,
    undefined,
    routerLoad
);
```

trace_explorer/src/global.css

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@100..900&family=
    ↪ JetBrains+Mono:ital,wght@0,100..800;1,100..800&display=swap');

@import "tailwindcss";

body {
    font-family: "Inter", sans-serif;
}

input,
code,
.mono {
    font-family: "JetBrains Mono", monospace;
}

.GLOBAL_loader_wrapper {
    width: 100%;
    height: 100%;
    cursor: progress;
    display: flex;
    align-items: center;
    justify-content: center;
    overflow: clip;
    opacity: 0;
    animation: animloaderwrap 0.6s ease-in-out forwards;
}

.GLOBAL_loader {
    width: 32px;
```

```
    height: 32px;
    display: inline-block;
    position: relative;
}

.GLOBAL_loader::after,
.GLOBAL_loader::before {
    content: '';
    box-sizing: border-box;
    width: 32px;
    height: 32px;
    border-radius: 50%;
    background: #7272ff;
    background: #a0a0ff;
    position: absolute;
    left: 0;
    top: 0;
    transform: scale(0);
    opacity: 1;
    animation: animloader 2s linear infinite;
}

.GLOBAL_loader::after {
    animation-delay: -1s;
}

@keyframes animloaderwrap {
    0% {
        opacity: 0;
    }

    100% {
        opacity: 1;
    }
}

@keyframes animloader {
    0% {
        transform: scale(0);
        opacity: 1;
    }

    100% {
        transform: scale(1);
        opacity: 0;
    }
}
```

trace_explorer/src/global.d.ts

```
/// <reference types="@solidjs/start/env" />
```

trace_explorer/src/obtain_component.js

```
import { createRoot, onCleanup, runWithOwner, Suspense } from "solid-js";
import { create_self } from "./create_self.js";
import { Dynamic } from "solid-js/web";
import { ClientOnly } from "@tanstack/solid-router";

const Error = () => <>
    <div class="GLOBAL_loader_wrapper">
        <p class="px-3 py-1 bg-red-200 border border-red-600 m-4"> Uncaught error </p>
    </div>
</>;
```

```
export const obtain_component = (render) => {
    const Component = ({ props }) => {
        const self = create_self();

        const [get_create_instance] = self.create_resource(() => {
            return runWithOwner(self.owner, async () => {
                try {
                    return await render({
                        ...props,
                        self,
                    });
                } catch (e) {
                    console.error("Uncaught error while rendering component", e);

                    return <>
                        <Error />
                    </>;
                }
            });
        });

        let dispose;

        onCleanup(() => dispose?.());

        return <>
            <Suspense fallback={<>
                <div class="GLOBAL_loader_wrapper">
                    <span class="GLOBAL_loader" />
                </div>
            </>}>
                {(() => {
                    const create_instance = get_create_instance();

                    if (create_instance === undefined) {
                        console.warn("Invalid instance factory", create_instance);
                        return <></>;
                    }

                    return createRoot((_dispose) => {
                        dispose?.();
                        dispose = _dispose;

                        try {
                            var instance = create_instance();
                        } catch (e) {
                            console.error("Uncaught error while obtaining component", e)
    ↪ ;

                            return <>
                                <Error />
                            </>;
                        }

                        return <>
                            <Dynamic component={() => instance} />
                        </>;
                    });
                })()}
            </Suspense >
        </>;
    };

    return (props) => <>
        <ClientOnly fallback={<>
            <div class="GLOBAL_loader_wrapper">
                <span class="GLOBAL_loader" />
            </div>
        </>}>
```

```
            <Component props={props} />
        </ClientOnly>
    </>;
};
```

trace_explorer/src/routeTree.gen.ts

```
/* eslint-disable */

// @ts-nocheck

// noinspection JSUnusedGlobalSymbols

// This file was automatically generated by TanStack Router.
// You should NOT make any changes in this file as it will be overwritten.
// Additionally, you should also exclude this file from your linter and/or formatter to
    ↪ prevent it from being checked or modified.

import { Route as rootRouteImport } from './routes/__root'
import { Route as IndexRouteImport } from './routes/index'

const IndexRoute = IndexRouteImport.update({
  id: '/',
  path: '/',
  getParentRoute: () => rootRouteImport,
} as any)

export interface FileRoutesByFullPath {
  '/': typeof IndexRoute
}
export interface FileRoutesByTo {
  '/': typeof IndexRoute
}
export interface FileRoutesById {
  __root__: typeof rootRouteImport
  '/': typeof IndexRoute
}
export interface FileRouteTypes {
  fileRoutesByFullPath: FileRoutesByFullPath
  fullPaths: '/'
  fileRoutesByTo: FileRoutesByTo
  to: '/'
  id: '__root__' | '/'
  fileRoutesById: FileRoutesById
}
export interface RootRouteChildren {
  IndexRoute: typeof IndexRoute
}

declare module '@tanstack/solid-router' {
  interface FileRoutesByPath {
    '/': {
      id: '/'
      path: '/'
      fullPath: '/'
      preLoaderRoute: typeof IndexRouteImport
      parentRoute: typeof rootRouteImport
    }
  }
}

const rootRouteChildren: RootRouteChildren = {
  IndexRoute: IndexRoute,
}
export const routeTree = rootRouteImport
  ._addFileChildren(rootRouteChildren)
  ._addFileTypes<FileRouteTypes>()
```

trace_explorer/src/router.tsx

```
import { createRouter as createTanstackSolidRouter } from "@tanstack/solid-router";
import { routeTree } from "./routeTree.gen";

export function createRouter() {
    const router = createTanstackSolidRouter({
        defaultErrorComponent: err => <div> {err.error.stack} </div>,
        routeTree,
        defaultPreload: "intent",
        defaultStaleTime: 5000,
        scrollRestoration: true
    });
    return router;
}

export const router = createRouter();

declare module "@tanstack/solid-router" {
    interface Register {
        router: ReturnType<typeof createRouter>;
    }
}
```

trace_explorer/src/routes/___root.tsx

```
import { Link, Outlet, createRootRoute } from "@tanstack/solid-router";

import { clientOnly } from "@solidjs/start";
import { Suspense } from "solid-js";

const Devtools = clientOnly(() => import("../components/Devtools"));

const RootComponent = () => {
    return (
        <>
            <Link to="/">Home</Link>
            <Suspense>
                <Outlet />
                <Devtools />
            </Suspense>
        </>
    );
};

export const Route = createRootRoute({
    component: RootComponent,
});
```

trace_explorer/src/routes/api/trpc/[trpc].ts

```
import type { APIEvent } from "@solidjs/start/server";
import { fetchRequestHandler } from "@trpc/server/adapters/fetch";
import { router } from "~/server/api/root";

const handler = (event: APIEvent) =>
  fetchRequestHandler({
    endpoint: "/api/trpc",
    req: event.request,
    router,
    createContext: () => event
  });

export const GET = handler;
```

```
export const POST = handler;
```

trace_explorer/src/routes/index.tsx

```
import { createFileRoute } from "@tanstack/solid-router";
import { For, Show } from "solid-js";
import { obtain_component } from "../obtain_component.js";
import { Trace, perform_tree_building, render_path_beautifully, find_deepest_lonely_path
    ↪ } from "../components/Trace.jsx";
import { TraceLog } from "../components/TraceLog.jsx";

const RouteComponent = obtain_component(({ self }) => {
    const tree = self.create_signal();
    const expanded_paths = self.create_signal(new Set());
    const curr_path = self.create_signal([]);
    const history = self.create_signal([]);
    const history_starred = self.create_signal([]);
    const paste_area = self.create_signal("");
    const dialog_ref = self.create_signal();
    const dropdown_button_ref = self.create_signal();

    const perform_upload_interaction = (event) => {
        const input = event.target as HTMLInputElement;
        const file = input.files?.[0];

        if (file) {
            const reader = new FileReader();

            reader.onload = (e) => {
                try {
                    const content = e.target?.result as string;
                    const parsed = JSON.parse(content);
                    const produced_tree = perform_tree_building(parsed);
                    tree.set(produced_tree);
                    expanded_paths.set(new Set());
                    history.set([]);
                    history_starred.set([]);

                    const initial_path = find_deepest_lonely_path(produced_tree);
                    curr_path.set(initial_path);
                } catch (error) {
                    alert("Could not parse trace very smoothly.");
                }
            };

            reader.readAsText(file);
        }
    };

    const perform_paste = () => {
        try {
            const parsed = JSON.parse(paste_area());
            const produced_tree = perform_tree_building(parsed);
            tree.set(produced_tree);
            expanded_paths.set(new Set());
            history.set([]);
            history_starred.set([]);
            paste_area.set("");

            const initial_path = find_deepest_lonely_path(produced_tree);
            curr_path.set(initial_path);
        } catch (error) {
            alert("Failed to smoothly parse trace.");
        }
    };

    const path_to_string = (path) => {
```

```
    return JSON.stringify(path.map((seg) => seg.index));
};

const update_expand_situation = (path) => {
    const new_expanded = new Set();

    for (let i = 0; i < path.length; i++) {
        const ancestor_path = path.slice(0, i + 1);
        const ancestor_path_str = path_to_string(ancestor_path);
        new_expanded.add(ancestor_path_str);
    }

    expanded_paths.set(new_expanded);
    curr_path.set([...path]);

    append_history(path);
};

const append_history = (path) => {
    const new_entry = {
        path: [...path],
        timestamp: Date.now(),
        starred: false,
    };

    const new_history = [new_entry, ...history()];

    if (new_history.length > 256) {
        new_history.splice(256);
    }

    history.set(new_history);
};

const jump_to_desired_path = (path) => {
    const new_expanded = new Set();

    for (let i = 0; i <= path.length; i++) {
        const sub_path = path.slice(0, i);
        new_expanded.add(path_to_string(sub_path));
    }

    expanded_paths.set(new_expanded);
    curr_path.set([...path]);

    append_history(path);
};

const toggle_star = (entry) => {
    if (entry.starred) {
        history_starred.set(history_starred().filter((e) => e !== entry));
        entry.starred = false;
    } else {
        entry.starred = true;
        history_starred.set([entry, ...history_starred()]);
    }

    history.set([...history()]);
};

return () => (
    <div class="min-h-screen bg-gray-100 p-4">
        <div class="max-w-7xl mx-auto">
            <div class="flex items-center justify-between mb-4 border-b-4 border-
↪ gray-600 pb-2">
                <h1 class="text-xl font-bold text-gray-800">
                    Essence C - Trace Explorer
                </h1>
                <Show when={tree()}>
                    <button
```

```
                        onClick={() => {
                            tree.set(null);
                        }}
                        class="bg-red-500 hover:bg-red-600 text-white font-bold py-1
↪  px-3 border-2 border-red-700 text-sm"
                    >
                        Clear Trace
                    </button>
                </Show>
            </div>

            <Show when={!tree()}>
                <div class="bg-white border-2 border-gray-400 p-6 mb-4 shadow-md">
                    <h2 class="text-xl font-bold mb-3 text-gray-700">Upload or Paste
↪  Trace</h2>

                    <div class="mb-4">
                        <label class="block mb-2 font-semibold text-gray-700">Upload
↪  File:</label>
                        <input
                            type="file"
                            accept=".json"
                            onChange={perform_upload_interaction}
                            class="border-2 border-gray-400 p-2 w-full"
                        />
                    </div>

                    <div class="mb-4">
                        <label class="block mb-2 font-semibold text-gray-700">Or
↪ Paste JSON:</label>
                        <textarea
                            value={paste_area()}
                            onInput={(e) => paste_area.set(e.currentTarget.value)}
                            class="border-2 border-gray-400 p-2 w-full h-32 mono
↪ text-sm"
                            placeholder="Paste trace JSON here..."
                        />
                        <button
                            onClick={perform_paste}
                            class="mt-2 bg-blue-500 hover:bg-blue-600 text-white
↪ font-bold py-2 px-4 border-2 border-blue-700"
                        >
                            Load Trace
                        </button>
                    </div>
                </div>
            </Show>

            <Show when={tree()}>
                <div class="bg-white border-2 border-gray-400 p-4 mb-4 shadow-md">
                    <div class="flex items-center gap-2 relative">
                        <div class="mono text-xs bg-gray-50 p-2 border border-gray
↪ -300 flex-1">
                            {render_path_beautifully(curr_path())}
                        </div>
                        <button
                            ref={(el) => dropdown_button_ref.set(el)}
                            onClick={() => {
                                const dialog = dialog_ref();
                                const button = dropdown_button_ref();
                                if (dialog && button) {
                                    if (dialog.open) {
                                        dialog.close();
                                    } else {
                                        const rect = button.getBoundingClientRect();
                                        const container_rect = button.closest(".bg-
↪ white")?.getBoundingClientRect();
                                        const container_width = container_rect?.
↪ width || window.innerWidth;
                                        dialog.style.position = "absolute";
```

```
                                        dialog.style.top = `${rect.bottom + 4}px`;
                                        dialog.style.left = `${container_rect?.left
↪ || 0}px`;
                                        dialog.style.width = `${container_width}px`;
                                        dialog.style.margin = "0";
                                        dialog.showModal();
                                    }
                                }
                            }}
                            class="bg-blue-500 hover:bg-blue-600 text-white font-
↪ bold py-2 px-3 border-2 border-blue-700 text-sm"
                        >
                            ▽
                        </button>
                    </div>

                    <dialog
                        ref={(el) => dialog_ref.set(el)}
                        onClick={(e) => {
                            const dialog = dialog_ref();
                            if (dialog && e.target === dialog) {
                                dialog.close();
                            }
                        }}
                        class="border-2 border-gray-400 p-4 bg-white shadow-lg
↪ backdrop:bg-transparent"
                        style={{
                            "max-height": "400px",
                        }}
                    >
                        <div class="overflow-auto" style="max-height: 350px;">
                            <Show when={history_starred().length > 0}>
                                <div class="mb-4">
                                    <h4 class="text-xs font-semibold mb-2 text-gray
↪ -600">Starred</h4>
                                    <For each={history_starred()}>
                                        {(entry) => (
                                            <div class="mb-2 p-2 bg-yellow-50 border
↪  border-yellow-400 hover:bg-yellow-100 cursor-pointer flex justify-between items-
↪ start">
                                                <div
                                                    onClick={() => {
                                                        jump_to_desired_path(entry.
↪ path);
                                                        dialog_ref()?.close();
                                                    }}
                                                    class="mono text-xs break-all
↪ flex-1"
                                                >
                                                    {render_path_beautifully(entry.
↪ path)}
                                                </div>
                                                <button
                                                    onClick={() => toggle_star(entry
↪ )}
                                                    class="ml-2 text-xs text-gray
↪ -500 hover:text-red-600 hover:font-bold transition-colors"
                                                >
                                                    X
                                                </button>
                                            </div>
                                        )}
                                    </For>
                                </div>
                            </Show>

                            <For each={history()}>
                                {(entry) => (
                                    <div class="mb-2 p-1.5 bg-gray-50 border border-
↪ gray-300 hover:bg-gray-100 flex justify-between items-start">
```

```
                                                <div
                                                    onClick={() => {
                                                        jump_to_desired_path(entry.path);
                                                        dialog_ref()?.close();
                                                    }}
                                                    class="mono text-xs break-all cursor-
    ↪ pointer flex-1"
                                                >
                                                    {render_path_beautifully(entry.path)}
                                                </div>
                                                <button
                                                    onClick={() => toggle_star(entry)}
                                                    class="ml-2 text-xs transition-colors"
                                                    classList={{
                                                        "text-gray-400 hover:text-yellow
    ↪ -500": !entry.starred,
                                                        "text-yellow-500 hover:text-red-600
    ↪ hover:font-bold": entry.starred
                                                    }}
                                                >
                                                    {entry.starred ? "X" : "★"}
                                                </button>
                                            </div>
                                        )}
                                    </For>
                                </div>
                            </dialog>
                        </div>

                        <Trace
                            props={{
                                tree,
                                expanded_paths,
                                current_path: curr_path,
                                toggle_expand: update_expand_situation,
                            }}
                        />

                        <div class="mt-4">
                            <TraceLog
                                props={{
                                    tree,
                                    expanded_paths,
                                }}
                            />
                        </div>
                    </Show>
                </div>
            </div>
        );
});

export const Route = createFileRoute("/")({
    component: RouteComponent
});
```

trace_explorer/src/server/api/root.ts

```
import { t } from "./t";

export const router = t.router({
    ping: t.procedure.query(() => "pong"),
});

export type Router = typeof router;
```

trace_explorer/src/server/api/t.ts

```
import { initTRPC } from "@trpc/server";

export const t = initTRPC.create();
```

trace_explorer/tsconfig.json

```
{
    "compilerOptions": {
        "target": "ESNext",
        "module": "ESNext",
        "moduleResolution": "bundler",
        "allowSyntheticDefaultImports": true,
        "esModuleInterop": true,
        "jsx": "preserve",
        "jsxImportSource": "solid-js",
        "allowJs": true,
        "strict": true,
        "noEmit": true,
        "types": [
            "vinxi/types/client"
        ],
        "isolatedModules": true,
        "paths": {
            "~/*": [
                "./src/*"
            ]
        }
    }
}
```

Appendix C – LLM Prompts

Appendix D – Interim Report

FYP Interim Report 2025-2026
Department of Computer Science

Student Name	David P. Callanan	**Student Number**	21444104
Supervisor	Dr. Phil Maguire	**ECTS Credits**	5
Project Title	Studying link-time optimizations in programming language development to facilitate the continuum of static and dynamic modules.		

Contents

1 Project Objectives **2**

2 Description of Work Completed **2**

2.1 Evidence of Work Completed 3

2.2 Literature Review 3

2.3 Use of GenAI and Tools 3

3 Future Work **4**

A Appendix: Screenshots **6**

B Appendix: Source Code **9**

C Appendix: LLM Prompts **75**

1 Project Objectives

The goal of this project is to develop a proof-of-concept programming language ("Essence C") that facilitates bespoke optimization techniques via novel language constructs. The specific objective is to research the continuum of static and dynamic modules in low-level software development. A module system will be designed to promote dispatch flexibility by abstracting away the underlying dispatch mechanism, enabling aggressive link-time optimizations when circumstances allow. The development of a prototype compiler will demonstrate the efficacy of the proposed solution.

Success will be evidenced by the following deliverables:

(1) A functional compiler that implements the designed module system;

(2) Documentation of the module system and other core language features;

(3) A simple IDE extension that provides syntax highlighting to the programmer;

(4) A body of research into the continuum of static and dynamic modules in software development, with kernel development as a case study;

(5) An implementation of link-time optimization techniques that leverage the designed module system;

(6) An evaluation of the effectiveness of these optimizations through the benchmarking of sample programs;

(7) A monorepo containing extensive git history, demonstrating ongoing development progress;

(8) A final written report detailing the research, design, implementation and evaluation of the project.

Some considerations will be out of scope for this project, such as the performance of compilation, the level of detail of compilation errors, and the implementation of standard library functions and advanced type system features.

2 Description of Work Completed

Preliminary work on syntax design and language parsing code has been completed, and background research into the LLVM toolchain has been conducted. AI assistance was also briefly assessed.

2.1 Evidence of Work Completed

Initially, a handful of pseudocode files were created to brainstorm syntax ideas for the language. Then, a simple VS Code extension was developed to provide syntax highlighting for these early syntax ideas to reduce eye strain. Next, work commenced on the compiler frontend, written in JavaScript. Some previously written code from personal projects was used (including a simple parsing library), justifying the decision to use JavaScript. Basic parse-tree generation was implemented for a portion of the preliminary syntax, and a custom "source scope" system was developed to handle the translation between filesystem resources and source references within the programming language. Then, the build system for the compiler backend (written in C++) was organized. A Makefile was used within a Dockerized environment, itself orchestrated by a Linux/WSL environment for best reproducibility. C++ was justified due to its first-class integration with the LLVM toolchain [1].

2.2 Literature Review

LLVM was found to be a practical compilation target for low-level programming languages due to (1) its target-independent code generation [2] and (2) its integration capabilities with existing programming languages thanks to ABI interoperability (including some recent improvements [3]). Furthermore, the extensive optimization passes shared amongst languages targetting LLVM [4] should make *Essence C* directly comparable with existing languages, allowing benchmarks to be solely influenced by the additional optimization techniques employed. For instance, C can be compiled with LLVM via the popular Clang compiler [5]. It was also noticed that LLVM has experimental support for inline assembly [6], preventing the layer of indirection of a separate compilation unit (useful in kernel development [7]). Additionally, LLVM benefits greatly from intermodular optimizations at link-time [8], because its intermediate representation (IR) preserves the necessary higher-level constructs. Critically, the LLVM API exposes a powerful interface to programatically manipulate existing LLVM IR files [9] [10]. It follows that inlined compatibility wrappers can be generated to minimize optimization barriers between compilation units originating from languages other than *Essence C*.

2.3 Use of GenAI and Tools

Two notable GenAI tools have been used to assist with the research and development of this project so far.

(1) The first is t3.chat, a web application that consolidates numerous large language

models into a streamlined chat interface [11]. The user can experiment with different models to understand which models are best suited for the task at hand. Both research and programming tasks have benefited from the use of this tool. The tool was primarily used to steer the work in the right direction, and to identify the concepts, libraries and resources through which further research could be conducted. For transparency, all LLM prompts have been included in **Appendix C**.

(2) The second is the tab-complete feature of GitHub Copilot. This tool has encouraged rapid development and iteration through its behaving as a glorified autocomplete. From experience, the tool's context is severely limited, forcing the developer to have genuine knowledge of the codebase and its architecture. This is beneficial for projects that require significant unique input from the developer, in contrast to rehash projects that benefit more directly from LLM assistance.

Some experimentation with GitHub Copilot agent mode is ongoing, with notable issues identified. Firstly, the development speed is impeded by slow model response times, lack of funds, and internet connectivity issues. Secondly, there is a great difficulty in referencing contributions made by this tool due to (1) tool calls that modify the codebase directly and (2) significant implicit pollution of the context window as the agent navigates the respository. As such, usage of this tool is temporarily suspended pending further assessment.

3 Future Work

The core of the project revolves around the design of the module system. The accompanying research will be a significant undertaking, including static vs dynamic dispatch, module instance tracking, constraint propagation, and the feasibility of IR interoperability persuant to the proposed optimization goals. This research will be tackled first, in parallel with the development of any necessary boilerplate code for the compiler backend. Then, some experimentation will be needed with LLVM IR generation before continuing with the compiler backend. Everything technical other than the compiler backend is expected to be completed by the end of January, leaving adequate time for the implementation and evaluation of the compiler proper.

References

[1] The LLVM Project. "Frequently asked questions (FAQ)," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/FAQ.html#in-what-language-is-llvm-written

[2] The LLVM Project. "The LLVM target-independent code generator," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/CodeGenerator.html

[3] N. Sreekumar. "GSoC 2025: Introducing an ABI lowering library," Accessed: Nov. 7, 2025. [Online]. Available: https://blog.llvm.org/posts/2025-08-25-abi-library/

[4] The LLVM Project. "LLVM's analysis and transform passes," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/Passes.html#domtree-dominator-tree-construction

[5] The Clang Project. "Clang: C language family frontend for LLVM," Accessed: Nov. 17, 2025. [Online]. Available: https://clang.llvm.org/

[6] The LLVM Project. "LLVM language reference manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LangRef.html#inline-assembler-expressions

[7] D. Roy. "GCC inline assembly and its usage in the linux kernel," Accessed: Nov. 6, 2025. [Online]. Available: https://dl.acm.org/doi/fullHtml/10.5555/3024956.3024958

[8] The LLVM Project. "LLVM link time optimization: Design and implementation," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LinkTimeOptimization.html

[9] The LLVM Project. "LLVM programmer's manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/ProgrammersManual.html

[10] The LLVM Project. "Writing an LLVM pass," Accessed: Nov. 2, 2025. [Online]. Available: https://llvm.org/docs/WritingAnLLVMNewPMPass.html

[11] DigiVirus. "Meet T3 chat AI: Your all-in-one, super-fast AI assistant," Accessed: Nov. 16, 2025. [Online]. Available: https://digivirus.in/meet-t3-chat-ai-your-all-in-one-super-fast-ai-assistant/